*Mini project report on*

# DISASTER RESPONSE & SMART RELIEF LOGISTICS ENGINE

*Submitted in partial fulfilment of the requirements for the award of degree of*

**Bachelor of Technology**

**in**

**Computer Science & Engineering**

**UE24CS351A – DBMS Project**

***Submitted by:***


| | |
|---|---|
| **Shreya C Rao** | **PES2UG24CS481** |
| **Shriya C Rao** | **PES2UG24CS496** |


under the guidance of


**Dr. Shilpa S**

Assistant Professor

PES University


**AUG - DEC 2026**


**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

FACULTY OF ENGINEERING

**PES UNIVERSITY**

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

# PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

# CERTIFICATE

*This is to certify that the mini project entitled*

## Disaster Response & Smart Relief Logistics Engine

*is a bonafide work carried out by*

| | |
|---|---|
| **Shreya C Rao** | **PES2UG24CS481** |
| **Shriya C Rao** | **PES2UG24CS496** |

In partial fulfilment for the completion of fifth semester DBMS Project (UE24CS351A) in the Program of Study - Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period AUG. 2026 – DEC. 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 5th semester academic requirements in respect of project work.

Signature
Dr. Shilpa S
Assistant Professor

# DECLARATION

We hereby declare that the DBMS Project entitled **Disaster Response & Smart Relief Logistics Engine** has been carried out by us under the guidance of **Dr. Shilpa S, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester AUG – DEC 2026.


| | |
|---|---|
| **Shreya C Rao** | **PES2UG24CS481** |
| **Shriya C Rao** | **PES2UG24CS496** |

# ACKNOWLEDGEMENT

We would like to express our gratitude to Dr. Shilpa S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE24CS351A - DBMS Project.

We take this opportunity to thank Dr. Sandesh B J, Professor, ChairPerson, Department of Computer Science and Engineering, PES University, for all the knowledge and support we have received from the department.

We are deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to us various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement we have received from our family and friends.

# ABSTRACT

During a flood, relief teams receive a large number of SOS messages at the same time, and most of the difficulty lies in coordinating them. The same family is often reported more than once, some shelters fill up while others nearby still have beds, and the same supplies can be promised to two places. This project, the Disaster Response & Smart Relief Logistics Engine (ReliefOps), is a web application that helps a relief team in Bengaluru record each SOS, match it to the nearest shelter with free beds, reserve relief supplies and dispatch them on vehicles.

Three databases are used, each for the kind of data it suits best. MySQL is the main database and stores shelters, stock, vehicles, users, SOS reports, reservations and dispatches. The relief rules are enforced inside MySQL using constraints, triggers, stored procedures, functions, transactions and views. MongoDB stores the victim intake forms, whose structure differs from household to household, and is used for aggregation reports. PostgreSQL with the pgvector extension stores sentence embeddings of the SOS text, so reports can be searched by meaning and duplicate reports can be detected.

The web application is built with Next.js and supports two roles, Relief Coordinator and Field Volunteer. Passwords are stored as bcrypt hashes, database credentials are read from environment variables, and each role connects to MySQL through its own restricted account. All 22 integration tests pass.

**Keywords: MySQL, MongoDB, pgvector, triggers, stored procedures, vector similarity search, Next.js, disaster relief.**

# TABLE OF CONTENTS


| Chapter No. | Title | Page No. |
|---|---|---|
| 1. | **INTRODUCTION** | **1** |
| 2. | **PROBLEM DEFINITION** | **3** |
| 3. | **ER MODEL** | **5** |
| 4. | **ER TO RELATIONAL MAPPING** | **7** |
| 5. | **DDL STATEMENTS** | **10** |
| 6. | **DML STATEMENTS** | **19** |
| 7. | **QUERIES (SIMPLE, UPDATE, DELETE, CORRELATED AND NESTED)** | **22** |
| 8. | **STORED PROCEDURES, FUNCTIONS AND TRIGGERS** | **31** |
| 9. | **ADVANCED DATABASE FEATURES** | **46** |
| 10. | **NOSQL DATABASE – MONGODB** | **55** |
| 11. | **VECTOR DATABASE – POSTGRESQL WITH PGVECTOR** | **66** |
| 12. | **FRONT END DEVELOPMENT** | **71** |
| 13. | **TESTING AND RESULTS** | **81** |
| 14. | **CONCLUSION AND FUTURE ENHANCEMENTS** | **84** |
| | **REFERENCES** | **85** |
| | **APPENDIX A: DEFINITIONS, ACRONYMS AND ABBREVIATIONS** | **86** |

# LIST OF TABLES


| Table No. | Title | Page No. |
|---|---|---|
| 1.1 | Technology stack | 2 |
| 2.1 | Functional requirements | 3 |
| 2.2 | Which database stores which data | 4 |
| 3.1 | Entities | 5 |
| 3.2 | Relationships | 5 |
| 4.1 | Primary and foreign keys | 9 |
| 8.1 | Stored functions | 31 |
| 8.2 | Stored procedures | 32 |
| 8.3 | Triggers and the rules they enforce | 39 |
| 9.1 | Advanced database features used in the project | 46 |
| 10.1 | Collections in the database relief_engine | 55 |
| 10.2 | Aggregation pipelines | 60 |
| 11.1 | Vector database design | 66 |
| 12.1 | Application roles | 71 |
| 12.2 | Main API routes | 71 |
| 13.1 | Test cases and results | 81 |
| A.1 | Definitions and abbreviations | 86 |

# LIST OF FIGURES


| Figure No. | Title | Page No. |
|---|---|---|
| 3.1 | ER diagram of the relief logistics database | 5 |
| 4.1 | Relational schema with primary keys and foreign keys | 8 |
| 5.1–5.8 | MySQL table structures (CREATE TABLE and DESC) | 10–15 |
| 5.9–5.10 | Indexes and table list | 15–16 |
| 5.11–5.15 | ALTER statements | 16–18 |
| 5.16 | ALTER and DROP demonstration | 18 |
| 6.1–6.3 | INSERT statements | 19–20 |
| 6.4–6.5 | Reservation and dispatch workflow | 20–21 |
| 6.6 | RESOURCE_ITEM rows with their usage notes | 21 |
| 7.1–7.5 | Simple queries with GROUP BY and aggregates (Q3, Q4, Q5, Q16, Q17) | 22–24 |
| 7.6–7.10 | Update operations | 24–25 |
| 7.11 | Delete operations | 25 |
| 7.12–7.13 | Correlated queries (Q8, Q9) | 26 |
| 7.14–7.15 | Nested queries (Q7, Q10) | 26 |
| 7.16–7.24 | Other queries: joins, window functions, CTE, CASE, UNION ALL and full-text search | 27–30 |
| 8.1–8.3 | Stored functions | 31–32 |
| 8.4–8.11 | Stored procedures | 33–36 |
| 8.12–8.16 | Smart matching demo (procedures, cursor and triggers) | 36–37 |
| 8.17–8.18 | Procedures added for the web application | 38–39 |
| 8.19–8.27 | Triggers on SHELTER, STOCK, SUPPLY_RESERVATION, DISTRIBUTION_LOG and VICTIM_SOS | 40–44 |
| 8.28–8.29 | Validation tests | 45 |
| 9.1–9.6 | Views and Q18 (using the views) | 47–49 |
| 9.7–9.9 | Explicit transactions (COMMIT, SAVEPOINT and ROLLBACK) | 49–50 |
| 9.10–9.11 | Indexing and EXPLAIN | 51 |
| 9.12–9.14 | Roles and privileges | 52–54 |
| 10.1 | One intake document with embedded household members, needs, details and timeline | 57 |
| 10.2 | Collections, required fields of the validator, and indexes | 58 |
| 10.3 | A document with an invalid disaster_type is rejected; a query on needs uses the multikey index | 60 |
| 10.4 | Most requested needs and people by age band | 63 |
| 10.5 | Vulnerable people per disaster type, and cases within 5 km of a shelter ($geoNear) | 65 |
| 11.1 | pgvector extension, the two tables and the structure of sos_embeddings | 68 |
| 11.2 | Reports most similar to SOS #4 and the relief items closest to it | 69 |
| 11.3 | Duplicate check combining vector similarity with distance and time filters | 70 |
| 11.4 | EXPLAIN: Index Scan using idx_sos_emb_hnsw | 70 |
| 12.1 | Login page | 72 |
| 12.2 | Coordinator dashboard: summary cards, priority queue (from v_open_sos_priority) and most requested needs (from MongoDB) | 72 |

| | | |
|---|---|---|
| 12.3 | Dashboard: shelter occupancy kept in sync by triggers, low stock, and the "needs attention" feed (UNION ALL) | 73 |
| 12.4 | SOS reports with status tabs, urgency filter and keyword (FULLTEXT) search | 73 |
| 12.5 | New SOS form with the duplicate-report warning from pgvector | 74 |
| 12.6 | SOS detail: intake form from MongoDB, recommended supplies and similar cases | 75 |
| 12.7 | Semantic search finds the fever case, while keyword search finds nothing | 76 |
| 12.8 | Inventory: restock, transfer between shelters, new item and low-stock alerts | 77 |
| 12.9 | Dispatch board: supplies ready to dispatch, vehicles on the road and history | 78 |
| 12.10 | Analytics page showing MongoDB aggregation results | 79 |
| 12.11 | A volunteer trying to open a coordinator page is blocked | 80 |
| 13.1 | npm test: all 22 integration tests pass | 82 |

# 1. INTRODUCTION

## 1.1 Overview

Bengaluru faces urban flooding almost every monsoon. When it happens, relief teams receive a large number of distress calls, mostly passed on by volunteers as short text messages. The shelters, relief stock and vehicles are spread across the city and handled by different people. Without one shared system, the same family may be reported several times while another waits, one shelter overflows while another nearby has empty beds, and supplies get promised from a shelter that has already given them away.

The Disaster Response & Smart Relief Logistics Engine (called ReliefOps in the application) solves this by keeping all relief data in one place. Every SOS is recorded once, ranked by urgency and matched to the nearest shelter that has free beds. Supplies are reserved so they cannot be double-booked, and a coordinator approves every vehicle dispatch. The important rules are written inside the database itself, so they hold even if someone bypasses the user interface.

## 1.2 Objectives

1. Design an ER model and a normalised relational schema in MySQL, and enforce the relief rules using keys, CHECK constraints, triggers, stored procedures and transactions.
2. Store the victim intake forms, which have a different structure for each household, in MongoDB and analyse them with aggregation pipelines.
3. Use PostgreSQL with pgvector to search SOS reports by meaning, warn about duplicate reports and recommend relief items.
4. Build a Next.js web application with two roles, Relief Coordinator and Field Volunteer, that works with all three databases through an API layer.
5. Follow basic security practice: hashed passwords, credentials in environment variables and a restricted database account for each role.

## 1.3 Scope

The system covers SOS intake and triage, shelter capacity and matching, stock at each shelter (restock, transfer and low-stock alerts), supply reservations, vehicle dispatch, semantic search, analytics and user management. Live GPS tracking, SMS intake, road-based routing and donation handling are outside the scope of this project. Distances are calculated as straight-line (Haversine) distances.

## 1.4 Technology Stack

**Table 1.1: Technology stack**

| Component | Technology | Used for |
|---|---|---|
| Relational database | MySQL 8 | Main data: shelters, stock, SOS, reservations, dispatches, users |
| Document database | MongoDB 7 | Victim intake forms and the activity (audit) log |
| Vector database | PostgreSQL 16 + pgvector | Embeddings of SOS text and relief items, similarity search |
| Embedding model | all-MiniLM-L6-v2 | Converts text into 384-number vectors, runs locally |
| Web application | Next.js 15, React, TypeScript | User interface and API route handlers |
| Security | bcryptjs, JWT, zod | Password hashing, login session, input validation |

# 2. PROBLEM DEFINITION

## 2.1 Problem Statement

During a flood, relief coordination usually fails for these reasons:

- **Duplicate and missed reports:** messages come from many volunteers in free text, so one household is reported many times while others are missed.
- **Wrong shelter:** people are sent to a shelter someone remembers instead of the nearest one that actually has free beds.
- **Stock errors:** two coordinators can promise the same last few kits, and stock can go negative on paper.
- **Unclear dispatch:** nobody knows which vehicle is free, and dispatches are not approved by a responsible person.

Keyword search also does not help much here. "Kid burning up" and "child with high fever" mean the same thing but share no words.

## 2.2 Proposed Solution

ReliefOps keeps one correct record of every SOS, shelter, stock line, reservation and dispatch in MySQL. Rules such as "stock can never go negative", "you cannot reserve more than what is available" and "only a coordinator can approve a dispatch" are enforced by triggers, procedures and constraints. The household details of each SOS are stored as one MongoDB document. The text of every SOS is also stored as a vector in PostgreSQL, so similar reports and useful supplies can be found even when the wording is different.

## 2.3 Functional Requirements

**Table 2.1: Functional requirements**

| ID | Requirement | Role |
|---|---|---|
| FR1 | Log in with email and password | All |
| FR2 | File an SOS with urgency, description, people count, location, needs and household details | All |
| FR3 | Show a warning if a similar SOS was filed nearby recently | All |
| FR4 | Match an SOS to the nearest active shelter with enough free beds | All |
| FR5 | Dashboard with priority queue, shelter occupancy, low stock and top needs | All |
| FR6 | Search SOS reports by meaning (semantic) and by keyword (full-text) | All |
| FR7 | Reserve and cancel supplies for an SOS | Coordinator |
| FR8 | Approve a dispatch on a vehicle and mark it delivered | Coordinator |

| | | |
|---|---|---|
| FR9 | Restock, transfer stock between shelters, add new items | Coordinator |
| FR10 | Auto-match all unmatched SOS, escalate old SOS, resolve an SOS | Coordinator |
| FR11 | View analytics, manage team accounts and see the audit log | Coordinator |

## 2.4 Database Choice and Justification

**Table 2.2: Which database stores which data**

| Data | Database | Reason |
|---|---|---|
| Shelters, stock, reservations, dispatches, vehicles, users, SOS | MySQL | The data is highly related and quantities must stay correct when many people use the system at once. This needs foreign keys, transactions, row locks, CHECK constraints and triggers. |
| Victim intake forms, audit log | MongoDB | Every household is different: 1 to 200 members, each with their own injuries and needs, and different fields for a flood and a landslide. One document per SOS fits this better than many mostly-empty tables. |
| Meaning of SOS text and item descriptions | PostgreSQL + pgvector | Similar meaning can be found with cosine similarity between embeddings. The vector sits in a normal SQL table, so it can be filtered by status, time and location in one query. |

MySQL is the main database (system of record). When a new SOS is filed, the application first saves the intake form in MongoDB, then calls the MySQL procedure sp_register_sos, links the MongoDB document to the new sos_id, and finally stores the embedding in pgvector. If the MySQL step fails, the MongoDB draft is deleted so the databases stay consistent.

# 3. ER MODEL

The ER model has eight entities. STOCK connects shelters and relief items (a many-to-many relationship), and SUPPLY_RESERVATION connects an SOS to the stock it draws from.

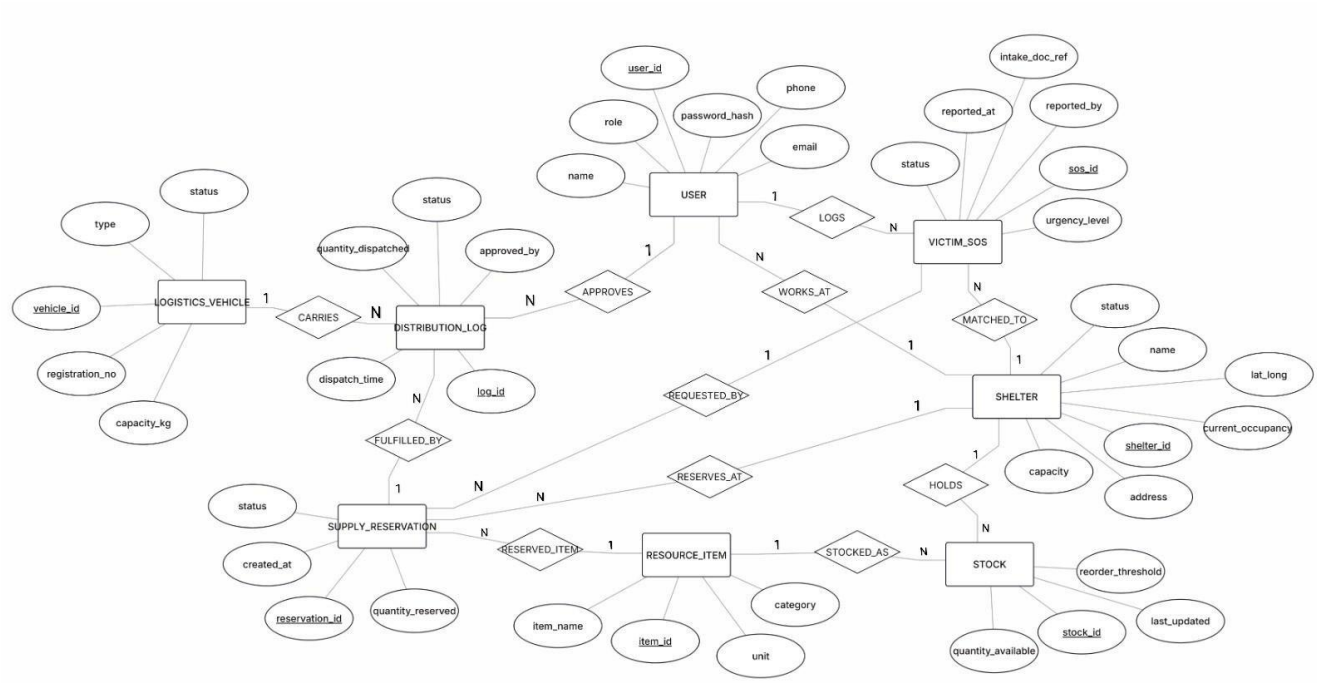



*Figure 3.1: ER diagram of the relief logistics database*

**Table 3.1: Entities**

| Entity | Description |
|---|---|
| SHELTER | Relief shelter with capacity, current occupancy, status and location |
| RESOURCE_ITEM | Relief item such as water, rice, blankets or medicines, with category and unit |
| LOGISTICS_VEHICLE | Truck, mini truck or ambulance with registration number and status |
| USER (APP_USER) | Relief Coordinator or Field Volunteer, optionally attached to a shelter |
| VICTIM_SOS | Distress report: urgency, status, people count, location, description |
| STOCK | Quantity of one item held at one shelter, with a reorder threshold |
| SUPPLY_RESERVATION | Quantity of an item reserved at a shelter for an SOS |
| DISTRIBUTION_LOG | A dispatch of reserved supplies on a vehicle, approved by a coordinator |

**Table 3.2: Relationships**

| Relationship | Between | Cardinality |
|---|---|---|
| WORKS_AT | USER – SHELTER | N : 1 |
| LOGS | USER – VICTIM_SOS | 1 : N |
| MATCHED_TO | VICTIM_SOS – SHELTER | N : 1 |
| HOLDS | SHELTER – RESOURCE_ITEM (through STOCK) | M : N |
| REQUESTED_BY | SUPPLY_RESERVATION – VICTIM_SOS | N : 1 |

| | | |
|---|---|---|
| RESERVES_FROM | SUPPLY_RESERVATION – STOCK | N : 1 |
| FULFILLED_BY | SUPPLY_RESERVATION – DISTRIBUTION_LOG | 1 : N |
| CARRIES / APPROVES | VEHICLE / USER – DISTRIBUTION_LOG | 1 : N |

# 4. ER TO RELATIONAL MAPPING

## 4.1 Steps of the Algorithm for the Chosen Problem

1. **Strong entities:** SHELTER, RESOURCE_ITEM, LOGISTICS_VEHICLE, USER and VICTIM_SOS each become a table with their simple attributes and a surrogate primary key (shelter_id, item_id and so on). USER is named APP_USER because USER is a reserved word in MySQL.
2. **Composite and derived attributes:** location is stored as one lat_long value. Free beds is a derived attribute, so it is not stored; it is calculated by the function fn_shelter_free_beds.
3. **1 : N relationships:** the primary key of the "one" side is added as a foreign key on the "many" side, for example APP_USER.shelter_id, VICTIM_SOS.reported_by and VICTIM_SOS.matched_shelter_id.
4. **M : N relationship:** SHELTER and RESOURCE_ITEM are linked through the new table STOCK with foreign keys item_id and shelter_id. The pair is UNIQUE and the relationship attributes (quantity_available, reorder_threshold) are kept in STOCK.
5. **Relationship with its own attributes:** a reservation has its own quantity, status and time, so it becomes the table SUPPLY_RESERVATION. Its composite foreign key (item_id, shelter_id) references STOCK, so a reservation can only use a stock line that exists. Each dispatch becomes a row in DISTRIBUTION_LOG.
6. **Referential actions and constraints:** CASCADE is used where the child has no meaning without the parent, RESTRICT where history must be kept, and SET NULL for optional links. Allowed values and ranges are added as CHECK constraints.

## 4.2 Complete Diagram of Relational Mapping

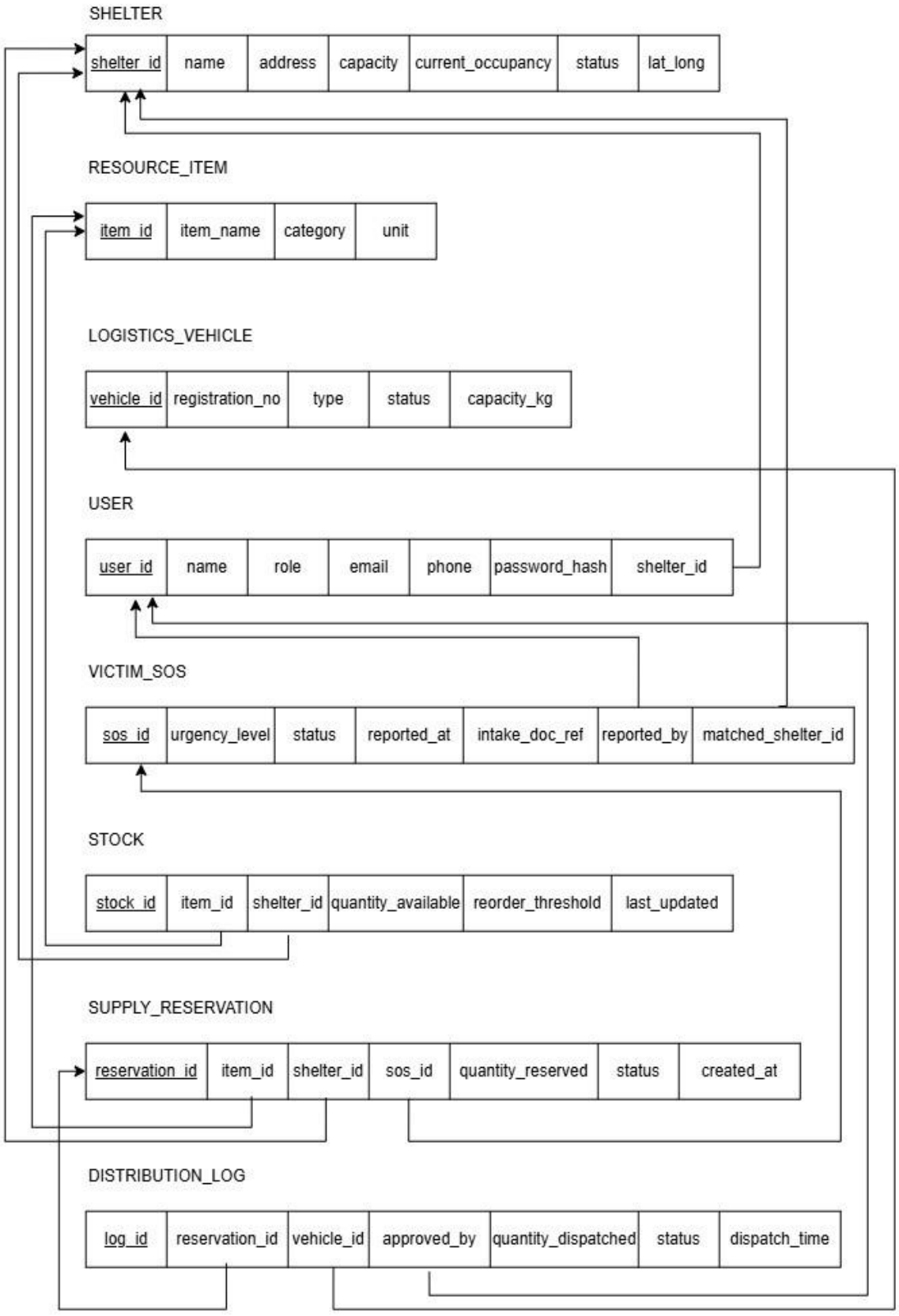



*Figure 4.1: Relational schema with primary keys and foreign keys*

**Table 4.1: Primary and foreign keys**

| Relation | Primary key | Foreign keys |
|---|---|---|
| SHELTER | shelter_id | — |
| RESOURCE_ITEM | item_id | — |
| LOGISTICS_VEHICLE | vehicle_id | — |
| APP_USER | user_id | shelter_id → SHELTER |
| VICTIM_SOS | sos_id | reported_by → APP_USER; matched_shelter_id → SHELTER |
| STOCK | stock_id | item_id → RESOURCE_ITEM; shelter_id → SHELTER |
| SUPPLY_RESERVATION | reservation_id | sos_id → VICTIM_SOS; (item_id, shelter_id) → STOCK |
| DISTRIBUTION_LOG | log_id | reservation_id → SUPPLY_RESERVATION; vehicle_id → LOGISTICS_VEHICLE; approved_by → APP_USER |

All tables are in Third Normal Form. Every non-key attribute depends only on the primary key, and names of shelters, items and users are referenced through foreign keys instead of being repeated.

# 5. DDL STATEMENTS

The schema is created in the database disaster_relief by 01_ddl.sql. Tables are created in foreign-key order: SHELTER, RESOURCE_ITEM and LOGISTICS_VEHICLE first, then APP_USER, VICTIM_SOS, STOCK, SUPPLY_RESERVATION and DISTRIBUTION_LOG.

## 5.1 Table Creation

Each table is created with its primary key, foreign keys and CHECK constraints (for example, occupancy can never exceed capacity and quantities can never be negative).

Tables:

```
mysql> CREATE DATABASE disaster_relief;
Query OK, 1 row affected (0.05 sec)

mysql> USE disaster_relief;
Database changed
mysql> SET default_storage_engine = InnoDB;
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE SHELTER (
    ->     shelter_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     name VARCHAR(120) NOT NULL,
    ->     address VARCHAR(255),
    ->     capacity INT NOT NULL CHECK (capacity >= 0),
    ->     current_occupancy INT NOT NULL DEFAULT 0 CHECK (current_occupancy >= 0),
    ->     status VARCHAR(20) NOT NULL DEFAULT 'active'
    ->            CHECK (status IN ('active','full','closed')),
    ->     lat_long VARCHAR(60),
    ->     CHECK (current_occupancy <= capacity)
    -> );
Query OK, 0 rows affected (0.08 sec)

mysql> DESC SHELTER;
+-------------------+--------------+------+-----+---------+----------------+
| Field             | Type         | Null | Key | Default | Extra          |
+-------------------+--------------+------+-----+---------+----------------+
| shelter_id        | int          | NO   | PRI | NULL    | auto_increment |
| name              | varchar(120) | NO   |     | NULL    |                |
| address           | varchar(255) | YES  |     | NULL    |                |
| capacity          | int          | NO   |     | NULL    |                |
| current_occupancy | int          | NO   |     | 0       |                |
| status            | varchar(20)  | NO   |     | active  |                |
| lat_long          | varchar(60)  | YES  |     | NULL    |                |
+-------------------+--------------+------+-----+---------+----------------+
7 rows in set (0.00 sec)
```

*Figure 5.1: SHELTER table structure*

```
mysql> CREATE TABLE RESOURCE_ITEM (
    ->     item_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     item_name VARCHAR(120) NOT NULL,
    ->     category VARCHAR(50) NOT NULL,
    ->     unit VARCHAR(20) NOT NULL
    -> );
Query OK, 0 rows affected (0.08 sec)

mysql> DESC RESOURCE_ITEM;
+-----------+--------------+------+-----+---------+----------------+
| Field     | Type         | Null | Key | Default | Extra          |
+-----------+--------------+------+-----+---------+----------------+
| item_id   | int          | NO   | PRI | NULL    | auto_increment |
| item_name | varchar(120) | NO   |     | NULL    |                |
| category  | varchar(50)  | NO   |     | NULL    |                |
| unit      | varchar(20)  | NO   |     | NULL    |                |
+-----------+--------------+------+-----+---------+----------------+
4 rows in set (0.00 sec)
```

*Figure 5.2: RESOURCE_ITEM table structure*

```
mysql> CREATE TABLE LOGISTICS_VEHICLE (
    ->     vehicle_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     registration_no VARCHAR(30) NOT NULL UNIQUE,
    ->     type VARCHAR(30) NOT NULL,
    ->     status VARCHAR(20) NOT NULL DEFAULT 'available'
    ->            CHECK (status IN ('available','in_transit','maintenance')),
    ->     capacity_kg DECIMAL(10,2) CHECK (capacity_kg > 0)
    -> );
Query OK, 0 rows affected (0.08 sec)

mysql> DESC LOGISTICS_VEHICLE;
+-----------------+--------------+------+-----+-----------+----------------+
| Field           | Type         | Null | Key | Default   | Extra          |
+-----------------+--------------+------+-----+-----------+----------------+
| vehicle_id      | int          | NO   | PRI | NULL      | auto_increment |
| registration_no | varchar(30)  | NO   | UNI | NULL      |                |
| type            | varchar(30)  | NO   |     | NULL      |                |
| status          | varchar(20)  | NO   |     | available |                |
| capacity_kg     | decimal(10,2)| YES  |     | NULL      |                |
+-----------------+--------------+------+-----+-----------+----------------+
5 rows in set (0.00 sec)
```

*Figure 5.3: LOGISTICS_VEHICLE table structure*

```
mysql> CREATE TABLE APP_USER (
    ->     user_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     name VARCHAR(120) NOT NULL,
    ->     role VARCHAR(30) NOT NULL
    ->         CHECK (role IN ('Relief Coordinator','Field Volunteer')),
    ->     email VARCHAR(120) NOT NULL UNIQUE,
    ->     phone VARCHAR(20),
    ->     password_hash VARCHAR(255) NOT NULL,
    ->     shelter_id INT,
    ->     CONSTRAINT fk_user_shelter
    ->         FOREIGN KEY (shelter_id) REFERENCES SHELTER(shelter_id)
    ->         ON DELETE SET NULL
    -> );
Query OK, 0 rows affected (0.17 sec)

mysql> DESC APP_USER;
+---------------+--------------+------+-----+---------+----------------+
| Field         | Type         | Null | Key | Default | Extra          |
+---------------+--------------+------+-----+---------+----------------+
| user_id       | int          | NO   | PRI | NULL    | auto_increment |
| name          | varchar(120) | NO   |     | NULL    |                |
| role          | varchar(30)  | NO   |     | NULL    |                |
| email         | varchar(120) | NO   | UNI | NULL    |                |
| phone         | varchar(20)  | YES  |     | NULL    |                |
| password_hash | varchar(255) | NO   |     | NULL    |                |
| shelter_id    | int          | YES  | MUL | NULL    |                |
+---------------+--------------+------+-----+---------+----------------+
7 rows in set (0.00 sec)
```

*Figure 5.4: APP_USER table structure*

```
mysql> CREATE TABLE VICTIM_SOS (
    ->     sos_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     urgency_level VARCHAR(20) NOT NULL
    ->                   CHECK (urgency_level IN ('low','medium','high','critical')),
    ->     status VARCHAR(20) NOT NULL DEFAULT 'unmatched'
    ->            CHECK (status IN ('unmatched','matched','resolved')),
    ->     reported_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->     intake_doc_ref VARCHAR(255),
    ->     reported_by INT NOT NULL,
    ->     matched_shelter_id INT,
    ->     CONSTRAINT fk_sos_reported_by
    ->         FOREIGN KEY (reported_by) REFERENCES APP_USER(user_id)
    ->         ON DELETE RESTRICT,
    ->     CONSTRAINT fk_sos_matched_shelter
    ->         FOREIGN KEY (matched_shelter_id) REFERENCES SHELTER(shelter_id)
    ->         ON DELETE SET NULL
    -> );
Query OK, 0 rows affected (0.12 sec)

mysql> DESC VICTIM_SOS;
+-------------------+-------------+------+-----+-------------------+-------------------+
| Field             | Type        | Null | Key | Default           | Extra             |
+-------------------+-------------+------+-----+-------------------+-------------------+
| sos_id            | int         | NO   | PRI | NULL              | auto_increment    |
| urgency_level     | varchar(20) | NO   |     | NULL              |                   |
| status            | varchar(20) | NO   |     | unmatched         |                   |
| reported_at       | datetime    | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| intake_doc_ref    | varchar(255)| YES  |     | NULL              |                   |
| reported_by       | int         | NO   | MUL | NULL              |                   |
| matched_shelter_id| int         | YES  | MUL | NULL              |                   |
+-------------------+-------------+------+-----+-------------------+-------------------+
7 rows in set (0.00 sec)
```

*Figure 5.5: VICTIM_SOS table structure*

```
mysql> CREATE TABLE STOCK (
    ->     stock_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     item_id INT NOT NULL,
    ->     shelter_id INT NOT NULL,
    ->     quantity_available INT NOT NULL DEFAULT 0 CHECK (quantity_available >= 0),
    ->     reorder_threshold INT NOT NULL DEFAULT 0 CHECK (reorder_threshold >= 0),
    ->     last_updated DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->     CONSTRAINT fk_stock_item
    ->         FOREIGN KEY (item_id) REFERENCES RESOURCE_ITEM(item_id)
    ->         ON DELETE CASCADE,
    ->     CONSTRAINT fk_stock_shelter
    ->         FOREIGN KEY (shelter_id) REFERENCES SHELTER(shelter_id)
    ->         ON DELETE CASCADE,
    ->     CONSTRAINT uq_stock_item_shelter UNIQUE (item_id, shelter_id)
    -> );
Query OK, 0 rows affected (0.10 sec)

mysql> DESC STOCK;
+--------------------+----------+------+-----+-------------------+-------------------+
| Field              | Type     | Null | Key | Default           | Extra             |
+--------------------+----------+------+-----+-------------------+-------------------+
| stock_id           | int      | NO   | PRI | NULL              | auto_increment    |
| item_id            | int      | NO   | MUL | NULL              |                   |
| shelter_id         | int      | NO   | MUL | NULL              |                   |
| quantity_available | int      | NO   |     | 0                 |                   |
| reorder_threshold  | int      | NO   |     | 0                 |                   |
| last_updated       | datetime | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+--------------------+----------+------+-----+-------------------+-------------------+
6 rows in set (0.00 sec)
```

*Figure 5.6: STOCK table structure*

```
mysql> CREATE TABLE SUPPLY_RESERVATION (
    ->     reservation_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     item_id INT NOT NULL,
    ->     shelter_id INT NOT NULL,
    ->     sos_id INT NOT NULL,
    ->     quantity_reserved INT NOT NULL CHECK (quantity_reserved > 0),
    ->     status VARCHAR(20) NOT NULL DEFAULT 'pending'
    ->            CHECK (status IN ('pending','committed','cancelled','fulfilled')),
    ->     created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->     CONSTRAINT fk_reservation_item
    ->         FOREIGN KEY (item_id) REFERENCES RESOURCE_ITEM(item_id)
    ->         ON DELETE RESTRICT,
    ->     CONSTRAINT fk_reservation_shelter
    ->         FOREIGN KEY (shelter_id) REFERENCES SHELTER(shelter_id)
    ->         ON DELETE RESTRICT,
    ->     CONSTRAINT fk_reservation_sos
    ->         FOREIGN KEY (sos_id) REFERENCES VICTIM_SOS(sos_id)
    ->         ON DELETE CASCADE,
    ->     CONSTRAINT fk_reservation_stock
    ->         FOREIGN KEY (item_id, shelter_id) REFERENCES STOCK(item_id, shelter_id)
    ->         ON DELETE RESTRICT
    -> );
Query OK, 0 rows affected (0.12 sec)

mysql> DESC SUPPLY_RESERVATION;
+------------------+-------------+------+-----+-------------------+-------------------+
| Field            | Type        | Null | Key | Default           | Extra             |
+------------------+-------------+------+-----+-------------------+-------------------+
| reservation_id   | int         | NO   | PRI | NULL              | auto_increment    |
| item_id          | int         | NO   | MUL | NULL              |                   |
| shelter_id       | int         | NO   | MUL | NULL              |                   |
| sos_id           | int         | NO   | MUL | NULL              |                   |
| quantity_reserved | int        | NO   |     | NULL              |                   |
| status           | varchar(20) | NO   |     | pending           |                   |
| created_at       | datetime    | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+------------------+-------------+------+-----+-------------------+-------------------+
7 rows in set (0.00 sec)
```

*Figure 5.7: SUPPLY_RESERVATION table structure*

```
mysql> CREATE TABLE DISTRIBUTION_LOG (
    ->     log_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     reservation_id INT NOT NULL,
    ->     vehicle_id INT,
    ->     approved_by INT NOT NULL,
    ->     quantity_dispatched INT NOT NULL CHECK (quantity_dispatched > 0),
    ->     status VARCHAR(20) NOT NULL DEFAULT 'dispatched'
    ->            CHECK (status IN ('dispatched','delivered','cancelled')),
    ->     dispatch_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->     CONSTRAINT fk_log_reservation
    ->         FOREIGN KEY (reservation_id) REFERENCES SUPPLY_RESERVATION(reservation_id)
    ->         ON DELETE RESTRICT,
    ->     CONSTRAINT fk_log_vehicle
    ->         FOREIGN KEY (vehicle_id) REFERENCES LOGISTICS_VEHICLE(vehicle_id)
    ->         ON DELETE SET NULL,
    ->     CONSTRAINT fk_log_approved_by
    ->         FOREIGN KEY (approved_by) REFERENCES APP_USER(user_id)
    ->         ON DELETE RESTRICT
    -> );
Query OK, 0 rows affected (0.13 sec)

mysql> DESC DISTRIBUTION_LOG;
+--------------------+-------------+------+-----+-------------------+-------------------+
| Field              | Type        | Null | Key | Default           | Extra             |
+--------------------+-------------+------+-----+-------------------+-------------------+
| log_id             | int         | NO   | PRI | NULL              | auto_increment    |
| reservation_id     | int         | NO   | MUL | NULL              |                   |
| vehicle_id         | int         | YES  | MUL | NULL              |                   |
| approved_by        | int         | NO   | MUL | NULL              |                   |
| quantity_dispatched | int        | NO   |     | NULL              |                   |
| status             | varchar(20) | NO   |     | dispatched        |                   |
| dispatch_time      | datetime    | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+--------------------+-------------+------+-----+-------------------+-------------------+
7 rows in set (0.00 sec)
```

*Figure 5.8: DISTRIBUTION_LOG table structure*

## 5.2 Indexes and Table List

Secondary indexes are created on the foreign-key and filter columns that the application searches most often.

```
mysql> CREATE INDEX idx_stock_shelter ON STOCK(shelter_id);
Query OK, 0 rows affected (0.11 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE INDEX idx_reservation_status ON SUPPLY_RESERVATION(status);
Query OK, 0 rows affected (0.08 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE INDEX idx_sos_status ON VICTIM_SOS(status);
Query OK, 0 rows affected (0.08 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE INDEX idx_distribution_dispatch_time ON DISTRIBUTION_LOG(dispatch_time);
Query OK, 0 rows affected (0.07 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

*Figure 5.9: Secondary indexes on STOCK, SUPPLY_RESERVATION, VICTIM_SOS and DISTRIBUTION_LOG*

```
mysql> SHOW TABLES;
+----------------------------+
| Tables_in_disaster_relief |
+----------------------------+
| app_user                   |
| distribution_log           |
| logistics_vehicle          |
| resource_item              |
| shelter                    |
| stock                      |
| supply_reservation         |
| victim_sos                 |
+----------------------------+
8 rows in set (0.04 sec)
```

*Figure 5.10: SHOW TABLES: the eight tables of disaster_relief*

## 5.3 ALTER Statements

ALTER TABLE is used to add the description, people count and location of an SOS, to add email and phone checks on APP_USER using REGEXP, and to add FULLTEXT indexes for keyword search.

```
mysql> USE disaster_relief;
Database changed
mysql> SET SQL_SAFE_UPDATES = 0;
Query OK, 0 rows affected (0.03 sec)

mysql> ALTER TABLE VICTIM_SOS
    ->     ADD COLUMN description TEXT AFTER intake_doc_ref,
    ->     ADD COLUMN people_count INT NOT NULL DEFAULT 1 AFTER description,
    ->     ADD COLUMN lat_long VARCHAR(60) AFTER people_count,
    ->     ADD CONSTRAINT chk_sos_people CHECK (people_count > 0);
Query OK, 0 rows affected (0.20 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> DESC VICTIM_SOS;
+-------------------+-------------+------+-----+-------------------+-------------------+
| Field             | Type        | Null | Key | Default           | Extra             |
+-------------------+-------------+------+-----+-------------------+-------------------+
| sos_id            | int         | NO   | PRI | NULL              | auto_increment    |
| urgency_level     | varchar(20) | NO   |     | NULL              |                   |
| status            | varchar(20) | NO   | MUL | unmatched         |                   |
| reported_at       | datetime    | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| intake_doc_ref    | varchar(255)| YES  |     | NULL              |                   |
| description       | text        | YES  |     | NULL              |                   |
| people_count      | int         | NO   |     | 1                 |                   |
| lat_long          | varchar(60) | YES  |     | NULL              |                   |
| reported_by       | int         | NO   | MUL | NULL              |                   |
| matched_shelter_id| int         | YES  | MUL | NULL              |                   |
+-------------------+-------------+------+-----+-------------------+-------------------+
10 rows in set (0.04 sec)
```

*Figure 5.11: ALTER TABLE VICTIM_SOS: description, people_count and lat_long added*

```
mysql> ALTER TABLE APP_USER
    ->     ADD CONSTRAINT chk_user_email
    ->         CHECK (email REGEXP '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$'),
    ->     ADD CONSTRAINT chk_user_phone
    ->         CHECK (phone IS NULL OR phone REGEXP '^[0-9]{10}$');
Query OK, 0 rows affected (0.15 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

*Figure 5.12: ALTER TABLE APP_USER: REGEXP CHECK constraints on email and phone*

```
mysql> ALTER TABLE VICTIM_SOS ADD FULLTEXT INDEX ft_sos_description (description);
Query OK, 0 rows affected, 1 warning (0.33 sec)
Records: 0  Duplicates: 0  Warnings: 1

mysql> ALTER TABLE RESOURCE_ITEM ADD FULLTEXT INDEX ft_item_name_cat (item_name, category);
Query OK, 0 rows affected, 1 warning (0.26 sec)
Records: 0  Duplicates: 0  Warnings: 1

mysql> CREATE INDEX idx_sos_triage ON VICTIM_SOS(status, urgency_level, reported_at);
Query OK, 0 rows affected (0.08 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE INDEX idx_shelter_status ON SHELTER(status);
Query OK, 0 rows affected (0.09 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

*Figure 5.13: Full-text indexes and the composite triage index*

```
mysql> CREATE TABLE STOCK_ALERT (
    ->     alert_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     stock_id INT NOT NULL,
    ->     quantity_at_alert INT NOT NULL,
    ->     reorder_threshold INT NOT NULL,
    ->     alert_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->     is_resolved BOOLEAN  NOT NULL DEFAULT FALSE,
    ->     CONSTRAINT fk_alert_stock
    ->         FOREIGN KEY (stock_id) REFERENCES STOCK(stock_id)
    ->         ON DELETE CASCADE
    -> );
Query OK, 0 rows affected (0.09 sec)
```

*Figure 5.14: CREATE TABLE STOCK_ALERT*

For the application, one more column was added to RESOURCE_ITEM. The usage notes describe when an item is needed, and they are the text used to create the item embeddings in pgvector (Chapter 11).

```
ALTER TABLE RESOURCE_ITEM
  ADD COLUMN usage_notes VARCHAR(500) NULL AFTER unit;
```

```
mysql> DESC RESOURCE_ITEM;
+-------------+--------------+------+-----+---------+----------------+
| Field       | Type         | Null | Key | Default | Extra          |
+-------------+--------------+------+-----+---------+----------------+
| item_id     | int          | NO   | PRI | NULL    | auto_increment |
| item_name   | varchar(120) | NO   | MUL | NULL    |                |
| category    | varchar(50)  | NO   |     | NULL    |                |
| unit        | varchar(20)  | NO   |     | NULL    |                |
| usage_notes | varchar(500) | YES  |     | NULL    |                |
+-------------+--------------+------+-----+---------+----------------+
5 rows in set (0.01 sec)
```

*Figure 5.15: RESOURCE_ITEM after adding usage_notes*

## 5.4 ALTER and DROP Demonstration

```
mysql> ALTER TABLE LOGISTICS_VEHICLE ADD COLUMN driver_name VARCHAR(80) NOT NULL DEFAULT 'Unassigned';
Query OK, 0 rows affected (0.09 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> ALTER TABLE LOGISTICS_VEHICLE MODIFY COLUMN driver_name VARCHAR(120) NOT NULL DEFAULT 'TBD';
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> ALTER TABLE LOGISTICS_VEHICLE DROP COLUMN driver_name;
Query OK, 0 rows affected (0.04 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> CREATE TABLE tmp_scratch (id INT PRIMARY KEY, note VARCHAR(50));
Query OK, 0 rows affected (0.02 sec)

mysql> ALTER TABLE tmp_scratch RENAME TO tmp_scratch_old;
Query OK, 0 rows affected (0.02 sec)

mysql> DROP TABLE tmp_scratch_old;
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DROP INDEX idx_shelter_status ON SHELTER;                -- optional clean-up demo
Query OK, 0 rows affected (0.04 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE INDEX idx_shelter_status ON SHELTER(status);      -- re-create it
Query OK, 0 rows affected (0.06 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> SET SQL_SAFE_UPDATES = 1;
Query OK, 0 rows affected (0.00 sec)
```

*Figure 5.16: DDL demo: ADD, MODIFY and DROP COLUMN, RENAME and DROP TABLE, DROP and re-create INDEX*

# 6. DML STATEMENTS

Sample data for Bengaluru is inserted by 02_dml_advanced.sql: 5 shelters, 8 relief items, 5 vehicles, 6 users (2 coordinators and 4 volunteers), stock for every shelter and 8 SOS reports with different urgency levels. Reservations and dispatches are created by calling the stored procedures, so the triggers update stock and status exactly as they would in real use.

## 6.1 INSERT Statements

```
mysql> INSERT INTO SHELTER (name, address, capacity, current_occupancy, status, lat_long) VALUES
    -> ('Banashankari Community Hall', 'Banashankari 3rd Stage, Bengaluru', 150, 40, 'active', '12.9255,77.5468'),
    -> ('Koramangala Indoor Stadium', 'Koramangala 4th Block, Bengaluru', 300, 120, 'active', '12.9352,77.6245'),
    -> ('Whitefield Government School', 'ITPL Main Road, Whitefield', 120, 120, 'full', '12.9698,77.7500'),
    -> ('Yelahanka Relief Centre', 'Yelahanka New Town, Bengaluru', 200, 50, 'active', '13.1007,77.5963'),
    -> ('Jayanagar Municipal Hall', 'Jayanagar 4th Block, Bengaluru', 80, 10, 'active', '12.9250,77.5938');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO RESOURCE_ITEM (item_name, category, unit) VALUES
    -> ('Drinking Water', 'Water', 'litre'),
    -> ('Rice', 'Food', 'kg'),
    -> ('Blankets', 'Shelter', 'piece'),
    -> ('First Aid Kit', 'Medical', 'kit'),
    -> ('Paracetamol Tablets','Medical','strip'),
    -> ('Baby Food', 'Food', 'pack'),
    -> ('Tarpaulin Sheets', 'Shelter', 'piece'),
    -> ('Sanitary Kit', 'Hygiene', 'kit');
Query OK, 8 rows affected (0.01 sec)
Records: 8  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO LOGISTICS_VEHICLE (registration_no, type, status, capacity_kg) VALUES
    -> ('KA-01-AB-1234', 'Truck', 'available', 3000.00),
    -> ('KA-05-CD-5678', 'Mini Truck', 'available', 1200.00),
    -> ('KA-03-EF-9012', 'Ambulance', 'available', 400.00),
    -> ('KA-02-GH-3456', 'Tempo', 'maintenance', 800.00),
    -> ('KA-04-IJ-7890', 'Truck', 'in_transit', 2500.00);
Query OK, 5 rows affected (0.05 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

*Figure 6.1: INSERT into SHELTER, RESOURCE_ITEM and LOGISTICS_VEHICLE*

```
mysql> INSERT INTO APP_USER (name, role, email, phone, password_hash, shelter_id) VALUES
    -> ('Ananya Sharma', 'Relief Coordinator', 'ananya.sharma@relief.org', '9876500001', '$2b$10$placeholderHashAnanya00000000000000000000000000000000', 1)
,
    -> ('Rohit Verma', 'Relief Coordinator', 'rohit.verma@relief.org', '9876500002', '$2b$10$placeholderHashRohit00000000000000000000000000000000', 2),
    -> ('Meera Iyer', 'Field Volunteer', 'meera.iyer@relief.org', '9876500003', '$2b$10$placeholderHashMeera00000000000000000000000000000000', 1),
    -> ('Karthik Nair', 'Field Volunteer', 'karthik.nair@relief.org', '9876500004', '$2b$10$placeholderHashKarthik00000000000000000000000000000000', 2),
    -> ('Farhan Ali', 'Field Volunteer', 'farhan.ali@relief.org', '9876500005', '$2b$10$placeholderHashFarhan00000000000000000000000000000000', 4),
    -> ('Divya Reddy', 'Field Volunteer', 'divya.reddy@relief.org', '9876500006', '$2b$10$placeholderHashDivya00000000000000000000000000000000', 3);
Query OK, 6 rows affected (0.03 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO STOCK (item_id, shelter_id, quantity_available, reorder_threshold) VALUES
    -> -- Shelter 1
    -> (1,1,500,100),(2,1,200,50),(3,1,80,30),(4,1,25,10),(5,1,60,20),
    -> -- Shelter 2
    -> (1,2,800,150),(2,2,350,80),(3,2,150,40),(4,2,40,15),(5,2,15,20),(6,2,30,10),(8,2,100,30),
    -> -- Shelter 3
    -> (1,3,300,100),(3,3,60,20),(7,3,50,20),
    -> -- Shelter 4
    -> (1,4,600,120),(2,4,400,100),(3,4,200,50),(4,4,30,10),(7,4,90,30),(8,4,12,25),
    -> -- Shelter 5
    -> (1,5,100,50),(3,5,25,30);
Query OK, 23 rows affected (0.01 sec)
Records: 23  Duplicates: 0  Warnings: 0
```

*Figure 6.2: INSERT into APP_USER and STOCK*

```
mysql>
mysql> INSERT INTO VICTIM_SOS
    ->  (urgency_level, status, reported_at, intake_doc_ref, description, people_count, lat_long, reported_by, matched_shelter_id)
    -> VALUES
    -> ('critical','matched', NOW() - INTERVAL 3  HOUR, 'intake/sos_001.pdf',
    ->  'Family of five stranded on rooftop as flood water rises; infant needs baby food and fever medicines', 5, '12.9330,77.6200', 3, 2),
    -> ('high', 'matched', NOW() - INTERVAL 5  HOUR, 'intake/sos_002.pdf',
    ->  'Elderly couple with limited mobility need drinking water and regular medicines after flooding', 2, '12.9260,77.5500', 3, 1),
    -> ('medium', 'unmatched', NOW() - INTERVAL 30 HOUR, NULL,
    ->  'Landslide blocked the village road; residents need tarpaulin sheets and blankets', 6, '13.0500,77.6000', 4, NULL),
    -> ('critical','unmatched', NOW() - INTERVAL 1  HOUR, 'intake/sos_004.pdf',
    ->  'Child with high fever and leg injury needs first aid kit and an ambulance', 3, '13.0900,77.5900', 5, NULL),
    -> ('low', 'unmatched', NOW() - INTERVAL 50 HOUR, NULL,
    ->  'Need drinking water and rice for two days', 2, '12.9300,77.5900', 3, NULL),
    -> ('high', 'unmatched', NOW() - INTERVAL 14 HOUR, 'intake/sos_006.pdf',
    ->  'Flooded apartment block with ten residents needing shelter and blankets', 10, '12.9300,77.6200', 4, NULL),
    -> ('medium', 'resolved', NOW() - INTERVAL 40 HOUR, NULL,
    ->  'Power outage at hospital annex; sanitary kits needed', 8, '13.1000,77.5900', 5, 4),
    -> ('high', 'unmatched', NOW() - INTERVAL 2  HOUR, 'intake/sos_008.pdf',
    ->  'Pregnant woman and three children trapped by flood; urgent medical support needed', 4, '12.9700,77.7400', 6, NULL);
Query OK, 8 rows affected (0.04 sec)
Records: 8  Duplicates: 0  Warnings: 0
```

*Figure 6.3: INSERT into VICTIM_SOS*

## 6.2 Reservations and Dispatches through Procedures

```
mysql> -- Reservations (stock is deducted by trigger; low-stock alerts get raised)
mysql> CALL sp_reserve_supply(1, 6, 2,  6, @r1);   -- baby food for SOS 1
Query OK, 0 rows affected (0.06 sec)

mysql> CALL sp_reserve_supply(1, 5, 2,  5, @r2);   -- paracetamol (15 -> 10, alert)
Query OK, 0 rows affected (0.04 sec)

mysql> CALL sp_reserve_supply(2, 1, 1, 30, @r3);   -- water for SOS 2
Query OK, 0 rows affected (0.01 sec)

mysql> CALL sp_reserve_supply(3, 7, 4, 40, @r4);   -- tarpaulins for SOS 3
Query OK, 0 rows affected (0.01 sec)

mysql> CALL sp_reserve_supply(3, 3, 4, 30, @r5);   -- blankets for SOS 3
Query OK, 0 rows affected (0.00 sec)

mysql> CALL sp_reserve_supply(4, 4, 4,  2, @r6);   -- first-aid kits for SOS 4
Query OK, 0 rows affected (0.01 sec)

mysql> CALL sp_reserve_supply(6, 8, 4, 10, @r7);   -- sanitary kits (12 -> 2, alert)
Query OK, 0 rows affected (0.01 sec)

mysql> CALL sp_reserve_supply(6, 3, 2, 20, @r8);   -- blankets for SOS 6
Query OK, 0 rows affected (0.05 sec)
```

*Figure 6.4: Reserving supplies with sp_reserve_supply*

```
mysql> -- Coordinators approve dispatches (vehicle -> in_transit, reservation -> committed)
mysql> CALL sp_approve_dispatch(@r1, 3, 2,  6, @l1);   -- ambulance, Rohit
Query OK, 0 rows affected (0.05 sec)

mysql> CALL sp_approve_dispatch(@r3, 1, 1, 30, @l2);   -- truck, Ananya
Query OK, 0 rows affected (0.01 sec)

mysql> CALL sp_approve_dispatch(@r4, 2, 2, 40, @l3);   -- mini truck, Rohit
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> -- First delivery completes (reservation -> fulfilled, ambulance -> available)
mysql> CALL sp_complete_delivery(@l1);
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> -- Direct INSERT also works (same triggers apply)
mysql> INSERT INTO DISTRIBUTION_LOG (reservation_id, vehicle_id, approved_by, quantity_dispatched)
    -> VALUES (@r2, 3, 2, 5);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> -- Restock through the procedure (resolves the paracetamol alert once above threshold)
mysql> CALL sp_restock(5, 2, 40);
Query OK, 0 rows affected (0.05 sec)
```

*Figure 6.5: Approving dispatches, completing a delivery and restocking*

## 6.3 Usage Notes for Relief Items

The usage notes are filled with a single UPDATE that uses a CASE expression on the item name:

```
UPDATE RESOURCE_ITEM SET usage_notes = CASE item_name
  WHEN 'Drinking Water'      THEN 'Safe drinking water for people cut off by floods,
...'
  WHEN 'Paracetamol Tablets' THEN 'Medicine for fever, high temperature, headache,
...'
  WHEN 'Baby Food'           THEN 'Nutrition for infants, babies and toddlers, ...'
  ...
  ELSE usage_notes END;
```

```
mysql> SELECT item_id, item_name, usage_notes FROM RESOURCE_ITEM;
+---------+--------------------+-----------------------------------------------------------------------------------------------------+
| item_id | item_name          | usage_notes                                                                                         |
+---------+--------------------+-----------------------------------------------------------------------------------------------------+
|       1 | Drinking Water     | Safe drinking water for people cut off by floods, thirst, dehydration, contaminated water supply, heat |
|       2 | Rice               | Staple food ration for families with no food, hunger, community kitchen, several days of meals      |
|       3 | Blankets           | Warmth for people who are cold, wet, sleeping outdoors or in a shelter, elderly and children at night |
|       4 | First Aid Kit      | Bandages and antiseptic for injuries, cuts, wounds, fractures, bleeding, sprains, minor burns       |
|       5 | Paracetamol Tablets | Medicine for fever, high temperature, headache, body pain, flu-like illness                         |
|       6 | Baby Food          | Nutrition for infants, babies and toddlers, formula milk, young children who cannot eat regular food |
|       7 | Tarpaulin Sheets   | Waterproof cover for damaged roofs, collapsed houses, temporary shelter, rain protection, landslide |
|       8 | Sanitary Kit       | Hygiene supplies, sanitary pads, soap, toothbrush, sanitation in camps, women and girls             |
+---------+--------------------+-----------------------------------------------------------------------------------------------------+
8 rows in set (0.00 sec)
```

*Figure 6.6: RESOURCE_ITEM rows with their usage notes*

# 7. QUERIES

The queries below answer questions a relief coordinator would actually ask, such as which SOS should be handled first, which shelters are crowded and which stock is running low.

## 7.1 Simple Queries with GROUP BY and Aggregates

- Q3: number of SOS and people affected for each urgency level and status (GROUP BY, COUNT, SUM).
- Q4: item categories with more than 300 units in stock (GROUP BY with HAVING).
- Q5: shelters with two or more open reservations (GROUP BY with HAVING).
- Q16: trips and units moved per vehicle (LEFT JOIN with aggregates).
- Q17: average waiting time of open SOS per urgency level (date functions with AVG).

```
mysql> SELECT urgency_level, status,
    ->        COUNT(*) AS sos_count,
    ->        SUM(people_count) AS people_affected
    -> FROM VICTIM_SOS
    -> GROUP BY urgency_level, status
    -> ORDER BY fn_urgency_rank(urgency_level) DESC, status;
+---------------+-----------+-----------+-----------------+
| urgency_level | status    | sos_count | people_affected |
+---------------+-----------+-----------+-----------------+
| critical      | matched   |         1 |               5 |
| critical      | unmatched |         1 |               3 |
| high          | matched   |         1 |               2 |
| high          | unmatched |         2 |              14 |
| medium        | resolved  |         1 |               8 |
| medium        | unmatched |         1 |               6 |
| low           | unmatched |         1 |               2 |
+---------------+-----------+-----------+-----------------+
7 rows in set (0.00 sec)
```

*Figure 7.1: Q3: SOS load by urgency level and status*

```
mysql> SELECT r.category,
    ->        COUNT(DISTINCT st.shelter_id) AS shelters_stocking,
    ->        SUM(st.quantity_available) AS total_qty
    -> FROM STOCK st
    -> JOIN RESOURCE_ITEM r ON r.item_id = st.item_id
    -> GROUP BY r.category
    -> HAVING SUM(st.quantity_available) > 300
    -> ORDER BY total_qty DESC;
+----------+--------------------+-----------+
| category | shelters_stocking | total_qty |
+----------+--------------------+-----------+
| Water    |                  5 |      2270 |
| Food     |                  3 |       974 |
| Shelter  |                  5 |       565 |
+----------+--------------------+-----------+
3 rows in set (0.00 sec)
```

*Figure 7.2: Q4: item categories with more than 300 units in stock*

```
mysql> SELECT sh.name, COUNT(*) AS open_reservations, SUM(sr.quantity_reserved) AS units_held
    -> FROM SUPPLY_RESERVATION sr
    -> JOIN SHELTER sh ON sh.shelter_id = sr.shelter_id
    -> WHERE sr.status IN ('pending', 'committed')
    -> GROUP BY sh.shelter_id, sh.name
    -> HAVING COUNT(*) >= 2
    -> ORDER BY open_reservations DESC;
+--------------------------+------------------+------------+
| name                     | open_reservations | units_held |
+--------------------------+------------------+------------+
| Yelahanka Relief Centre  |                4 |         82 |
| Koramangala Indoor Stadium |                2 |         25 |
+--------------------------+------------------+------------+
2 rows in set (0.00 sec)
```

*Figure 7.3: Q5: shelters with two or more open reservations*

```
mysql> SELECT lv.registration_no, lv.type, lv.status,
    ->        COUNT(d.log_id) AS trips,
    ->        COALESCE(SUM(d.quantity_dispatched), 0) AS units_moved
    -> FROM LOGISTICS_VEHICLE lv
    -> LEFT JOIN DISTRIBUTION_LOG d ON d.vehicle_id = lv.vehicle_id
    -> GROUP BY lv.vehicle_id, lv.registration_no, lv.type, lv.status
    -> ORDER BY trips DESC, lv.registration_no;
+-----------------+------------+-------------+-------+-------------+
| registration_no | type       | status      | trips | units_moved |
+-----------------+------------+-------------+-------+-------------+
| KA-03-EF-9012   | Ambulance  | in_transit  |     2 |          11 |
| KA-01-AB-1234   | Truck      | in_transit  |     1 |          30 |
| KA-05-CD-5678   | Mini Truck | in_transit  |     1 |          40 |
| KA-02-GH-3456   | Tempo      | maintenance |     0 |           0 |
| KA-04-IJ-7890   | Truck      | in_transit  |     0 |           0 |
+-----------------+------------+-------------+-------+-------------+
5 rows in set (0.00 sec)
```

*Figure 7.4: Q16: vehicle utilisation*

```
mysql> SELECT urgency_level, COUNT(*) AS open_sos,
    ->        ROUND(AVG(TIMESTAMPDIFF(HOUR, reported_at, NOW())), 1) AS avg_hours_waiting,
    ->        MAX(TIMESTAMPDIFF(HOUR, reported_at, NOW())) AS max_hours_waiting
    -> FROM VICTIM_SOS
    -> WHERE status <> 'resolved'
    -> GROUP BY urgency_level
    -> ORDER BY fn_urgency_rank(urgency_level) DESC;
+---------------+----------+------------------+------------------+
| urgency_level | open_sos | avg_hours_waiting | max_hours_waiting |
+---------------+----------+------------------+------------------+
| critical      |        2 |              3.0 |                4 |
| high          |        3 |              8.0 |               15 |
| medium        |        1 |             31.0 |               31 |
| low           |        1 |             51.0 |               51 |
+---------------+----------+------------------+------------------+
4 rows in set (0.03 sec)
```

*Figure 7.5: Q17: average waiting time of open SOS*

## 7.2 Update Operations

- Escalate unmatched SOS that have waited 12 hours or more by one urgency level (UPDATE with CASE).
- Mark a vehicle as available after maintenance, and reassign a volunteer to another shelter.
- Raise the reorder threshold of all Medical items by 5 (UPDATE with JOIN).
- Bring the water stock at Jayanagar up to the average of all shelters (UPDATE with a subquery).

```
mysql> UPDATE VICTIM_SOS
    -> SET urgency_level = CASE urgency_level
    ->                         WHEN 'low' THEN 'medium'
    ->                         WHEN 'medium' THEN 'high'
    ->                         WHEN 'high' THEN 'critical'
    ->                         ELSE urgency_level END
    -> WHERE status = 'unmatched'
    ->   AND TIMESTAMPDIFF(HOUR, reported_at, NOW()) >= 12;
Query OK, 3 rows affected (0.04 sec)
Rows matched: 3  Changed: 3  Warnings: 0
```

*Figure 7.6: UPDATE 1: escalate unmatched SOS waiting 12 hours or more*

```
mysql> UPDATE LOGISTICS_VEHICLE SET status = 'available' WHERE registration_no = 'KA-02-GH-3456';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

*Figure 7.7: UPDATE 2: vehicle back from maintenance*

```
mysql> UPDATE APP_USER SET shelter_id = 4 WHERE email = 'divya.reddy@relief.org';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

*Figure 7.8: UPDATE 3: reassign a volunteer to another shelter*

```
mysql> UPDATE STOCK st
    -> JOIN RESOURCE_ITEM r ON r.item_id = st.item_id
    -> SET st.reorder_threshold = st.reorder_threshold + 5
    -> WHERE r.category = 'Medical';
Query OK, 5 rows affected (0.01 sec)
Rows matched: 5  Changed: 5  Warnings: 0
```

*Figure 7.9: UPDATE 4: raise the reorder threshold of Medical items (multi-table UPDATE)*

```
mysql> UPDATE STOCK
    -> SET quantity_available = (SELECT avg_q FROM
    ->         (SELECT ROUND(AVG(quantity_available)) AS avg_q FROM STOCK WHERE item_id = 1) t)
    -> WHERE item_id = 1 AND shelter_id = 5;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

*Figure 7.10: UPDATE 5: bring Jayanagar water stock up to the average (UPDATE with subquery)*

## 7.3 Delete Operations

Deletes are protected by triggers and foreign keys. A duplicate SOS without reservations can be deleted, cancelled dispatches and old resolved alerts can be removed, and a stock row is deleted only if no reservation depends on it.

```
mysql> -- D1  Remove a duplicate/spam SOS (allowed: it has no active reservations)
mysql> CALL sp_register_sos(3, 'low', 'duplicate test report - please ignore', 1, NULL, NULL, @dup);
Query OK, 1 row affected (0.04 sec)

mysql> DELETE FROM VICTIM_SOS WHERE sos_id = @dup;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> -- D2  Cancel a dispatch, then purge cancelled dispatch logs
mysql> UPDATE DISTRIBUTION_LOG SET status = 'cancelled' WHERE log_id = @l3;   -- vehicle freed, reservation -> pending
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> DELETE FROM DISTRIBUTION_LOG WHERE status = 'cancelled';
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> -- D3  Purge resolved low-stock alerts older than 30 days
mysql> DELETE FROM STOCK_ALERT
    -> WHERE is_resolved = TRUE AND alert_time < NOW() - INTERVAL 30 DAY;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- D4  Delete a stock row that no reservation depends on
mysql> DELETE FROM STOCK WHERE item_id = 3 AND shelter_id = 5;
Query OK, 1 row affected (0.04 sec)
```

*Figure 7.11: DELETE operations*

## 7.4 Correlated Queries

- Q8: stock rows that are below the average quantity of the same item. The inner query uses the item of the outer row, so it runs once for each stock row.
- Q9: open SOS that do not have any supplies reserved yet (NOT EXISTS).

```
mysql> SELECT sh.name AS shelter, r.item_name, st.quantity_available
    -> FROM STOCK st
    -> JOIN SHELTER sh ON sh.shelter_id = st.shelter_id
    -> JOIN RESOURCE_ITEM r ON r.item_id = st.item_id
    -> WHERE st.quantity_available <
    ->       (SELECT AVG(s2.quantity_available) FROM STOCK s2 WHERE s2.item_id = st.item_id)
    -> ORDER BY r.item_name, st.quantity_available;
+------------------------------+---------------------+--------------------+
| shelter                      | item_name           | quantity_available |
+------------------------------+---------------------+--------------------+
| Jayanagar Municipal Hall     | Blankets            |                 25 |
| Whitefield Government School | Blankets            |                 60 |
| Banashankari Community Hall  | Blankets            |                 80 |
| Jayanagar Municipal Hall     | Drinking Water      |                100 |
| Whitefield Government School | Drinking Water      |                300 |
| Banashankari Community Hall  | First Aid Kit       |                 25 |
| Yelahanka Relief Centre      | First Aid Kit       |                 28 |
| Koramangala Indoor Stadium   | Paracetamol Tablets |                 50 |
| Banashankari Community Hall  | Rice                |                200 |
| Yelahanka Relief Centre      | Sanitary Kit        |                  2 |
+------------------------------+---------------------+--------------------+
10 rows in set (0.00 sec)
```

*Figure 7.12: Q8: stock rows below the average for the same item (correlated subquery)*

```
mysql> SELECT v.sos_id, v.urgency_level, v.description
    -> FROM VICTIM_SOS v
    -> WHERE v.status <> 'resolved'
    ->   AND NOT EXISTS (SELECT 1 FROM SUPPLY_RESERVATION sr WHERE sr.sos_id = v.sos_id);
+--------+---------------+-----------------------------------------------------------------------------------+
| sos_id | urgency_level | description                                                                       |
+--------+---------------+-----------------------------------------------------------------------------------+
|      5 | low           | Need drinking water and rice for two days                                         |
|      8 | high          | Pregnant woman and three children trapped by flood; urgent medical support needed |
+--------+---------------+-----------------------------------------------------------------------------------+
2 rows in set (0.00 sec)
```

*Figure 7.13: Q9: open SOS with no supplies reserved (NOT EXISTS)*

## 7.5 Nested Queries

- Q7: shelters whose occupancy percentage is above the average (scalar subquery).
- Q10: vehicles that have never been dispatched (NOT IN subquery).

```
mysql> SELECT name, current_occupancy, capacity,
    ->        ROUND(100 * current_occupancy / capacity, 1) AS occupancy_pct
    -> FROM SHELTER
    -> WHERE current_occupancy / capacity >
    ->       (SELECT AVG(current_occupancy / capacity) FROM SHELTER WHERE capacity > 0)
    -> ORDER BY occupancy_pct DESC;
+------------------------------+-------------------+----------+---------------+
| name                         | current_occupancy | capacity | occupancy_pct |
+------------------------------+-------------------+----------+---------------+
| Whitefield Government School |               120 |      120 |         100.0 |
+------------------------------+-------------------+----------+---------------+
1 row in set (0.00 sec)
```

*Figure 7.14: Q7: shelters more crowded than the average (scalar subquery)*

```
mysql> SELECT vehicle_id, registration_no, type, status
    -> FROM LOGISTICS_VEHICLE
    -> WHERE vehicle_id NOT IN (SELECT vehicle_id FROM DISTRIBUTION_LOG WHERE vehicle_id IS NOT NULL);
+------------+-----------------+-------+-------------+
| vehicle_id | registration_no | type  | status      |
+------------+-----------------+-------+-------------+
|          4 | KA-02-GH-3456   | Tempo | maintenance |
|          5 | KA-04-IJ-7890   | Truck | in_transit  |
+------------+-----------------+-------+-------------+
2 rows in set (0.04 sec)
```

*Figure 7.15: Q10: vehicles that have never been dispatched (NOT IN)*

## 7.6 Other Queries

- Q1: open critical and high SOS, most urgent and oldest first (filter with ORDER BY).
- Q2 and Q6: SOS with reporter and shelter, and full reservation details (INNER, LEFT and multi-table JOIN).
- Q11: shelters ranked by free beds with a running total (window functions RANK and SUM OVER).
- Q12: two nearest shelters with enough beds for every unmatched SOS (CTE with ROW_NUMBER).
- Q13–Q15: supply lines per SOS, a stock health label using CASE, and one "needs attention" list using UNION ALL.
- Full-text search on SOS descriptions using MATCH ... AGAINST.

```
mysql> SELECT sos_id, urgency_level, status, people_count, reported_at
    -> FROM VICTIM_SOS
    -> WHERE urgency_level IN ('critical', 'high') AND status <> 'resolved'
    -> ORDER BY fn_urgency_rank(urgency_level) DESC, reported_at ASC;
+--------+---------------+-----------+--------------+---------------------+
| sos_id | urgency_level | status    | people_count | reported_at         |
+--------+---------------+-----------+--------------+---------------------+
|      1 | critical      | matched   |            5 | 2026-10-03 13:31:07 |
|      4 | critical      | unmatched |            3 | 2026-10-03 15:31:07 |
|      6 | high          | unmatched |           10 | 2026-10-03 02:31:07 |
|      2 | high          | matched   |            2 | 2026-10-03 11:31:07 |
|      8 | high          | unmatched |            4 | 2026-10-03 14:31:07 |
+--------+---------------+-----------+--------------+---------------------+
5 rows in set (0.00 sec)
```

*Figure 7.16: Q1: open critical and high SOS, most urgent and oldest first*

```
mysql> SELECT v.sos_id, v.urgency_level, u.name AS reported_by, u.role,
    ->        COALESCE(s.name, '-- not yet matched --') AS shelter
    -> FROM VICTIM_SOS v
    -> JOIN APP_USER u ON u.user_id = v.reported_by
    -> LEFT JOIN SHELTER s ON s.shelter_id = v.matched_shelter_id
    -> ORDER BY v.sos_id;
+--------+---------------+-------------+----------------+----------------------------+
| sos_id | urgency_level | reported_by | role           | shelter                    |
+--------+---------------+-------------+----------------+----------------------------+
|      1 | critical      | Meera Iyer  | Field Volunteer | Koramangala Indoor Stadium |
|      2 | high          | Meera Iyer  | Field Volunteer | Banashankari Community Hall |
|      3 | medium        | Karthik Nair | Field Volunteer | -- not yet matched --      |
|      4 | critical      | Farhan Ali  | Field Volunteer | -- not yet matched --      |
|      5 | low           | Meera Iyer  | Field Volunteer | -- not yet matched --      |
|      6 | high          | Karthik Nair | Field Volunteer | -- not yet matched --      |
|      7 | medium        | Farhan Ali  | Field Volunteer | Yelahanka Relief Centre    |
|      8 | high          | Divya Reddy | Field Volunteer | -- not yet matched --      |
+--------+---------------+-------------+----------------+----------------------------+
8 rows in set (0.03 sec)
```

*Figure 7.17: Q2: SOS with reporter and shelter (INNER JOIN and LEFT JOIN)*

```
mysql> SELECT sr.reservation_id, v.urgency_level, r.item_name,
    ->        sr.quantity_reserved, r.unit, sh.name AS supplied_from, sr.status
    -> FROM SUPPLY_RESERVATION sr
    -> JOIN VICTIM_SOS v ON v.sos_id = sr.sos_id
    -> JOIN RESOURCE_ITEM r ON r.item_id = sr.item_id
    -> JOIN SHELTER sh ON sh.shelter_id = sr.shelter_id
    -> ORDER BY fn_urgency_rank(v.urgency_level) DESC, sr.reservation_id;
+---------------+---------------+-------------------+------------------+-------+---------------------------+-----------+
| reservation_id | urgency_level | item_name         | quantity_reserved | unit  | supplied_from             | status    |
+---------------+---------------+-------------------+------------------+-------+---------------------------+-----------+
|             1 | critical      | Baby Food         |                6 | pack  | Koramangala Indoor Stadium | fulfilled |
|             2 | critical      | Paracetamol Tablets |                5 | strip | Koramangala Indoor Stadium | committed |
|             6 | critical      | First Aid Kit     |                2 | kit   | Yelahanka Relief Centre   | pending   |
|             3 | high          | Drinking Water    |               30 | litre | Banashankari Community Hall | committed |
|             7 | high          | Sanitary Kit      |               10 | kit   | Yelahanka Relief Centre   | pending   |
|             8 | high          | Blankets          |               20 | piece | Koramangala Indoor Stadium | pending   |
|             4 | medium        | Tarpaulin Sheets  |               40 | piece | Yelahanka Relief Centre   | committed |
|             5 | medium        | Blankets          |               30 | piece | Yelahanka Relief Centre   | pending   |
+---------------+---------------+-------------------+------------------+-------+---------------------------+-----------+
8 rows in set (0.00 sec)
```

*Figure 7.18: Q6: full reservation details (multi-table JOIN)*

```
mysql> SELECT name, capacity - current_occupancy AS free_beds,
    ->        RANK() OVER (ORDER BY capacity - current_occupancy DESC) AS free_rank,
    ->        SUM(current_occupancy) OVER () AS total_sheltered
    -> FROM SHELTER;
+------------------------------+-----------+-----------+------------------+
| name                         | free_beds | free_rank | total_sheltered  |
+------------------------------+-----------+-----------+------------------+
| Koramangala Indoor Stadium   |       180 |         1 |              340 |
| Yelahanka Relief Centre      |       150 |         2 |              340 |
| Banashankari Community Hall  |       110 |         3 |              340 |
| Jayanagar Municipal Hall     |        70 |         4 |              340 |
| Whitefield Government School |         0 |         5 |              340 |
+------------------------------+-----------+-----------+------------------+
5 rows in set (0.01 sec)
```

*Figure 7.19: Q11: shelters ranked by free beds (window functions)*

```
mysql> WITH open_sos AS (
    ->     SELECT sos_id, urgency_level, people_count, lat_long
    ->     FROM VICTIM_SOS WHERE status = 'unmatched'
    -> ),
    -> candidates AS (
    ->     SELECT o.sos_id, o.urgency_level, s.name AS shelter,
    ->            ROUND(fn_distance_km(o.lat_long, s.lat_long), 2) AS dist_km,
    ->            fn_shelter_free_beds(s.shelter_id) AS free_beds,
    ->            ROW_NUMBER() OVER (PARTITION BY o.sos_id
    ->                               ORDER BY fn_distance_km(o.lat_long, s.lat_long)) AS rn
    ->     FROM open_sos o
    ->     JOIN SHELTER s ON s.status = 'active'
    ->                   AND s.capacity - s.current_occupancy >= o.people_count
    -> )
    -> SELECT sos_id, urgency_level, shelter, dist_km, free_beds, rn AS choice_no
    -> FROM candidates
    -> WHERE rn <= 2
    -> ORDER BY fn_urgency_rank(urgency_level) DESC, sos_id, rn;
+--------+---------------+----------------------------+---------+-----------+-----------+
| sos_id | urgency_level | shelter                    | dist_km | free_beds | choice_no |
+--------+---------------+----------------------------+---------+-----------+-----------+
|      4 | critical      | Yelahanka Relief Centre    |    1.37 |       150 |         1 |
|      4 | critical      | Koramangala Indoor Stadium |   17.61 |       180 |         2 |
|      6 | high          | Koramangala Indoor Stadium |    0.76 |       180 |         1 |
|      6 | high          | Jayanagar Municipal Hall   |    2.89 |        70 |         2 |
|      8 | high          | Koramangala Indoor Stadium |    13.1 |       180 |         1 |
|      8 | high          | Jayanagar Municipal Hall   |   16.61 |        70 |         2 |
|      3 | medium        | Yelahanka Relief Centre    |    5.65 |       150 |         1 |
|      3 | medium        | Koramangala Indoor Stadium |   13.04 |       180 |         2 |
|      5 | low           | Jayanagar Municipal Hall   |    0.69 |        70 |         1 |
|      5 | low           | Koramangala Indoor Stadium |    3.78 |       180 |         2 |
+--------+---------------+----------------------------+---------+-----------+-----------+
10 rows in set (0.04 sec)
```

*Figure 7.20: Q12: two nearest shelters for every unmatched SOS (CTE with ROW_NUMBER)*

```
mysql> SELECT v.sos_id, v.urgency_level,
    ->        COUNT(sr.reservation_id) AS reservation_lines,
    ->        COALESCE(SUM(sr.quantity_reserved), 0) AS units_reserved,
    ->        SUM(sr.status = 'fulfilled') AS fulfilled_lines
    -> FROM VICTIM_SOS v
    -> LEFT JOIN SUPPLY_RESERVATION sr ON sr.sos_id = v.sos_id
    -> GROUP BY v.sos_id, v.urgency_level
    -> HAVING COUNT(sr.reservation_id) > 0
    -> ORDER BY units_reserved DESC;
+--------+---------------+-------------------+----------------+-----------------+
| sos_id | urgency_level | reservation_lines | units_reserved | fulfilled_lines |
+--------+---------------+-------------------+----------------+-----------------+
|      3 | medium        |                 2 |             70 |               0 |
|      2 | high          |                 1 |             30 |               0 |
|      6 | high          |                 2 |             30 |               0 |
|      1 | critical      |                 2 |             11 |               1 |
|      4 | critical      |                 1 |              2 |               0 |
+--------+---------------+-------------------+----------------+-----------------+
5 rows in set (0.00 sec)
```

*Figure 7.21: Q13: supply lines per SOS (LEFT JOIN, GROUP BY and HAVING)*

```
mysql> SELECT sh.name AS shelter, r.item_name, st.quantity_available, st.reorder_threshold,
    ->        CASE WHEN st.quantity_available = 0 THEN 'OUT OF STOCK'
    ->             WHEN st.quantity_available <= st.reorder_threshold THEN 'LOW'
    ->             WHEN st.quantity_available <= st.reorder_threshold*2 THEN 'WATCH'
    ->             ELSE 'OK' END AS health
    -> FROM STOCK st
    -> JOIN SHELTER sh ON sh.shelter_id = st.shelter_id
    -> JOIN RESOURCE_ITEM r ON r.item_id = st.item_id
    -> ORDER BY st.quantity_available / NULLIF(st.reorder_threshold, 0)
    -> LIMIT 10;
+-----------------------------+---------------------+--------------------+-------------------+--------+
| shelter                     | item_name           | quantity_available | reorder_threshold | health |
+-----------------------------+---------------------+--------------------+-------------------+--------+
| Yelahanka Relief Centre     | Sanitary Kit        |                  2 |                25 | LOW    |
| Jayanagar Municipal Hall    | Blankets            |                 25 |                30 | LOW    |
| Yelahanka Relief Centre     | Tarpaulin Sheets    |                 50 |                30 | WATCH  |
| Jayanagar Municipal Hall    | Drinking Water      |                100 |                50 | WATCH  |
| Koramangala Indoor Stadium  | Baby Food           |                 24 |                10 | OK     |
| Banashankari Community Hall | First Aid Kit       |                 25 |                10 | OK     |
| Koramangala Indoor Stadium  | Paracetamol Tablets |                 50 |                20 | OK     |
| Whitefield Government School| Tarpaulin Sheets    |                 50 |                20 | OK     |
| Banashankari Community Hall | Blankets            |                 80 |                30 | OK     |
| Koramangala Indoor Stadium  | First Aid Kit       |                 40 |                15 | OK     |
+-----------------------------+---------------------+--------------------+-------------------+--------+
10 rows in set (0.00 sec)
```

*Figure 7.22: Q14: stock health label (CASE)*

```
mysql> SELECT 'LOW STOCK' AS alert_type, CONCAT(shelter, ' - ', item_name) AS detail FROM v_low_stock
    -> UNION ALL
    -> SELECT 'SHELTER FULL', name FROM SHELTER WHERE status = 'full'
    -> UNION ALL
    -> SELECT 'VEHICLE IN MAINTENANCE', registration_no FROM LOGISTICS_VEHICLE WHERE status = 'maintenance';
+----------------------+---------------------------------------+
| alert_type           | detail                                |
+----------------------+---------------------------------------+
| LOW STOCK            | Yelahanka Relief Centre - Sanitary Kit |
| LOW STOCK            | Jayanagar Municipal Hall - Blankets    |
| SHELTER FULL         | Whitefield Government School           |
| VEHICLE IN MAINTENANCE | KA-02-GH-3456                        |
+----------------------+---------------------------------------+
4 rows in set (0.00 sec)
```

*Figure 7.23: Q15: single "attention needed" feed (UNION ALL)*

```
mysql> -- Natural-language mode, ranked by relevance
mysql> SELECT sos_id, urgency_level, description,
    ->        MATCH(description) AGAINST('flood medicines') AS relevance
    -> FROM VICTIM_SOS
    -> WHERE MATCH(description) AGAINST('flood medicines' IN NATURAL LANGUAGE MODE)
    -> ORDER BY relevance DESC;
+--------+---------------+-----------------------------------------------------------------------------------------------------+--------------------+
| sos_id | urgency_level | description                                                                                         | relevance          |
+--------+---------------+-----------------------------------------------------------------------------------------------------+--------------------+
|      1 | critical      | Family of five stranded on rooftop as flood water rises; infant needs baby food and fever medicines | 0.7249524593353271 |
|      2 | high          | Elderly couple with limited mobility need drinking water and regular medicines after flooding       | 0.3624762296676636 |
|      8 | high          | Pregnant woman and three children trapped by flood; urgent medical support needed                   | 0.3624762296676636 |
+--------+---------------+-----------------------------------------------------------------------------------------------------+--------------------+
3 rows in set (0.00 sec)

mysql>
mysql> -- Boolean mode: must contain flood*, must contain medic*, must NOT contain infant
mysql> SELECT sos_id, urgency_level, description
    -> FROM VICTIM_SOS
    -> WHERE MATCH(description) AGAINST('+flood* +medic* -infant' IN BOOLEAN MODE);
+--------+---------------+-----------------------------------------------------------------------------------------------------+
| sos_id | urgency_level | description                                                                                         |
+--------+---------------+-----------------------------------------------------------------------------------------------------+
|      2 | high          | Elderly couple with limited mobility need drinking water and regular medicines after flooding       |
|      8 | high          | Pregnant woman and three children trapped by flood; urgent medical support needed                   |
+--------+---------------+-----------------------------------------------------------------------------------------------------+
2 rows in set (0.00 sec)

mysql>
mysql> -- Search the item catalogue
mysql> SELECT item_id, item_name, category
    -> FROM RESOURCE_ITEM
    -> WHERE MATCH(item_name, category) AGAINST('medical shelter' IN NATURAL LANGUAGE MODE);
+---------+---------------------+----------+
| item_id | item_name           | category |
+---------+---------------------+----------+
|       3 | Blankets            | Shelter  |
|       4 | First Aid Kit       | Medical  |
|       5 | Paracetamol Tablets | Medical  |
|       7 | Tarpaulin Sheets    | Shelter  |
+---------+---------------------+----------+
4 rows in set (0.00 sec)
```

*Figure 7.24: Full-text search on SOS descriptions (MATCH ... AGAINST)*

# 8. STORED PROCEDURES, FUNCTIONS AND TRIGGERS

## 8.1 Stored Procedures and Functions

**Table 8.1: Stored functions**

| Function | Purpose |
|---|---|
| fn_shelter_free_beds(shelter_id) | Returns capacity minus current occupancy of a shelter |
| fn_urgency_rank(level) | Converts urgency to a number (critical = 4 ... low = 1) for sorting |
| fn_distance_km(p1, p2) | Haversine distance in km between two "lat,long" points |

```
mysql> DELIMITER $$
mysql>
mysql> -- Free beds in a shelter
mysql> CREATE FUNCTION fn_shelter_free_beds(p_shelter_id INT)
    -> RETURNS INT
    -> READS SQL DATA
    -> BEGIN
    ->     DECLARE v_free INT;
    ->     SELECT capacity - current_occupancy INTO v_free
    ->       FROM SHELTER WHERE shelter_id = p_shelter_id;
    ->     RETURN v_free;
    -> END$$
Query OK, 0 rows affected (0.05 sec)
```

*Figure 8.1: fn_shelter_free_beds: free beds in a shelter*

```
mysql> -- Numeric priority for urgency levels (higher = more urgent)
mysql> CREATE FUNCTION fn_urgency_rank(p_level VARCHAR(20))
    -> RETURNS INT
    -> DETERMINISTIC NO SQL
    -> BEGIN
    ->     RETURN CASE p_level
    ->                WHEN 'critical' THEN 4
    ->                WHEN 'high' THEN 3
    ->                WHEN 'medium' THEN 2
    ->                WHEN 'low' THEN 1
    ->                ELSE 0
    ->            END;
    -> END$$
Query OK, 0 rows affected (0.03 sec)
```

*Figure 8.2: fn_urgency_rank: numeric priority for urgency levels*

```
mysql> -- Haversine distance (km) between two 'lat,long' strings
mysql> CREATE FUNCTION fn_distance_km(p1 VARCHAR(60), p2 VARCHAR(60))
    -> RETURNS DOUBLE
    -> DETERMINISTIC NO SQL
    -> BEGIN
    ->     DECLARE lat1, lon1, lat2, lon2 DOUBLE;
    ->     IF p1 IS NULL OR p2 IS NULL THEN
    ->         RETURN NULL;
    ->     END IF;
    ->     SET lat1 = CAST(TRIM(SUBSTRING_INDEX(p1, ',', 1))  AS DECIMAL(10,6));
    ->     SET lon1 = CAST(TRIM(SUBSTRING_INDEX(p1, ',', -1)) AS DECIMAL(10,6));
    ->     SET lat2 = CAST(TRIM(SUBSTRING_INDEX(p2, ',', 1))  AS DECIMAL(10,6));
    ->     SET lon2 = CAST(TRIM(SUBSTRING_INDEX(p2, ',', -1)) AS DECIMAL(10,6));
    ->     RETURN 6371 * 2 * ASIN(SQRT(
    ->                POW(SIN(RADIANS(lat2 - lat1) / 2), 2) +
    ->                COS(RADIANS(lat1)) * COS(RADIANS(lat2)) *
    ->                POW(SIN(RADIANS(lon2 - lon1) / 2), 2)));
    -> END$$
Query OK, 0 rows affected (0.05 sec)
```

*Figure 8.3: fn_distance_km: Haversine distance between two locations*

Every procedure that changes data runs inside a transaction. If anything fails, an EXIT HANDLER rolls back the whole transaction and passes the error message back, so no half-finished change is left behind.

**Table 8.2: Stored procedures**

| Procedure | What it does |
|---|---|
| sp_register_sos | Inserts a new SOS and returns its id |
| sp_match_sos | Finds the nearest active shelter with enough free beds, locks it (FOR UPDATE) and assigns the SOS |
| sp_auto_match_unmatched | Uses a CURSOR to match every unmatched SOS, most urgent first |
| sp_reserve_supply | Creates a reservation; triggers check and deduct the stock |
| sp_approve_dispatch | Creates a dispatch; triggers check the approver, vehicle and quantity |
| sp_complete_delivery | Marks a dispatch as delivered and frees the vehicle |
| sp_restock / sp_transfer_stock | Adds stock / moves stock between two shelters in one transaction |
| sp_nearest_stock | Lists shelters that hold enough of an item, nearest first (used with vector search) |
| sp_resolve_sos | Resolves an SOS; triggers free the beds and release pending reservations |
| sp_cancel_reservation | Cancels an open reservation; a trigger returns the stock |

```
mysql> -- Register a new SOS
mysql> CREATE PROCEDURE sp_register_sos(
    ->     IN p_reported_by INT,
    ->     IN p_urgency VARCHAR(20),
    ->     IN p_description TEXT,
    ->     IN p_people INT,
    ->     IN p_lat_long VARCHAR(60),
    ->     IN p_intake_doc VARCHAR(255),
    ->     OUT p_sos_id INT)
    -> BEGIN
    ->     INSERT INTO VICTIM_SOS
    ->         (urgency_level, description, people_count, lat_long, intake_doc_ref, reported_by)
    ->     VALUES
    ->         (p_urgency, p_description, p_people, p_lat_long, p_intake_doc, p_reported_by);
    ->     SET p_sos_id = LAST_INSERT_ID();
    -> END$$
Query OK, 0 rows affected (0.04 sec)
```

*Figure 8.4: sp_register_sos: register a new SOS*

```
mysql> -- Match one SOS to the nearest ACTIVE shelter that has enough free beds
mysql> CREATE PROCEDURE sp_match_sos(IN p_sos_id INT)
    -> BEGIN
    ->     DECLARE v_status VARCHAR(20);
    ->     DECLARE v_people INT;
    ->     DECLARE v_ll VARCHAR(60);
    ->     DECLARE v_shelter INT;
    ->
    ->     DECLARE CONTINUE HANDLER FOR NOT FOUND BEGIN END;   -- SELECT..INTO with no row = NULL
    ->     DECLARE EXIT HANDLER FOR SQLEXCEPTION
    ->     BEGIN
    ->         ROLLBACK;
    ->         RESIGNAL;
    ->     END;
    ->
    ->     START TRANSACTION;
    ->
    ->     SELECT status, people_count, lat_long INTO v_status, v_people, v_ll
    ->       FROM VICTIM_SOS WHERE sos_id = p_sos_id FOR UPDATE;
    ->
    ->     IF v_status IS NULL THEN
    ->         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'SOS not found';
    ->     END IF;
    ->     IF v_status <> 'unmatched' THEN
    ->         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'SOS is already matched or resolved';
    ->     END IF;
    ->
    ->     SELECT shelter_id INTO v_shelter
    ->       FROM SHELTER
    ->      WHERE status = 'active'
    ->        AND capacity - current_occupancy >= v_people
    ->      ORDER BY (fn_distance_km(lat_long, v_ll) IS NULL),
    ->               fn_distance_km(lat_long, v_ll),
    ->               (capacity - current_occupancy) DESC
    ->      LIMIT 1
    ->        FOR UPDATE;
    ->
    ->     IF v_shelter IS NULL THEN
    ->         SIGNAL SQLSTATE '45000'
    ->             SET MESSAGE_TEXT = 'No active shelter has enough free beds';
    ->     END IF;
    ->
    ->     UPDATE SHELTER SET current_occupancy = current_occupancy + v_people
    ->      WHERE shelter_id = v_shelter;                       -- trigger flips status to full if needed
    ->     UPDATE VICTIM_SOS SET matched_shelter_id = v_shelter, status = 'matched'
    ->      WHERE sos_id = p_sos_id;
    ->
    ->     COMMIT;
    -> END$$
Query OK, 0 rows affected (0.04 sec)
```

*Figure 8.5: sp_match_sos: match one SOS to the nearest active shelter*

```
mysql> -- Match ALL unmatched SOS, most urgent first (CURSOR + loop)
mysql> CREATE PROCEDURE sp_auto_match_unmatched(OUT p_matched INT, OUT p_failed INT)
    -> BEGIN
    ->     DECLARE v_done INT DEFAULT 0;
    ->     DECLARE v_err INT DEFAULT 0;
    ->     DECLARE v_sos INT;
    ->
    ->     DECLARE cur CURSOR FOR
    ->         SELECT sos_id FROM VICTIM_SOS
    ->          WHERE status = 'unmatched'
    ->          ORDER BY fn_urgency_rank(urgency_level) DESC, reported_at ASC;
    ->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;
    ->     DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET v_err = 1;
    ->
    ->     SET p_matched = 0;
    ->     SET p_failed  = 0;
    ->
    ->     OPEN cur;
    ->     match_loop: LOOP
    ->         FETCH cur INTO v_sos;
    ->         IF v_done = 1 THEN
    ->             LEAVE match_loop;
    ->         END IF;
    ->
    ->         SET v_err = 0;
    ->         CALL sp_match_sos(v_sos);
    ->         IF v_err = 0 THEN
    ->             SET p_matched = p_matched + 1;
    ->         ELSE
    ->             SET p_failed = p_failed + 1;
    ->         END IF;
    ->     END LOOP;
    ->     CLOSE cur;
    -> END$$
Query OK, 0 rows affected (0.04 sec)
```

*Figure 8.6: sp_auto_match_unmatched: match all unmatched SOS using a cursor*

```
mysql> -- Reserve supplies for an SOS (transaction; triggers validate + deduct stock)
mysql> CREATE PROCEDURE sp_reserve_supply(
    ->     IN p_sos_id INT,
    ->     IN p_item_id INT,
    ->     IN p_shelter_id INT,
    ->     IN p_qty INT,
    ->     OUT p_reservation_id INT)
    -> BEGIN
    ->     DECLARE EXIT HANDLER FOR SQLEXCEPTION
    ->     BEGIN
    ->         ROLLBACK;
    ->         RESIGNAL;
    ->     END;
    ->
    ->     START TRANSACTION;
    ->     INSERT INTO SUPPLY_RESERVATION (item_id, shelter_id, sos_id, quantity_reserved)
    ->     VALUES (p_item_id, p_shelter_id, p_sos_id, p_qty);
    ->     SET p_reservation_id = LAST_INSERT_ID();
    ->     COMMIT;
    -> END$$
Query OK, 0 rows affected (0.04 sec)
```

*Figure 8.7: sp_reserve_supply: reserve supplies for an SOS*

```
mysql> -- Coordinator approves a dispatch
mysql> CREATE PROCEDURE sp_approve_dispatch(
    ->     IN p_reservation_id INT,
    ->     IN p_vehicle_id INT,
    ->     IN p_approved_by INT,
    ->     IN p_qty INT,
    ->     OUT p_log_id INT)
    -> BEGIN
    ->     DECLARE EXIT HANDLER FOR SQLEXCEPTION
    ->     BEGIN
    ->         ROLLBACK;
    ->         RESIGNAL;
    ->     END;
    ->
    ->     START TRANSACTION;
    ->     INSERT INTO DISTRIBUTION_LOG (reservation_id, vehicle_id, approved_by, quantity_dispatched)
    ->     VALUES (p_reservation_id, p_vehicle_id, p_approved_by, p_qty);
    ->     SET p_log_id = LAST_INSERT_ID();
    ->     COMMIT;
    -> END$$
Query OK, 0 rows affected (0.04 sec)
```

*Figure 8.8: sp_approve_dispatch: coordinator approves a dispatch*

```
mysql> -- Mark a dispatch as delivered
mysql> CREATE PROCEDURE sp_complete_delivery(IN p_log_id INT)
    -> BEGIN
    ->     DECLARE EXIT HANDLER FOR SQLEXCEPTION
    ->     BEGIN
    ->         ROLLBACK;
    ->         RESIGNAL;
    ->     END;
    ->
    ->     START TRANSACTION;
    ->     UPDATE DISTRIBUTION_LOG SET status = 'delivered'
    ->      WHERE log_id = p_log_id AND status = 'dispatched';
    ->     IF ROW_COUNT() = 0 THEN
    ->         SIGNAL SQLSTATE '45000'
    ->             SET MESSAGE_TEXT = 'No dispatched log found with this id';
    ->     END IF;
    ->     COMMIT;
    -> END$$
Query OK, 0 rows affected (0.03 sec)
```

*Figure 8.9: sp_complete_delivery: mark a dispatch as delivered*

```
mysql> -- Add stock (insert the stock row if the shelter never held the item)
mysql> CREATE PROCEDURE sp_restock(IN p_item_id INT, IN p_shelter_id INT, IN p_qty INT)
    -> BEGIN
    ->     DECLARE EXIT HANDLER FOR SQLEXCEPTION
    ->     BEGIN
    ->         ROLLBACK;
    ->         RESIGNAL;
    ->     END;
    ->
    ->     IF p_qty <= 0 THEN
    ->         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Restock quantity must be positive';
    ->     END IF;
    ->
    ->     START TRANSACTION;
    ->     INSERT INTO STOCK (item_id, shelter_id, quantity_available)
    ->     VALUES (p_item_id, p_shelter_id, p_qty)
    ->     ON DUPLICATE KEY UPDATE quantity_available = quantity_available + VALUES(quantity_available);
    ->     COMMIT;
    -> END$$
Query OK, 0 rows affected (0.04 sec)
```

*Figure 8.10: sp_restock: add stock to a shelter*

```
mysql> -- Move stock between two shelters atomically (rows locked in id order -> no deadlock)
mysql> CREATE PROCEDURE sp_transfer_stock(
    ->     IN p_item_id INT, IN p_from_shelter INT, IN p_to_shelter INT, IN p_qty INT)
    -> BEGIN
    ->     DECLARE EXIT HANDLER FOR SQLEXCEPTION
    ->     BEGIN
    ->         ROLLBACK;
    ->         RESIGNAL;
    ->     END;
    ->
    ->     IF p_from_shelter = p_to_shelter OR p_qty <= 0 THEN
    ->         SIGNAL SQLSTATE '45000'
    ->             SET MESSAGE_TEXT = 'Invalid transfer: shelters must differ and quantity must be positive';
    ->     END IF;
    ->
    ->     START TRANSACTION;
    ->
    ->     SELECT stock_id FROM STOCK
    ->      WHERE item_id = p_item_id AND shelter_id IN (p_from_shelter, p_to_shelter)
    ->      ORDER BY shelter_id
    ->        FOR UPDATE;
    ->
    ->     UPDATE STOCK SET quantity_available = quantity_available - p_qty
    ->      WHERE item_id = p_item_id AND shelter_id = p_from_shelter;   -- CHECK(>=0) rolls back if short
    ->
    ->     INSERT INTO STOCK (item_id, shelter_id, quantity_available)
    ->     VALUES (p_item_id, p_to_shelter, p_qty)
    ->     ON DUPLICATE KEY UPDATE quantity_available = quantity_available + VALUES(quantity_available);
    ->
    ->     COMMIT;
    -> END$$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
```

*Figure 8.11: sp_transfer_stock: move stock between two shelters*

```
mysql> CALL sp_register_sos(5, 'high',
    ->      'Collapsed wall near market; family of four needs shelter and first aid',
    ->      4, '13.0950,77.5950', NULL, @new_sos);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT @new_sos AS new_sos_id;
+------------+
| new_sos_id |
+------------+
|          9 |
+------------+
1 row in set (0.00 sec)
```

*Figure 8.12: Smart matching: a volunteer files a new SOS through the procedure*

```
mysql> CALL sp_match_sos(@new_sos);
Query OK, 0 rows affected (0.01 sec)
```

*Figure 8.13: Smart matching: matching one SOS*

```
mysql> CALL sp_auto_match_unmatched(@matched, @failed);
Query OK, 0 rows affected (0.02 sec)

mysql> SELECT @matched AS matched_count, @failed AS failed_count;
+---------------+--------------+
| matched_count | failed_count |
+---------------+--------------+
|             5 |            0 |
+---------------+--------------+
1 row in set (0.00 sec)
```

*Figure 8.14: Smart matching: matching every unmatched SOS, most urgent first*

```
mysql> SELECT v.sos_id, v.urgency_level, v.people_count, v.status, s.name AS shelter,
    ->        ROUND(fn_distance_km(v.lat_long, s.lat_long), 2) AS dist_km
    -> FROM VICTIM_SOS v
    -> LEFT JOIN SHELTER s ON s.shelter_id = v.matched_shelter_id
    -> ORDER BY v.sos_id;
+--------+---------------+--------------+----------+---------------------------+---------+
| sos_id | urgency_level | people_count | status   | shelter                   | dist_km |
+--------+---------------+--------------+----------+---------------------------+---------+
|      1 | critical      |            5 | matched  | Koramangala Indoor Stadium |    0.55 |
|      2 | high          |            2 | matched  | Banashankari Community Hall |    0.35 |
|      3 | high          |            6 | matched  | Yelahanka Relief Centre    |    5.65 |
|      4 | critical      |            3 | matched  | Yelahanka Relief Centre    |    1.37 |
|      5 | medium        |            2 | matched  | Jayanagar Municipal Hall   |    0.69 |
|      6 | critical      |           10 | matched  | Koramangala Indoor Stadium |    0.76 |
|      7 | medium        |            8 | resolved | Yelahanka Relief Centre    |    0.69 |
|      8 | high          |            4 | matched  | Koramangala Indoor Stadium |    13.1 |
|      9 | high          |            4 | matched  | Yelahanka Relief Centre    |    0.65 |
+--------+---------------+--------------+----------+---------------------------+---------+
9 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM v_shelter_status;   -- occupancy/status updated by triggers
+------------+----------------------------+----------+-------------------+-----------+---------------+--------+
| shelter_id | name                       | capacity | current_occupancy | free_beds | occupancy_pct | status |
+------------+----------------------------+----------+-------------------+-----------+---------------+--------+
|          1 | Banashankari Community Hall |      150 |                40 |       110 |          26.7 | active |
|          2 | Koramangala Indoor Stadium |      300 |               134 |       166 |          44.7 | active |
|          3 | Whitefield Government School |      120 |               120 |         0 |         100.0 | full   |
|          4 | Yelahanka Relief Centre    |      200 |                63 |       137 |          31.5 | active |
|          5 | Jayanagar Municipal Hall   |       80 |                12 |        68 |          15.0 | active |
+------------+----------------------------+----------+-------------------+-----------+---------------+--------+
5 rows in set (0.00 sec)
```

*Figure 8.15: Smart matching: where each SOS was placed (Whitefield was full)*

```
mysql> UPDATE VICTIM_SOS SET status = 'resolved' WHERE sos_id = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM v_shelter_status WHERE shelter_id = 1;
+------------+----------------------------+----------+-------------------+-----------+---------------+--------+
| shelter_id | name                       | capacity | current_occupancy | free_beds | occupancy_pct | status |
+------------+----------------------------+----------+-------------------+-----------+---------------+--------+
|          1 | Banashankari Community Hall |      150 |                38 |       112 |          25.3 | active |
+------------+----------------------------+----------+-------------------+-----------+---------------+--------+
1 row in set (0.00 sec)
```

*Figure 8.16: Smart matching: resolving SOS 2 frees beds and releases reservations*

The last three procedures were added for the web application in 03_app_integration.sql. sp_nearest_stock is the SQL half of the supply recommendation: pgvector finds which items are relevant, and this procedure finds where they are in stock.

```
CREATE PROCEDURE sp_nearest_stock(IN p_item_id INT, IN p_lat_long VARCHAR(60),
                                  IN p_min_qty INT, IN p_limit INT)
BEGIN
  SELECT st.shelter_id, sh.name AS shelter_name, st.quantity_available,
         r.item_name, ROUND(fn_distance_km(sh.lat_long, p_lat_long), 2) AS distance_km
  FROM STOCK st
  JOIN SHELTER sh       ON sh.shelter_id = st.shelter_id
  JOIN RESOURCE_ITEM r  ON r.item_id = st.item_id
  WHERE st.item_id = p_item_id AND sh.status <> 'closed'
    AND st.quantity_available >= GREATEST(p_min_qty, 1)
  ORDER BY distance_km, st.quantity_available DESC
  LIMIT p_limit;
END
```

```
mysql> SELECT ROUTINE_NAME, ROUTINE_TYPE, SECURITY_TYPE, CREATED FROM information_schema.ROUTINES
    -> WHERE ROUTINE_SCHEMA = 'disaster_relief'
    ->   AND ROUTINE_NAME IN ('sp_nearest_stock', 'sp_resolve_sos', 'sp_cancel_reservation');
+----------------------+-------------+---------------+---------------------+
| ROUTINE_NAME         | ROUTINE_TYPE | SECURITY_TYPE | CREATED             |
+----------------------+-------------+---------------+---------------------+
| sp_cancel_reservation | PROCEDURE    | DEFINER       | 2026-11-04 17:38:46 |
| sp_nearest_stock      | PROCEDURE    | DEFINER       | 2026-11-04 17:38:46 |
| sp_resolve_sos        | PROCEDURE    | DEFINER       | 2026-11-04 17:38:46 |
+----------------------+-------------+---------------+---------------------+
3 rows in set (0.00 sec)

mysql> -- First Aid Kits (item 4), nearest to Yelahanka, at least 1 unit
    -> CALL sp_nearest_stock(4, '13.0900,77.5900', 1, 3);
+------------+----------------------------+-------------------+--------------+------+-------------+
| shelter_id | shelter_name               | quantity_available | item_name     | unit | distance_km |
+------------+----------------------------+-------------------+--------------+------+-------------+
|          4 | Yelahanka Relief Centre     |                28 | First Aid Kit | kit  |        1.37 |
|          2 | Koramangala Indoor Stadium  |                40 | First Aid Kit | kit  |       17.61 |
|          1 | Banashankari Community Hall |                25 | First Aid Kit | kit  |       18.88 |
+------------+----------------------------+-------------------+--------------+------+-------------+
3 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> -- Drinking Water (item 1), nearest to Koramangala, at least 50 litres
    -> CALL sp_nearest_stock(1, '12.9330,77.6200', 50, 3);
+------------+----------------------------+-------------------+---------------+-------+-------------+
| shelter_id | shelter_name               | quantity_available | item_name      | unit  | distance_km |
+------------+----------------------------+-------------------+---------------+-------+-------------+
|          2 | Koramangala Indoor Stadium  |               800 | Drinking Water | litre |        0.55 |
|          5 | Jayanagar Municipal Hall    |               554 | Drinking Water | litre |        2.98 |
|          1 | Banashankari Community Hall |               470 | Drinking Water | litre |        7.98 |
+------------+----------------------------+-------------------+---------------+-------+-------------+
3 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

*Figure 8.17: Procedures added for the application, and sp_nearest_stock for two items*

```
mysql> SELECT stock_id, quantity_available FROM STOCK
    -> WHERE (item_id, shelter_id) = (SELECT item_id, shelter_id FROM SUPPLY_RESERVATION WHERE reservation_id = 7);
+----------+-------------------+
| stock_id | quantity_available |
+----------+-------------------+
|       21 |                 2 |
+----------+-------------------+
1 row in set (0.00 sec)

mysql> CALL sp_cancel_reservation(7);
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT reservation_id, status FROM SUPPLY_RESERVATION WHERE reservation_id = 7;
+----------------+-----------+
| reservation_id | status    |
+----------------+-----------+
|              7 | cancelled |
+----------------+-----------+
1 row in set (0.00 sec)

mysql> -- trigger trg_reservation_au gave the reserved quantity back
    -> SELECT stock_id, quantity_available FROM STOCK
    -> WHERE (item_id, shelter_id) = (SELECT item_id, shelter_id FROM SUPPLY_RESERVATION WHERE reservation_id = 7);
+----------+-------------------+
| stock_id | quantity_available |
+----------+-------------------+
|       21 |                12 |
+----------+-------------------+
1 row in set (0.00 sec)

mysql> -- cancelling it again is rejected by the procedure
    -> CALL sp_cancel_reservation(7);
ERROR 1644 (45000): Reservation not found or already closed

mysql> -- resolving an SOS that is already resolved
    -> CALL sp_resolve_sos(2);
ERROR 1644 (45000): SOS not found or already resolved
```

*Figure 8.18: sp_cancel_reservation returns the stock; invalid repeat actions are rejected*

## 8.2 Triggers

Sixteen triggers keep related tables in sync automatically, so the application never has to update stock or status by hand.

**Table 8.3: Triggers and the rules they enforce**

| Table | What the triggers do |
|---|---|
| SHELTER | Status changes to full or active based on occupancy; a shelter with people cannot be closed |
| STOCK | Updates last_updated; creates a STOCK_ALERT when stock reaches the threshold and resolves it after restock |
| SUPPLY_RESERVATION | Rejects reservations above available stock or for a resolved SOS; deducts stock; returns stock on cancel |
| DISTRIBUTION_LOG | Only a coordinator can approve; vehicle must be available; quantity cannot exceed what was reserved; updates vehicle and reservation status |
| VICTIM_SOS | A resolved SOS cannot be reopened; resolving frees beds; an SOS with active reservations cannot be deleted |

```
mysql> -- SHELTER: keep status in sync with occupancy
mysql> CREATE TRIGGER trg_shelter_bi
    -> BEFORE INSERT ON SHELTER FOR EACH ROW
    -> BEGIN
    ->     IF NEW.status <> 'closed' THEN
    ->         SET NEW.status = IF(NEW.current_occupancy >= NEW.capacity, 'full', 'active');
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> CREATE TRIGGER trg_shelter_bu
    -> BEFORE UPDATE ON SHELTER FOR EACH ROW
    -> BEGIN
    ->     IF NEW.status = 'closed' AND OLD.status <> 'closed' AND NEW.current_occupancy > 0 THEN
    ->         SIGNAL SQLSTATE '45000'
    ->             SET MESSAGE_TEXT = 'Cannot close a shelter that still has occupants';
    ->     END IF;
    ->     IF NEW.status <> 'closed' THEN
    ->         SET NEW.status = IF(NEW.current_occupancy >= NEW.capacity, 'full', 'active');
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.04 sec)
```

*Figure 8.19: SHELTER triggers trg_shelter_bi and trg_shelter_bu*

```
mysql> -- STOCK: timestamp + low-stock alerts
mysql> CREATE TRIGGER trg_stock_bu
    -> BEFORE UPDATE ON STOCK FOR EACH ROW
    -> BEGIN
    ->     SET NEW.last_updated = NOW();
    -> END$$
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> CREATE TRIGGER trg_stock_au
    -> AFTER UPDATE ON STOCK FOR EACH ROW
    -> BEGIN
    ->     -- stock dropped to/below threshold -> raise an alert
    ->     IF NEW.quantity_available < OLD.quantity_available
    ->        AND NEW.quantity_available <= NEW.reorder_threshold THEN
    ->         INSERT INTO STOCK_ALERT (stock_id, quantity_at_alert, reorder_threshold)
    ->         VALUES (NEW.stock_id, NEW.quantity_available, NEW.reorder_threshold);
    ->     END IF;
    ->     -- stock back above threshold -> close open alerts
    ->     IF NEW.quantity_available > NEW.reorder_threshold THEN
    ->         UPDATE STOCK_ALERT SET is_resolved = TRUE
    ->          WHERE stock_id = NEW.stock_id AND is_resolved = FALSE;
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.01 sec)
```

*Figure 8.20: STOCK triggers trg_stock_bu and trg_stock_au (timestamp and low-stock alerts)*

```
mysql> CREATE TRIGGER trg_reservation_bi
    -> BEFORE INSERT ON SUPPLY_RESERVATION FOR EACH ROW
    -> BEGIN
    ->     DECLARE v_avail INT;
    ->     DECLARE v_sos_status VARCHAR(20);
    ->
    ->     SELECT status INTO v_sos_status FROM VICTIM_SOS WHERE sos_id = NEW.sos_id;
    ->     IF v_sos_status = 'resolved' THEN
    ->         SIGNAL SQLSTATE '45000'
    ->             SET MESSAGE_TEXT = 'Cannot reserve supplies for a resolved SOS';
    ->     END IF;
    ->
    ->     SELECT quantity_available INTO v_avail
    ->       FROM STOCK
    ->      WHERE item_id = NEW.item_id AND shelter_id = NEW.shelter_id
    ->        FOR UPDATE;                                   -- row lock
    ->
    ->     IF v_avail IS NULL OR v_avail < NEW.quantity_reserved THEN
    ->         SIGNAL SQLSTATE '45000'
    ->             SET MESSAGE_TEXT = 'Insufficient stock at this shelter for the requested reservation';
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> CREATE TRIGGER trg_reservation_ai
    -> AFTER INSERT ON SUPPLY_RESERVATION FOR EACH ROW
    -> BEGIN
    ->     IF NEW.status IN ('pending', 'committed') THEN
    ->         UPDATE STOCK
    ->            SET quantity_available = quantity_available - NEW.quantity_reserved
    ->          WHERE item_id = NEW.item_id AND shelter_id = NEW.shelter_id;
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.01 sec)
```

*Figure 8.21: SUPPLY_RESERVATION triggers trg_reservation_bi and trg_reservation_ai*

```
mysql>
mysql> CREATE TRIGGER trg_reservation_bu
    -> BEFORE UPDATE ON SUPPLY_RESERVATION FOR EACH ROW
    -> BEGIN
    ->     IF OLD.status IN ('cancelled', 'fulfilled') AND NEW.status <> OLD.status THEN
    ->         SIGNAL SQLSTATE '45000'
    ->             SET MESSAGE_TEXT = 'Cancelled/fulfilled reservations cannot change status';
    ->     END IF;
    ->     IF NEW.quantity_reserved <> OLD.quantity_reserved
    ->        OR NEW.item_id <> OLD.item_id OR NEW.shelter_id <> OLD.shelter_id THEN
    ->         SIGNAL SQLSTATE '45000'
    ->             SET MESSAGE_TEXT = 'Reservation details are immutable; cancel and create a new one';
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> CREATE TRIGGER trg_reservation_au
    -> AFTER UPDATE ON SUPPLY_RESERVATION FOR EACH ROW
    -> BEGIN
    ->     -- cancelling releases the reserved quantity back to stock
    ->     IF NEW.status = 'cancelled' AND OLD.status IN ('pending', 'committed') THEN
    ->         UPDATE STOCK
    ->            SET quantity_available = quantity_available + OLD.quantity_reserved
    ->          WHERE item_id = OLD.item_id AND shelter_id = OLD.shelter_id;
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.05 sec)
```

*Figure 8.22: SUPPLY_RESERVATION triggers trg_reservation_bu and trg_reservation_au*

```
mysql> -- DISTRIBUTION_LOG: validate dispatch + keep related rows in sync
mysql> CREATE TRIGGER trg_distribution_bi
    -> BEFORE INSERT ON DISTRIBUTION_LOG FOR EACH ROW
    -> BEGIN
    ->     DECLARE v_res_status VARCHAR(20);
    ->     DECLARE v_res_qty INT;
    ->     DECLARE v_already INT;
    ->     DECLARE v_veh_status VARCHAR(20);
    ->     DECLARE v_role VARCHAR(30);
    ->
    ->     SELECT status, quantity_reserved INTO v_res_status, v_res_qty
    ->       FROM SUPPLY_RESERVATION WHERE reservation_id = NEW.reservation_id;
    ->     IF v_res_status IS NULL OR v_res_status NOT IN ('pending', 'committed') THEN
    ->         SIGNAL SQLSTATE '45000'
    ->             SET MESSAGE_TEXT = 'Reservation is not open for dispatch';
    ->     END IF;
    ->
    ->     SELECT COALESCE(SUM(quantity_dispatched), 0) INTO v_already
    ->       FROM DISTRIBUTION_LOG
    ->      WHERE reservation_id = NEW.reservation_id AND status <> 'cancelled';
    ->     IF v_already + NEW.quantity_dispatched > v_res_qty THEN
    ->         SIGNAL SQLSTATE '45000'
    ->             SET MESSAGE_TEXT = 'Dispatch quantity exceeds the reserved quantity';
    ->     END IF;
    ->
    ->     IF NEW.vehicle_id IS NOT NULL THEN
    ->         SELECT status INTO v_veh_status
    ->           FROM LOGISTICS_VEHICLE WHERE vehicle_id = NEW.vehicle_id;
    ->         IF v_veh_status <> 'available' THEN
    ->             SIGNAL SQLSTATE '45000'
    ->                 SET MESSAGE_TEXT = 'Vehicle is not available for dispatch';
    ->         END IF;
    ->     END IF;
    ->
    ->     SELECT role INTO v_role FROM APP_USER WHERE user_id = NEW.approved_by;
    ->     IF v_role IS NULL OR v_role <> 'Relief Coordinator' THEN
    ->         SIGNAL SQLSTATE '45000'
    ->             SET MESSAGE_TEXT = 'Only a Relief Coordinator can approve a dispatch';
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.05 sec)
```

*Figure 8.23: DISTRIBUTION_LOG trigger trg_distribution_bi (validate a dispatch)*

```
mysql>
mysql> CREATE TRIGGER trg_distribution_ai
    -> AFTER INSERT ON DISTRIBUTION_LOG FOR EACH ROW
    -> BEGIN
    ->     UPDATE SUPPLY_RESERVATION SET status = 'committed'
    ->      WHERE reservation_id = NEW.reservation_id AND status = 'pending';
    ->     IF NEW.vehicle_id IS NOT NULL THEN
    ->         UPDATE LOGISTICS_VEHICLE SET status = 'in_transit'
    ->          WHERE vehicle_id = NEW.vehicle_id;
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> CREATE TRIGGER trg_distribution_bu
    -> BEFORE UPDATE ON DISTRIBUTION_LOG FOR EACH ROW
    -> BEGIN
    ->     IF OLD.status <> 'dispatched' AND NEW.status <> OLD.status THEN
    ->         SIGNAL SQLSTATE '45000'
    ->             SET MESSAGE_TEXT = 'Delivered/cancelled dispatches cannot change status';
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.01 sec)
```

*Figure 8.24: DISTRIBUTION_LOG triggers trg_distribution_ai and trg_distribution_bu*

```
mysql>
mysql> CREATE TRIGGER trg_distribution_au
    -> AFTER UPDATE ON DISTRIBUTION_LOG FOR EACH ROW
    -> BEGIN
    ->     DECLARE v_total_qty INT;
    ->     DECLARE v_delivered INT;
    ->     DECLARE v_active    INT;
    ->
    ->     IF OLD.status = 'dispatched' AND NEW.status IN ('delivered', 'cancelled') THEN
    ->         -- free the vehicle
    ->         IF NEW.vehicle_id IS NOT NULL THEN
    ->             UPDATE LOGISTICS_VEHICLE SET status = 'available'
    ->              WHERE vehicle_id = NEW.vehicle_id;
    ->         END IF;
    ->
    ->         SELECT quantity_reserved INTO v_total_qty
    ->           FROM SUPPLY_RESERVATION WHERE reservation_id = NEW.reservation_id;
    ->
    ->         IF NEW.status = 'delivered' THEN
    ->             SELECT COALESCE(SUM(quantity_dispatched), 0) INTO v_delivered
    ->               FROM DISTRIBUTION_LOG
    ->              WHERE reservation_id = NEW.reservation_id AND status = 'delivered';
    ->             IF v_delivered >= v_total_qty THEN
    ->                 UPDATE SUPPLY_RESERVATION SET status = 'fulfilled'
    ->                  WHERE reservation_id = NEW.reservation_id;
    ->             END IF;
    ->         ELSE
    ->             -- cancelled dispatch: if nothing else is moving, reservation goes back to pending
    ->             SELECT COUNT(*) INTO v_active
    ->               FROM DISTRIBUTION_LOG
    ->              WHERE reservation_id = NEW.reservation_id
    ->                AND status IN ('dispatched', 'delivered');
    ->             IF v_active = 0 THEN
    ->                 UPDATE SUPPLY_RESERVATION SET status = 'pending'
    ->                  WHERE reservation_id = NEW.reservation_id AND status = 'committed';
    ->             END IF;
    ->         END IF;
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.05 sec)
```

*Figure 8.25: DISTRIBUTION_LOG trigger trg_distribution_au*

```
mysql> -- VICTIM_SOS: status rules, occupancy release, delete guard
mysql> CREATE TRIGGER trg_sos_bu
    -> BEFORE UPDATE ON VICTIM_SOS FOR EACH ROW
    -> BEGIN
    ->     IF OLD.status = 'resolved' AND NEW.status <> 'resolved' THEN
    ->         SIGNAL SQLSTATE '45000'
    ->             SET MESSAGE_TEXT = 'A resolved SOS cannot be re-opened';
    ->     END IF;
    ->     IF NEW.matched_shelter_id IS NOT NULL AND OLD.matched_shelter_id IS NULL
    ->        AND NEW.status = 'unmatched' THEN
    ->         SET NEW.status = 'matched';
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE TRIGGER trg_sos_au
    -> AFTER UPDATE ON VICTIM_SOS FOR EACH ROW
    -> BEGIN
    ->     IF OLD.status <> 'resolved' AND NEW.status = 'resolved' THEN
    ->         -- people leave the shelter
    ->         IF NEW.matched_shelter_id IS NOT NULL THEN
    ->             UPDATE SHELTER
    ->                SET current_occupancy = GREATEST(current_occupancy - NEW.people_count, 0)
    ->              WHERE shelter_id = NEW.matched_shelter_id;
    ->         END IF;
    ->         -- release stock held by reservations that were never dispatched
    ->         UPDATE SUPPLY_RESERVATION SET status = 'cancelled'
    ->          WHERE sos_id = NEW.sos_id AND status = 'pending';
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.02 sec)
```

*Figure 8.26: VICTIM_SOS triggers trg_sos_bu and trg_sos_au*

```
mysql>
mysql> CREATE TRIGGER trg_sos_bd
    -> BEFORE DELETE ON VICTIM_SOS FOR EACH ROW
    -> BEGIN
    ->     IF EXISTS (SELECT 1 FROM SUPPLY_RESERVATION
    ->                 WHERE sos_id = OLD.sos_id AND status IN ('pending', 'committed')) THEN
    ->         SIGNAL SQLSTATE '45000'
    ->             SET MESSAGE_TEXT = 'Cannot delete an SOS that has active reservations';
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> CREATE TRIGGER trg_sos_ad
    -> AFTER DELETE ON VICTIM_SOS FOR EACH ROW
    -> BEGIN
    ->     IF OLD.status = 'matched' AND OLD.matched_shelter_id IS NOT NULL THEN
    ->         UPDATE SHELTER
    ->            SET current_occupancy = GREATEST(current_occupancy - OLD.people_count, 0)
    ->          WHERE shelter_id = OLD.matched_shelter_id;
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.05 sec)
```

*Figure 8.27: VICTIM_SOS triggers trg_sos_bd and trg_sos_ad*

**Validation tests**

Each statement below is expected to fail, which shows that the CHECK constraints, triggers and foreign keys block invalid data.

```
mysql> -- V1 CHECK: negative capacity
mysql> INSERT INTO SHELTER (name, capacity) VALUES ('Bad Shelter', -5);
ERROR 3819 (HY000): Check constraint 'shelter_chk_1' is violated.
mysql>
mysql> -- V2 CHECK: occupancy > capacity
mysql> UPDATE SHELTER SET current_occupancy = capacity + 1 WHERE shelter_id = 1;
ERROR 3819 (HY000): Check constraint 'shelter_chk_4' is violated.
mysql>
mysql> -- V3 CHECK: invalid email / phone
mysql> INSERT INTO APP_USER (name, role, email, phone, password_hash)
    -> VALUES ('X', 'Field Volunteer', 'not-an-email', '12345', 'hash');
ERROR 3819 (HY000): Check constraint 'chk_user_email' is violated.
mysql>
mysql> -- V4 CHECK: invalid role
mysql> INSERT INTO APP_USER (name, role, email, password_hash)
    -> VALUES ('X', 'Admin', 'x@relief.org', 'hash');
ERROR 3819 (HY000): Check constraint 'app_user_chk_1' is violated.
mysql>
mysql> -- V5 TRIGGER: over-reserving stock
mysql> CALL sp_reserve_supply(4, 4, 4, 9999, @x);
ERROR 1644 (45000): Insufficient stock at this shelter for the requested reservation
mysql>
mysql> -- V6 TRIGGER: only a coordinator can approve a dispatch (user 3 is a volunteer)
mysql> CALL sp_approve_dispatch(@r5, 2, 3, 10, @x);
ERROR 1644 (45000): Only a Relief Coordinator can approve a dispatch
mysql>
mysql> -- V7 TRIGGER: dispatch more than reserved
mysql> CALL sp_approve_dispatch(@r5, 1, 1, 9999, @x);
ERROR 1644 (45000): Dispatch quantity exceeds the reserved quantity
mysql>
mysql> -- V8 TRIGGER: close a shelter that still has people in it
mysql> UPDATE SHELTER SET status = 'closed' WHERE shelter_id = 1;
ERROR 1644 (45000): Cannot close a shelter that still has occupants
mysql>
mysql> -- V9 TRIGGER: re-open a resolved SOS
mysql> UPDATE VICTIM_SOS SET status = 'unmatched' WHERE sos_id = 2;
ERROR 1644 (45000): A resolved SOS cannot be re-opened
mysql>
mysql> -- V10 TRIGGER: delete an SOS that still has active reservations
mysql> DELETE FROM VICTIM_SOS WHERE sos_id = 3;
ERROR 1644 (45000): Cannot delete an SOS that has active reservations
```

*Figure 8.28: Validation tests V1–V10: CHECK constraints and triggers reject invalid data*

```
mysql> -- V11 FK (RESTRICT): delete a user who reported SOS
mysql> DELETE FROM APP_USER WHERE user_id = 3;
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`disaster_relief`.`victim_sos`, CONSTRAINT `fk_sos_reported_by` F
REIGN KEY (`reported_by`) REFERENCES `app_user` (`user_id`) ON DELETE RESTRICT)
mysql>
mysql> -- V12 Transaction rollback: transfer more stock than exists
mysql> CALL sp_transfer_stock(1, 5, 1, 999999);
+----------+
| stock_id |
+----------+
|        1 |
|       22 |
+----------+
2 rows in set (0.00 sec)

ERROR 3819 (HY000): Check constraint 'stock_chk_1' is violated.
mysql>
```

*Figure 8.29: Validation tests V11–V12: foreign key RESTRICT and transaction rollback*

# 9. ADVANCED DATABASE FEATURES

The project uses all eight advanced database features listed in the requirements. The table below shows where each one is used. Stored procedures, functions and triggers are covered in Chapter 8, NoSQL aggregation in Chapter 10 and vector similarity search in Chapter 11.

**Table 9.1: Advanced database features used in the project**

| Feature | Where it is used |
|---|---|
| Transactions | Every write procedure (START TRANSACTION, COMMIT, ROLLBACK on error); row locks with FOR UPDATE in matching and transfer; SAVEPOINT demo |
| Views | 9 views, e.g. v_open_sos_priority (dashboard queue), v_low_stock, v_shelter_directory; volunteers read views instead of base tables |
| Stored procedures / functions | 11 procedures and 3 functions for matching, reservation, dispatch, restock and transfer |
| Triggers | 16 triggers that update stock, shelter status and vehicle status and block invalid actions |
| Indexing | Foreign-key and composite indexes in MySQL; 2dsphere and multikey indexes in MongoDB; HNSW index in pgvector |
| Full-text search | FULLTEXT index on SOS description, used by the keyword search box (MATCH ... AGAINST) |
| NoSQL aggregation | MongoDB pipelines with $unwind, $group, $bucket, $filter and $geoNear for analytics |
| Vector similarity search | pgvector cosine search for semantic search, duplicate warning and supply recommendation |

## 9.1 Views

Five views were created in Part 2. Four more were added for the application: v_shelter_item_stock, v_shelter_directory, v_reservation_detail and v_volunteer_activity. The dashboard and SOS pages read directly from these views.

```
mysql> CREATE OR REPLACE VIEW v_shelter_status AS
    -> SELECT shelter_id, name, capacity, current_occupancy,
    ->        capacity - current_occupancy AS free_beds,
    ->        ROUND(100 * current_occupancy / NULLIF(capacity, 0), 1) AS occupancy_pct,
    ->        status
    -> FROM SHELTER;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE OR REPLACE VIEW v_low_stock AS
    -> SELECT st.stock_id, sh.name AS shelter, r.item_name, r.category, r.unit,
    ->        st.quantity_available, st.reorder_threshold,
    ->        st.reorder_threshold - st.quantity_available AS shortfall
    -> FROM STOCK st
    -> JOIN SHELTER sh ON sh.shelter_id = st.shelter_id
    -> JOIN RESOURCE_ITEM r ON r.item_id = st.item_id
    -> WHERE st.quantity_available <= st.reorder_threshold;
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> CREATE OR REPLACE VIEW v_open_sos_priority AS
    -> SELECT v.sos_id, v.urgency_level, fn_urgency_rank(v.urgency_level) AS priority_rank,
    ->        v.status, v.people_count, v.description,
    ->        TIMESTAMPDIFF(HOUR, v.reported_at, NOW()) AS hours_waiting,
    ->        u.name AS reported_by_name,
    ->        sh.name AS matched_shelter
    -> FROM VICTIM_SOS v
    -> JOIN APP_USER u ON u.user_id = v.reported_by
    -> LEFT JOIN SHELTER sh ON sh.shelter_id = v.matched_shelter_id
    -> WHERE v.status <> 'resolved';
Query OK, 0 rows affected (0.01 sec)
```

*Figure 9.1: CREATE VIEW v_shelter_status, v_low_stock and v_open_sos_priority*

```
mysql>
mysql> CREATE OR REPLACE VIEW v_dispatch_overview AS
    -> SELECT d.log_id, d.dispatch_time, d.status AS dispatch_status,
    ->        r.item_name, d.quantity_dispatched, r.unit,
    ->        sh.name AS from_shelter,
    ->        lv.registration_no AS vehicle, lv.type AS vehicle_type,
    ->        u.name AS approved_by_name,
    ->        sr.sos_id
    -> FROM DISTRIBUTION_LOG d
    -> JOIN SUPPLY_RESERVATION sr ON sr.reservation_id = d.reservation_id
    -> JOIN RESOURCE_ITEM r ON r.item_id = sr.item_id
    -> JOIN SHELTER sh ON sh.shelter_id = sr.shelter_id
    -> JOIN APP_USER u ON u.user_id = d.approved_by
    -> LEFT JOIN LOGISTICS_VEHICLE lv ON lv.vehicle_id = d.vehicle_id;
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> CREATE OR REPLACE VIEW v_item_stock_summary AS
    -> SELECT r.item_id, r.item_name, r.category,
    ->        COUNT(st.stock_id) AS shelters_stocking,
    ->        SUM(st.quantity_available) AS total_available
    -> FROM RESOURCE_ITEM r
    -> LEFT JOIN STOCK st ON st.item_id = r.item_id
    -> GROUP BY r.item_id, r.item_name, r.category;
Query OK, 0 rows affected (0.01 sec)
```

*Figure 9.2: CREATE VIEW v_dispatch_overview and v_item_stock_summary*

```
mysql> SELECT * FROM v_shelter_status ORDER BY occupancy_pct DESC;
+------------+-----------------------------+----------+------------------+-----------+---------------+--------+
| shelter_id | name                        | capacity | current_occupancy | free_beds | occupancy_pct | status |
+------------+-----------------------------+----------+------------------+-----------+---------------+--------+
|          3 | Whitefield Government School |      120 |              120 |         0 |         100.0 | full   |
|          2 | Koramangala Indoor Stadium  |      300 |              120 |       180 |          40.0 | active |
|          1 | Banashankari Community Hall |      150 |               40 |       110 |          26.7 | active |
|          4 | Yelahanka Relief Centre     |      200 |               50 |       150 |          25.0 | active |
|          5 | Jayanagar Municipal Hall    |       80 |               10 |        70 |          12.5 | active |
+------------+-----------------------------+----------+------------------+-----------+---------------+--------+
5 rows in set (0.00 sec)

mysql> SELECT * FROM v_low_stock ORDER BY shortfall DESC;
+----------+-----------------------+--------------+----------+-------+--------------------+-------------------+-----------+
| stock_id | shelter               | item_name    | category | unit  | quantity_available | reorder_threshold | shortfall |
+----------+-----------------------+--------------+----------+-------+--------------------+-------------------+-----------+
|       21 | Yelahanka Relief Centre | Sanitary Kit | Hygiene  | kit   |                  2 |                25 |        23 |
|       23 | Jayanagar Municipal Hall | Blankets    | Shelter  | piece |                 25 |                30 |         5 |
+----------+-----------------------+--------------+----------+-------+--------------------+-------------------+-----------+
2 rows in set (0.00 sec)
```

*Figure 9.3: Q18: querying v_shelter_status and v_low_stock*

```
mysql> SELECT * FROM v_open_sos_priority ORDER BY priority_rank DESC, hours_waiting DESC;
+--------+--------------+---------------+----------+--------------+-------------------------
```

```
mysql> SHOW FULL TABLES WHERE Table_type = 'VIEW';
+---------------------------+------------+
| Tables_in_disaster_relief | Table_type |
+---------------------------+------------+
| v_dispatch_overview       | VIEW       |
| v_item_stock_summary      | VIEW       |
| v_low_stock               | VIEW       |
| v_open_sos_priority       | VIEW       |
| v_reservation_detail      | VIEW       |
| v_shelter_directory       | VIEW       |
| v_shelter_item_stock      | VIEW       |
| v_shelter_status          | VIEW       |
| v_volunteer_activity      | VIEW       |
+---------------------------+------------+
9 rows in set (0.00 sec)

mysql> SELECT shelter_id, name, capacity, current_occupancy, free_beds, occupancy_pct, status
    -> FROM v_shelter_directory;
+------------+-----------------------------+----------+-------------------+-----------+---------------+--------+
| shelter_id | name                        | capacity | current_occupancy | free_beds | occupancy_pct | status |
+------------+-----------------------------+----------+-------------------+-----------+---------------+--------+
|          1 | Banashankari Community Hall |      150 |                38 |       112 |          25.3 | active |
|          2 | Koramangala Indoor Stadium  |      300 |               134 |       166 |          44.7 | active |
|          3 | Whitefield Government School |      120 |               120 |         0 |         100.0 | full   |
|          4 | Yelahanka Relief Centre     |      200 |                63 |       137 |          31.5 | active |
|          5 | Jayanagar Municipal Hall    |       80 |                12 |        68 |          15.0 | active |
+------------+-----------------------------+----------+-------------------+-----------+---------------+--------+
5 rows in set (0.00 sec)
```

*Figure 9.6: All views in the database and the v_shelter_directory view*

## 9.2 Transactions

A stock transfer must take stock out of one shelter and add it to another, or do nothing at all. sp_transfer_stock does both updates in one transaction, and if the source does not have enough stock the CHECK constraint fails and everything is rolled back.

```
mysql> CALL sp_transfer_stock(1, 4, 5, 100);          -- 100 L water: Yelahanka -> Jayanagar
+----------+
| stock_id |
+----------+
|       16 |
|       22 |
+----------+
2 rows in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)
```

*Figure 9.7: Atomic stock transfer through sp_transfer_stock*

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>     UPDATE STOCK SET quantity_available = quantity_available - 10
    ->      WHERE item_id = 2 AND shelter_id = 1;       -- rice leaves shelter 1
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>     SAVEPOINT after_first_move;
Query OK, 0 rows affected (0.00 sec)

mysql>     UPDATE STOCK SET quantity_available = quantity_available - 50
    ->      WHERE item_id = 2 AND shelter_id = 4;       -- mistake: wrong shelter
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>     ROLLBACK TO SAVEPOINT after_first_move;      -- undo only the mistake
Query OK, 0 rows affected (0.00 sec)

mysql>     UPDATE STOCK SET quantity_available = quantity_available + 10
    ->      WHERE item_id = 2 AND shelter_id = 2;       -- rice arrives at shelter 2
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)
```

*Figure 9.8: Manual transaction with SAVEPOINT and ROLLBACK TO SAVEPOINT*

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>     UPDATE SHELTER SET capacity = capacity + 1000;
Query OK, 5 rows affected (0.00 sec)
Rows matched: 5  Changed: 5  Warnings: 0

mysql>     SELECT shelter_id, capacity FROM SHELTER;    -- visible inside the transaction
+------------+----------+
| shelter_id | capacity |
+------------+----------+
|          1 |     1150 |
|          2 |     1300 |
|          3 |     1120 |
|          4 |     1200 |
|          5 |     1080 |
+------------+----------+
5 rows in set (0.00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT shelter_id, capacity FROM SHELTER;        -- original values are back
+------------+----------+
| shelter_id | capacity |
+------------+----------+
|          1 |      150 |
|          2 |      300 |
|          3 |      120 |
|          4 |      200 |
|          5 |       80 |
+------------+----------+
5 rows in set (0.00 sec)
```

*Figure 9.9: Full ROLLBACK demo*

## 9.3 Indexing

EXPLAIN is used to compare a query with and without the composite triage index on VICTIM_SOS (status, urgency_level, reported_at), and to show that the FULLTEXT index is used for keyword search.

```
mysql> SHOW INDEX FROM VICTIM_SOS;
+------------+------------+---------------------+--------------+-------------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table      | Non_unique | Key_name            | Seq_in_index | Column_name       | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+------------+------------+---------------------+--------------+-------------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| victim_sos |          0 | PRIMARY             |            1 | sos_id            | A         |           9 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| victim_sos |          1 | fk_sos_reported_by  |            1 | reported_by       | A         |           4 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| victim_sos |          1 | fk_sos_matched_shelter |         1 | matched_shelter_id | A        |           4 |     NULL |   NULL | YES  | BTREE      |         |               | YES     | NULL       |
| victim_sos |          1 | idx_sos_status      |            1 | status            | A         |           2 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| victim_sos |          1 | idx_sos_triage      |            1 | status            | A         |           2 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| victim_sos |          1 | idx_sos_triage      |            2 | urgency_level     | A         |           5 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| victim_sos |          1 | idx_sos_triage      |            3 | reported_at       | A         |           9 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| victim_sos |          1 | ft_sos_description  |            1 | description       | NULL      |           9 |     NULL |   NULL | YES  | FULLTEXT   |         |               | YES     | NULL       |
+------------+------------+---------------------+--------------+-------------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
8 rows in set (0.06 sec)
```

*Figure 9.10: SHOW INDEX FROM VICTIM_SOS*

```
mysql>
mysql> -- Uses idx_sos_triage (status, urgency_level, reported_at) - no filesort
mysql> EXPLAIN SELECT sos_id, reported_at
    ->          FROM VICTIM_SOS
    ->          WHERE status = 'matched' AND urgency_level = 'critical'
    ->          ORDER BY reported_at;
+----+-------------+------------+------------+------+-------------------------------+----------------+---------+-------------+------+----------+-------------+
| id | select_type | table      | partitions | type | possible_keys                 | key            | key_len | ref         | rows | filtered | Extra       |
+----+-------------+------------+------------+------+-------------------------------+----------------+---------+-------------+------+----------+-------------+
|  1 | SIMPLE      | VICTIM_SOS | NULL       | ref  | idx_sos_status,idx_sos_triage | idx_sos_triage | 164     | const,const |    3 |   100.00 | Using index |
+----+-------------+------------+------------+------+-------------------------------+----------------+---------+-------------+------+----------+-------------+
1 row in set, 1 warning (0.00 sec)

mysql>
mysql> -- Compare with a forced full table scan
mysql> EXPLAIN SELECT sos_id, reported_at
    ->          FROM VICTIM_SOS IGNORE INDEX (idx_sos_triage, idx_sos_status)
    ->          WHERE status = 'matched' AND urgency_level = 'critical'
    ->          ORDER BY reported_at;
+----+-------------+------------+------------+------+---------------+------+---------+------+------+----------+-----------------------------+
| id | select_type | table      | partitions | type | possible_keys | key  | key_len | ref  | rows | filtered | Extra                       |
+----+-------------+------------+------------+------+---------------+------+---------+------+------+----------+-----------------------------+
|  1 | SIMPLE      | VICTIM_SOS | NULL       | ALL  | NULL          | NULL | NULL    | NULL |    9 |    11.11 | Using where; Using filesort |
+----+-------------+------------+------------+------+---------------+------+---------+------+------+----------+-----------------------------+
1 row in set, 1 warning (0.00 sec)

mysql>
mysql> -- Unique composite index (item_id, shelter_id) serves the stock lookups
mysql> EXPLAIN SELECT quantity_available FROM STOCK WHERE item_id = 1 AND shelter_id = 2;
+----+-------------+-------+------------+-------+-----------------------------------+-----------------------+---------+-------------+------+----------+-------+
| id | select_type | table | partitions | type  | possible_keys                     | key                   | key_len | ref         | rows | filtered | Extra |
+----+-------------+-------+------------+-------+-----------------------------------+-----------------------+---------+-------------+------+----------+-------+
|  1 | SIMPLE      | STOCK | NULL       | const | uq_stock_item_shelter,idx_stock_shelter | uq_stock_item_shelter | 8       | const,const |    1 |   100.00 | NULL  |
+----+-------------+-------+------------+-------+-----------------------------------+-----------------------+---------+-------------+------+----------+-------+
1 row in set, 1 warning (0.00 sec)

mysql>
mysql> -- Full-text index in use
mysql> EXPLAIN SELECT sos_id FROM VICTIM_SOS
    ->          WHERE MATCH(description) AGAINST('flood' IN NATURAL LANGUAGE MODE);
+----+-------------+------------+------------+----------+----------------------+----------------------+---------+-------+------+----------+-----------------------------------+
| id | select_type | table      | partitions | type     | possible_keys        | key                  | key_len | ref   | rows | filtered | Extra                             |
+----+-------------+------------+------------+----------+----------------------+----------------------+---------+-------+------+----------+-----------------------------------+
|  1 | SIMPLE      | VICTIM_SOS | NULL       | fulltext | ft_sos_description | ft_sos_description | 0       | const |    1 |   100.00 | Using where; Ft_hints: sorted |
+----+-------------+------------+------------+----------+----------------------+----------------------+---------+-------+------+----------+-----------------------------------+
1 row in set, 1 warning (0.00 sec)

mysql>
```

*Figure 9.11: EXPLAIN with and without the triage index, the unique composite index and the full-text index*

## 9.4 Roles and Privileges

Two database roles, relief_coordinator and field_volunteer, were created in Part 2. For the application, a separate MySQL login was created for each role, and the Next.js server uses a different connection for each. This way the database itself limits what a signed-in user can do:

- **relief_auth:** used only for login; can read APP_USER and SHELTER.
- **relief_coord_app:** has the relief_coordinator role (read, insert, update and execute procedures).
- **relief_vol_app:** has the field_volunteer role: can read views, file an SOS and run only the procedures a volunteer needs. Because procedures run with the definer's rights, a volunteer can match an SOS without being given UPDATE on SHELTER.

```
mysql> CREATE ROLE IF NOT EXISTS 'relief_coordinator', 'field_volunteer';
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> GRANT SELECT, INSERT, UPDATE ON disaster_relief.* TO 'relief_coordinator';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT DELETE ON disaster_relief.STOCK_ALERT TO 'relief_coordinator';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT EXECUTE ON disaster_relief.* TO 'relief_coordinator';
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> GRANT SELECT ON disaster_relief.v_shelter_status TO 'field_volunteer';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT SELECT ON disaster_relief.v_low_stock TO 'field_volunteer';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT SELECT ON disaster_relief.v_open_sos_priority TO 'field_volunteer';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT SELECT, INSERT ON disaster_relief.VICTIM_SOS TO 'field_volunteer';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT EXECUTE ON PROCEDURE disaster_relief.sp_register_sos TO 'field_volunteer';
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> SHOW GRANTS FOR 'field_volunteer';
+-----------------------------------------------------------------------------------------+
| Grants for field_volunteer@%                                                            |
+-----------------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `field_volunteer`@`%`                                             |
| GRANT SELECT ON `disaster_relief`.`v_low_stock` TO `field_volunteer`@`%`                |
| GRANT SELECT ON `disaster_relief`.`v_open_sos_priority` TO `field_volunteer`@`%`        |
| GRANT SELECT ON `disaster_relief`.`v_shelter_status` TO `field_volunteer`@`%`           |
| GRANT SELECT, INSERT ON `disaster_relief`.`victim_sos` TO `field_volunteer`@`%`         |
| GRANT EXECUTE ON PROCEDURE `disaster_relief`.`sp_register_sos` TO `field_volunteer`@`%` |
+-----------------------------------------------------------------------------------------+
6 rows in set (0.00 sec)
```

*Figure 9.12: Roles relief_coordinator and field_volunteer and their grants*

```
mysql> SELECT user, host FROM mysql.user WHERE user LIKE 'relief%';
+--------------------+------+
| user               | host |
+--------------------+------+
| relief_auth        | %    |
| relief_coord_app   | %    |
| relief_coordinator | %    |
| relief_vol_app     | %    |
+--------------------+------+
4 rows in set (0.00 sec)

mysql> SHOW GRANTS FOR 'relief_vol_app'@'%' USING 'field_volunteer';
+-------------------------------------------------------------------------------------------+
| Grants for relief_vol_app@%                                                               |
+-------------------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `relief_vol_app`@`%`                                                |
| GRANT SELECT ON `disaster_relief`.`RESOURCE_ITEM` TO `relief_vol_app`@`%`                 |
| GRANT SELECT, INSERT ON `disaster_relief`.`VICTIM_SOS` TO `relief_vol_app`@`%`            |
| GRANT SELECT ON `disaster_relief`.`v_item_stock_summary` TO `relief_vol_app`@`%`          |
| GRANT SELECT ON `disaster_relief`.`v_low_stock` TO `relief_vol_app`@`%`                   |
| GRANT SELECT ON `disaster_relief`.`v_open_sos_priority` TO `relief_vol_app`@`%`           |
| GRANT SELECT ON `disaster_relief`.`v_reservation_detail` TO `relief_vol_app`@`%`          |
| GRANT SELECT ON `disaster_relief`.`v_shelter_directory` TO `relief_vol_app`@`%`           |
| GRANT SELECT ON `disaster_relief`.`v_shelter_item_stock` TO `relief_vol_app`@`%`          |
| GRANT SELECT ON `disaster_relief`.`v_shelter_status` TO `relief_vol_app`@`%`              |
| GRANT EXECUTE ON PROCEDURE `disaster_relief`.`sp_match_sos` TO `relief_vol_app`@`%`       |
| GRANT EXECUTE ON PROCEDURE `disaster_relief`.`sp_nearest_stock` TO `relief_vol_app`@`%`   |
| GRANT EXECUTE ON PROCEDURE `disaster_relief`.`sp_register_sos` TO `relief_vol_app`@`%`    |
| GRANT EXECUTE ON FUNCTION `disaster_relief`.`fn_distance_km` TO `relief_vol_app`@`%`      |
| GRANT EXECUTE ON FUNCTION `disaster_relief`.`fn_urgency_rank` TO `relief_vol_app`@`%`     |
| GRANT `field_volunteer`@`%` TO `relief_vol_app`@`%`                                       |
+-------------------------------------------------------------------------------------------+
16 rows in set (0.00 sec)

mysql> SHOW GRANTS FOR 'relief_auth'@'%';
+-----------------------------------------------------------------+
| Grants for relief_auth@%                                        |
+-----------------------------------------------------------------+
| GRANT USAGE ON *.* TO `relief_auth`@`%`                         |
| GRANT SELECT ON `disaster_relief`.`APP_USER` TO `relief_auth`@`%` |
| GRANT SELECT ON `disaster_relief`.`SHELTER` TO `relief_auth`@`%`  |
+-----------------------------------------------------------------+
3 rows in set (0.00 sec)
```

*Figure 9.13: Application database users and their privileges*

```
mysql> SELECT CURRENT_USER(), CURRENT_ROLE();
+------------------+----------------------+
| CURRENT_USER()   | CURRENT_ROLE()       |
+------------------+----------------------+
| relief_vol_app@% | `field_volunteer`@`%` |
+------------------+----------------------+
1 row in set (0.00 sec)

mysql> SELECT shelter_name, item_name, quantity_available FROM v_shelter_item_stock LIMIT 3;
+----------------------------+---------------+--------------------+
| shelter_name               | item_name     | quantity_available |
+----------------------------+---------------+--------------------+
| Banashankari Community Hall | Drinking Water |               470 |
| Banashankari Community Hall | Rice           |               190 |
| Banashankari Community Hall | Blankets       |                80 |
+----------------------------+---------------+--------------------+
3 rows in set (0.00 sec)

mysql> DELETE FROM SHELTER WHERE shelter_id = 1;
ERROR 1142 (42000): DELETE command denied to user 'relief_vol_app'@'localhost' for table 'SHELTER'

mysql> UPDATE STOCK SET quantity_available = 9999 WHERE stock_id = 1;
ERROR 1142 (42000): UPDATE command denied to user 'relief_vol_app'@'localhost' for table 'STOCK'
```

*Figure 9.14: Logged in as relief_vol_app: views can be read, but DELETE and UPDATE are denied*

# 10. NOSQL DATABASE – MONGODB

## 10.1 Why MongoDB

An intake form describes a household, and households are very different. One form may have a single person and another may have twenty, each with their own age, injuries, medical needs and mobility. A flood form also asks different questions (water level, roof access, boat needed) than a landslide form (road blocked, houses damaged). In MySQL this would need several extra tables with many empty columns. In MongoDB each SOS has one document that is saved and read in a single operation.

The two databases are linked both ways: VICTIM_SOS.intake_doc_ref in MySQL stores the id of the MongoDB document, and the document stores the sos_id.

## 10.2 Data Model

**Table 10.1: Collections in the database relief_engine**

| Collection | What it stores | Indexes |
|---|---|---|
| intake_forms | One document per SOS: disaster type, GeoJSON location, address, reporter, household[], needs[], disaster-specific details{} and a timeline[] of field notes | unique sos_id, 2dsphere on location, multikey on needs and household.injuries, compound on disaster_type + created_at |
| field_activity | Audit log: time, user, action, entity and extra details | time, user + time, entity |

Household members and field notes are embedded inside the form because they are always read together with it and are never shared with another form.

Problems Output Debug Console Terminal Ports

```
relief_engine> // --- 4. One intake document ---
| printjson(db.intake_forms.findOne({ sos_id: 1 }))
{
  _id: ObjectId('6aeb5390546b1c3031530475'),
  sos_id: 1,
  status: 'linked',
  disaster_type: 'flood',
  location: {
    type: 'Point',
    coordinates: [
      77.62,
      12.933
    ]
  },
  address_text: 'Ejipura main road, near Koramangala',
  reporter: {
    user_id: 3,
    name: 'Meera Iyer'
  },
  household: [
    {
      age: 38,
      gender: 'male',
      injuries: [],
      medical_needs: [],
      is_pregnant: false,
      mobility: 'ok'
    },
    {
      age: 34,
      gender: 'female',
      injuries: [],
      medical_needs: [],
      is_pregnant: false,
      mobility: 'ok'
    },
```

Problems Output Debug Console Terminal Ports

```
    {
      age: 66,
      gender: 'female',
      injuries: [],
      medical_needs: [
        'blood pressure tablets'
      ],
      is_pregnant: false,
      mobility: 'limited'
    },
    {
      age: 7,
      gender: 'male',
      injuries: [],
      medical_needs: [],
      is_pregnant: false,
      mobility: 'ok'
    },
    {
      age: 1,
      gender: 'female',
      injuries: [],
      medical_needs: [
        'fever'
      ],
      is_pregnant: false,
      mobility: 'ok'
    }
  ],
  needs: [
    'baby food',
    'medicines',
    'evacuation',
    'drinking water'
  ],
  details: {
    water_level_ft: 6,
    roof_access: true,
    boat_needed: true
  },
```




*Figure 10.1: One intake document with embedded household members, needs, details and timeline*

## 10.3 Schema Validation and Indexes

Even though MongoDB is schema-flexible, wrong data should still be rejected. The intake_forms collection has a $jsonSchema validator that:

- requires status, disaster_type, location, reporter, household, needs and created_at,
- allows only fixed values for disaster_type (flood, landslide, building_collapse, fire, cyclone, other) and mobility,
- checks that location is a valid GeoJSON point, ages are between 0 and 120 and a household has 1 to 200 members.

```
relief_engine> print('--- 1. Collections ---');
| printjson(db.getCollectionNames());
--- 1. Collections ---
[
  'field_activity',
  'intake_forms'
]

relief_engine> print('--- 2. Schema validation: required fields ---');
| printjson(
|   db.getCollectionInfos({ name: 'intake_forms' })[0].options.validator.$jsonSchema.required
| );
--- 2. Schema validation: required fields ---
[
  'status',
  'disaster_type',
  'location',
  'reporter',
  'household',
  'needs',
  'created_at'
]

relief_engine> print('--- 3. Indexes ---');
| db.intake_forms.getIndexes().forEach(i =>
|   print('  ' + i.name + ': ' + JSON.stringify(i.key) + (i.unique ? ' UNIQUE' : ''))
| );
--- 3. Indexes ---
  _id_: {"_id":1}
  uq_sos_id: {"sos_id":1} UNIQUE
  geo_location: {"location":"2dsphere"}
  mk_needs: {"needs":1}
  type_created: {"disaster_type":1,"created_at":-1}
  mk_injuries: {"household.injuries":1}

relief_engine>
```

*Figure 10.2: Collections, required fields of the validator, and indexes*

```
|
relief_engine> print('--- 5. Schema validation test: invalid document ---');
|
| try {
|   db.intake_forms.insertOne({
|     status: 'draft',
|     disaster_type: 'meteor',
|     location: { type: 'Point', coordinates: [77.6, 12.9] },
|     reporter: { user_id: 1, name: 'Test' },
|     household: [{}],
|     needs: [],
|     created_at: new Date()
|   });
| } catch (e) {
|   print('Document rejected: ' + e.message.split('\n')[0]);
| }
--- 5. Schema validation test: invalid document ---
Document rejected: Document failed validation
```

```
relief_engine> print('--- 6. Multikey query: cases needing medicines ---');
|
| db.intake_forms.find(
|   { needs: 'medicines' },
|   { _id: 0, sos_id: 1, needs: 1 }
| ).forEach(printjson);
--- 6. Multikey query: cases needing medicines ---
{
  sos_id: 1,
  needs: [
    'baby food',
    'medicines',
    'evacuation',
    'drinking water'
  ]
}
{
  sos_id: 2,
  needs: [
    'drinking water',
    'medicines'
  ]
}
{
  sos_id: 4,
  needs: [
    'first aid',
    'ambulance',
    'medicines'
  ]
}
{
  sos_id: 8,
  needs: [
    'evacuation',
    'medicines',
    'first aid'
  ]
}
```

*Figure 10.3: A document with an invalid disaster_type is rejected; a query on needs uses the multikey index*

## 10.4 Aggregation Pipelines

Aggregation pipelines turn the intake forms into useful numbers for the coordinator. These results are shown on the dashboard and the analytics page.

**Table 10.2: Aggregation pipelines**

| Pipeline | Main stages | Question it answers |
|---|---|---|
| Most requested needs | $unwind needs → $group → $sort | Which supplies are needed the most? |
| People by age band | $unwind household → $bucket | How many infants, elderly and injured people are there? |
| Vulnerable people per disaster | $project with $filter → $group | Where are vulnerable people concentrated? |
| Cases near a shelter | $geoNear (5 km) | How many cases are close to this shelter? |
| Activity per day | $dateToString → $group | Who has been active in the field? |

```
db.intake_forms.aggregate([
  { $match:  { status: 'linked' } },
  { $unwind: '$needs' },
  { $group:  { _id: '$needs', cases: { $sum: 1 },
               people: { $sum: { $size: '$household' } } } },
  { $sort:   { cases: -1 } }
])
```

```
relief_engine> // --- 7. Most requested needs ---
| print('--- Most requested needs ---');
|
| db.intake_forms.aggregate([
|   { $match: { status: 'linked' } },
|   { $unwind: '$needs' },
|   { $group: {
|       _id: '$needs',
|       cases: { $sum: 1 },
|       people: { $sum: { $size: '$household' } }
|   }},
|   { $sort: { cases: -1 } }
| ]).forEach(printjson);
--- Most requested needs ---
{
  _id: 'medicines',
  cases: 4,
  people: 14
}
{
  _id: 'drinking water',
  cases: 4,
  people: 19
}
{
  _id: 'first aid',
  cases: 3,
  people: 11
}
{
  _id: 'evacuation',
  cases: 3,
  people: 19
}
{
  _id: 'blankets',
  cases: 2,
  people: 16
}
{
  _id: 'food',
  cases: 2,
  people: 8
}
```

```
{
  _id: 'shelter bed',
  cases: 2,
  people: 14
}
{
  _id: 'hygiene kit',
  cases: 1,
  people: 8
}
{
  _id: 'baby food',
  cases: 1,
  people: 5
}
{
  _id: 'tarpaulin / roof cover',
  cases: 1,
  people: 6
}
{
  _id: 'ambulance',
  cases: 1,
  people: 3
}
```

```
relief_engine> print('--- 8. People by age band ---');
|
| db.intake_forms.aggregate([
|   { $unwind: '$household' },
|   {
|     $bucket: {
|       groupBy: { $ifNull: ['$household.age', -1] },
|       boundaries: [-1, 0, 5, 18, 60, 121],
|       default: 'unknown',
|       output: {
|         people: { $sum: 1 },
|         injured: {
|           $sum: {
|             $cond: [
|               { $gt: [{ $size: { $ifNull: ['$household.injuries', []] } }, 0] },
|               1,
|               0
|             ]
|           }
|         }
|       }
|     }
|   }
| ]).forEach(printjson);
```

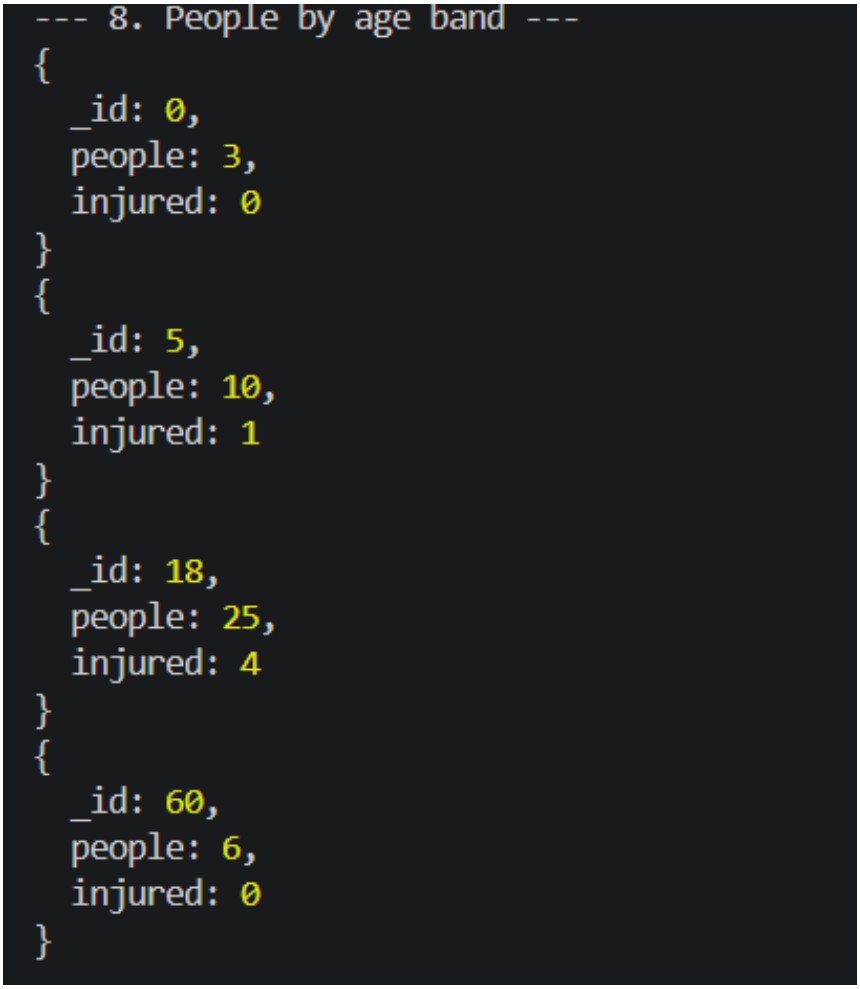



*Figure 10.4: Most requested needs and people by age band*

```
relief_engine> print('--- 9. Vulnerable people per disaster type ---');
|
| db.intake_forms.aggregate([
|   {
|     $project: {
|       disaster_type: 1,
|       members: { $size: '$household' },
|       vulnerable: {
|         $size: {
|           $filter: {
|             input: '$household',
|             as: 'm',
|             cond: {
|               $or: [
|                 { $lt: [{ $ifNull: ['$$m.age', 30] }, 5] },
|                 { $gte: [{ $ifNull: ['$$m.age', 30] }, 60] },
|                 { $eq: ['$$m.is_pregnant', true] },
|                 { $in: ['$$m.mobility', ['limited', 'immobile']] }
|               ]
|             }
|           }
|         }
|       }
|     }
|   },
|   {
|     $group: {
|       _id: '$disaster_type',
|       cases: { $sum: 1 },
|       people: { $sum: '$members' },
|       vulnerable: { $sum: '$vulnerable' }
|     }
|   },
|   { $sort: { cases: -1 } }
| ]).forEach(printjson);
```

```
--- 9. Vulnerable people per disaster type ---
{
  _id: 'flood',
  cases: 6,
  people: 26,
  vulnerable: 9
}
{
  _id: 'other',
  cases: 1,
  people: 8,
  vulnerable: 1
}
{
  _id: 'building_collapse',
  cases: 1,
  people: 4,
  vulnerable: 0
}
{
  _id: 'landslide',
  cases: 1,
  people: 6,
  vulnerable: 1
}

relief_engine>
```

```
relief_engine> print('--- 10. Cases within 5 km of Koramangala Indoor Stadium ---');
|
| db.intake_forms.aggregate([
|   {
|     $geoNear: {
|       near: { type: 'Point', coordinates: [77.6245, 12.9352] },
|       distanceField: 'distance_m',
|       maxDistance: 5000,
|       spherical: true
|     }
|   },
|   {
|     $project: {
|       _id: 0,
|       sos_id: 1,
|       disaster_type: 1,
|       distance_m: { $round: ['$distance_m', 0] },
|       people: { $size: '$household' }
|     }
|   }
| ]).forEach(printjson);
--- 10. Cases within 5 km of Koramangala Indoor Stadium ---
{
  sos_id: 1,
  disaster_type: 'flood',
  distance_m: 546,
  people: 5
}
{
  sos_id: 6,
  disaster_type: 'flood',
  distance_m: 757,
  people: 10
}
{
  sos_id: 5,
  disaster_type: 'flood',
  distance_m: 3788,
  people: 2
}
```

*Figure 10.5: Vulnerable people per disaster type, and cases within 5 km of a shelter ($geoNear)*

# 11. VECTOR DATABASE – POSTGRESQL WITH PGVECTOR

## 11.1 Why Vector Search

SOS messages are short and informal. MySQL full-text search only finds reports that share words with the query, so a search for "kid burning up with temperature" does not find "Child with high fever and leg injury". An embedding model converts a sentence into a list of numbers (a vector) such that sentences with similar meaning get vectors that are close to each other. Comparing these vectors lets the system find reports by meaning instead of by exact words.

pgvector adds a vector column type, distance operators and vector indexes to PostgreSQL. Since the vectors are stored in a normal SQL table, the same query can also filter by status, time and location.

## 11.2 Embedding Design

**Table 11.1: Vector database design**

| Question | Our choice |
|---|---|
| What is converted into embeddings | Each SOS (description + needs + disaster type) and each relief item (name + category + usage notes). The search text typed by the user is embedded at query time. |
| Embedding model | all-MiniLM-L6-v2 sentence transformer, run locally inside the Node.js server using transformers.js. No external API is called, so victim data stays on the server. |
| How vectors are stored | vector(384) column in two tables, sos_embeddings and item_embeddings, along with metadata such as urgency, status, time and location. |
| Similarity metric | Cosine similarity. The <=> operator gives cosine distance, and similarity = 1 − distance. |
| How results are retrieved | k-nearest-neighbour search ordered by <=>, using an HNSW index, with optional filters on status, distance and time. |

## 11.3 Tables and Index

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE sos_embeddings (
  sos_id       INTEGER PRIMARY KEY,        -- same id as MySQL VICTIM_SOS
  embedding    vector(384) NOT NULL,
  urgency      VARCHAR(20) NOT NULL,
  status       VARCHAR(20) NOT NULL,
  people_count INTEGER NOT NULL,
  reported_at  TIMESTAMPTZ NOT NULL,
  lat DOUBLE PRECISION, lon DOUBLE PRECISION,
  source_text  TEXT NOT NULL
);

CREATE INDEX idx_sos_emb_hnsw ON sos_embeddings
  USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64);
```

item_embeddings has the same structure for relief items. HNSW (Hierarchical Navigable Small World) is an approximate nearest-neighbour index, so the search does not need to compare the query with every row.

```
relief_vectors=# SELECT extname, extversion FROM pg_extension;
 extname | extversion
---------+------------
 plpgsql | 1.0
 vector  | 0.8.7
(2 rows)

relief_vectors=# \dt
              List of relations
 Schema |      Name       | Type  |  Owner
--------+-----------------+-------+----------
 public | item_embeddings | table | postgres
 public | sos_embeddings  | table | postgres
(2 rows)

relief_vectors=# \d sos_embeddings
                                      Table "public.sos_embeddings"
    Column     |           Type           | Collation | Nullable |                  Default
---------------+--------------------------+-----------+----------+-------------------------------------------
 sos_id        | integer                  |           | not null |
 embedding     | vector(384)              |           | not null |
 urgency       | character varying(20)    |           | not null |
 status        | character varying(20)    |           | not null |
 people_count  | integer                  |           | not null |
 reported_at   | timestamp with time zone |           | not null |
 lat           | double precision         |           |          |
 lon           | double precision         |           |          |
 disaster_type | character varying(30)    |           |          |
 source_text   | text                     |           | not null |
 model         | character varying(80)    |           | not null | 'Xenova/all-MiniLM-L6-v2'::character varying
 embedded_at   | timestamp with time zone |           | not null | now()
Indexes:
    "sos_embeddings_pkey" PRIMARY KEY, btree (sos_id)
    "idx_sos_emb_hnsw" hnsw (embedding vector_cosine_ops) WITH (m='16', ef_construction='64')
    "idx_sos_emb_status_time" btree (status, reported_at)
    "idx_sos_emb_urgency" btree (urgency)
Check constraints:
    "sos_embeddings_check" CHECK ((lat IS NULL) = (lon IS NULL))
    "sos_embeddings_lat_check" CHECK (lat >= '-90'::integer::double precision AND lat <= 90::double precision)
    "sos_embeddings_lon_check" CHECK (lon >= '-180'::integer::double precision AND lon <= 180::double precision)
    "sos_embeddings_people_count_check" CHECK (people_count > 0)
    "sos_embeddings_status_check" CHECK (status::text = ANY (ARRAY['unmatched'::character varying, 'matched'::character
 varying, 'resolved'::character varying]::text[]))
    "sos_embeddings_urgency_check" CHECK (urgency::text = ANY (ARRAY['low'::character varying, 'medium'::character vary
ing, 'high'::character varying, 'critical'::character varying]::text[]))

relief_vectors=# \d item_embeddings
                                     Table "public.item_embeddings"
   Column    |           Type           | Collation | Nullable |                  Default
-------------+--------------------------+-----------+----------+-------------------------------------------
 item_id     | integer                  |           | not null |
 embedding   | vector(384)              |           | not null |
 item_name   | character varying(120)   |           | not null |
 category    | character varying(50)    |           | not null |
 unit        | character varying(20)    |           | not null |
 source_text | text                     |           | not null |
 model       | character varying(80)    |           | not null | 'Xenova/all-MiniLM-L6-v2'::character varying
 embedded_at | timestamp with time zone |           | not null | now()
Indexes:
    "item_embeddings_pkey" PRIMARY KEY, btree (item_id)
    "idx_item_emb_category" btree (category)
    "idx_item_emb_hnsw" hnsw (embedding vector_cosine_ops) WITH (m='16', ef_construction='64')

relief_vectors=#
```

*Figure 11.1: pgvector extension, the two tables and the structure of sos_embeddings*

## 11.4 How the Application Uses Vector Search

All the search logic is written as SQL functions in PostgreSQL. The application uses them in three places:

- **Semantic search (search_sos):** the search text is embedded and the closest SOS reports are returned, shown next to the MySQL keyword results.
- **Duplicate warning (find_duplicate_sos):** while a new SOS is being filed, the system looks for reports with similarity of at least 0.55 that are also within 3 km and less than 72 hours old.
- **Supply recommendation (recommend_items):** the SOS text and each need are matched against the item embeddings. The MySQL procedure sp_nearest_stock then finds the nearest shelters that have those items in stock.

```
-- five reports closest in meaning to SOS #4
SELECT sos_id,
       1 - (embedding <=> (SELECT embedding FROM sos_embeddings WHERE sos_id = 4)) AS
similarity
FROM sos_embeddings
ORDER BY embedding <=> (SELECT embedding FROM sos_embeddings WHERE sos_id = 4)
LIMIT 5;
```

```
relief_vectors=# SELECT
    sos_id,
    ROUND(similarity::numeric, 3) AS similarity,
    urgency,
    status,
    LEFT(source_text, 50) AS text
FROM similar_sos(4, 5, 0);
 sos_id | similarity | urgency  |  status  |                        text
--------+------------+----------+----------+-----------------------------------------------------
      1 |      0.636 | critical | matched  | Family of five stranded on rooftop as flood water
      8 |      0.636 | high     | matched  | Pregnant woman and three children trapped by flood
      2 |      0.514 | high     | resolved | Elderly couple with limited mobility need drinking
      7 |      0.456 | medium   | resolved | Power outage at hospital annex; sanitary kits need
      9 |      0.455 | high     | matched  | Collapsed wall near market; family of four needs s
(5 rows)

relief_vectors=# -- --- Relief items closest to SOS #4 (cosine distance) ---
SELECT
    item_name,
    category,
    ROUND(
        (1 - (embedding <=> (
            SELECT embedding
            FROM sos_embeddings
            WHERE sos_id = 4
        )))::numeric, 3
    ) AS similarity
FROM item_embeddings
ORDER BY embedding <=> (
    SELECT embedding
    FROM sos_embeddings
    WHERE sos_id = 4
)
LIMIT 4;
      item_name      | category | similarity
---------------------+----------+------------
 First Aid Kit       | Medical  |      0.577
 Paracetamol Tablets | Medical  |      0.417
 Drinking Water      | Water    |      0.333
 Sanitary Kit        | Hygiene  |      0.316
(4 rows)
```

*Figure 11.2: Reports most similar to SOS #4 and the relief items closest to it*

```
relief_vectors=# -- --- 3. Duplicate SOS detection ---
SELECT
    d.sos_id,
    ROUND(d.similarity::numeric, 3) AS similarity,
    ROUND(d.distance_km::numeric, 2) AS km,
    d.urgency,
    d.status
FROM sos_embeddings s,
     find_duplicate_sos(s.embedding, s.lat, s.lon, 3, 1440, 0.55, 5) d
WHERE s.sos_id = 1
  AND d.sos_id <> s.sos_id;
 sos_id | similarity |  km  | urgency  | status
--------+------------+------+----------+---------
      6 |      0.703 | 0.33 | critical | matched
(1 row)

relief_vectors=# -- --- Distance calculation ---
SELECT ROUND(
    km_between(12.9330, 77.6200, 12.9352, 77.6245)::numeric,
    3
) AS km;
  km
-------
 0.546
(1 row)
```

*Figure 11.3: Duplicate check combining vector similarity with distance and time filters*

Because the demo table is small, PostgreSQL would normally just scan it. With sequential scans turned off, EXPLAIN shows that the nearest-neighbour query uses the HNSW index:

```
relief_vectors=# -- --- 4. HNSW index usage ---
SET enable_seqscan = off;
SET
relief_vectors=# EXPLAIN
SELECT sos_id
FROM sos_embeddings
ORDER BY embedding <=> (
    SELECT embedding
    FROM sos_embeddings
    WHERE sos_id = 4
)
LIMIT 5;
                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------------------------
 Limit  (cost=25.36..26.67 rows=5 width=12)
   InitPlan 1 (returns $0)
     ->  Index Scan using sos_embeddings_pkey on sos_embeddings sos_embeddings_1  (cost=0.14..8.16 rows=1 width=32)
           Index Cond: (sos_id = 4)
   ->  Index Scan using idx_sos_emb_hnsw on sos_embeddings  (cost=17.20..59.20 rows=160 width=12)
         Order By: (embedding <=> $0)
(6 rows)
```

*Figure 11.4: EXPLAIN: Index Scan using idx_sos_emb_hnsw*

# 12. FRONT END DEVELOPMENT

## 12.1 Architecture

The user interface is implemented using Next.js and React. The Next.js Route Handlers provide the API layer for authentication, SOS management, shelter operations, inventory, dispatch and semantic search. The application connects to MySQL for structured data, MongoDB for household intake forms and PostgreSQL with pgvector for semantic matching. Environment variables are used for database configuration and application security.

## 12.2 User Roles

**Table 12.1: Application roles**

| Role | Can do | MySQL account |
|---|---|---|
| Field Volunteer | Dashboard, file and match SOS, semantic search, view shelters, add field notes | relief_vol_app |
| Relief Coordinator | Everything a volunteer can do, plus reservations, dispatch, inventory, resolving SOS, analytics and team management | relief_coord_app |

## 12.3 Security

- Passwords are stored only as bcrypt hashes, and repeated failed logins from the same IP are limited.
- The login session is a signed JWT in an httpOnly cookie that expires after 8 hours.
- Database passwords and the session secret are read from environment variables (.env.local), never written in the code.
- All queries are parameterised, which prevents SQL injection.

## 12.4 Main API Routes

**Table 12.2: Main API routes**

| Route | Role | Database work |
|---|---|---|
| /api/auth/login | public | Reads APP_USER, compares bcrypt hash, sets the session cookie |
| /api/sos | all | List with filters and FULLTEXT search; new SOS saved in MySQL, MongoDB and pgvector |
| /api/sos/duplicates | all | pgvector find_duplicate_sos() |
| /api/sos/[id]/match | all | CALL sp_match_sos |
| /api/sos/[id]/insights | all | pgvector recommend_items() and similar_sos() + MySQL sp_nearest_stock |
| /api/search | all | pgvector search_sos() and MySQL MATCH ... AGAINST |
| /api/reservations | coordinator | sp_reserve_supply, sp_cancel_reservation |
| /api/dispatch | coordinator | sp_approve_dispatch, sp_complete_delivery |
| /api/inventory/* | coordinator | Views, sp_restock, sp_transfer_stock |
| /api/analytics | coordinator | SQL reports and MongoDB aggregation pipelines |

## 12.5 Application Screens

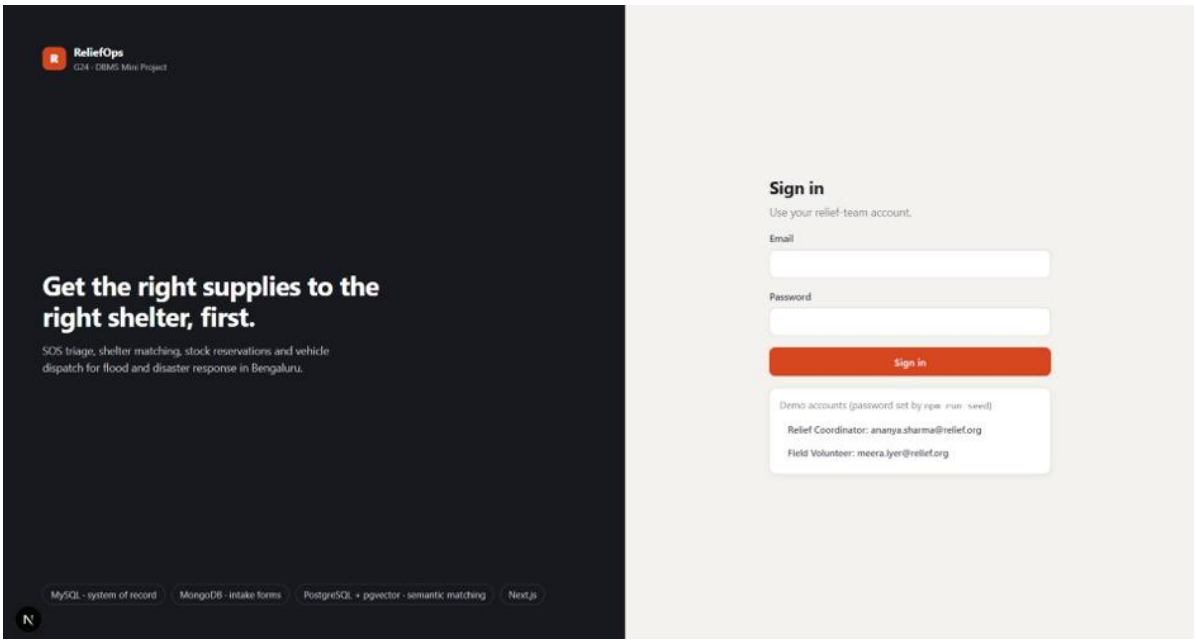



*Figure 12.1: Login page*

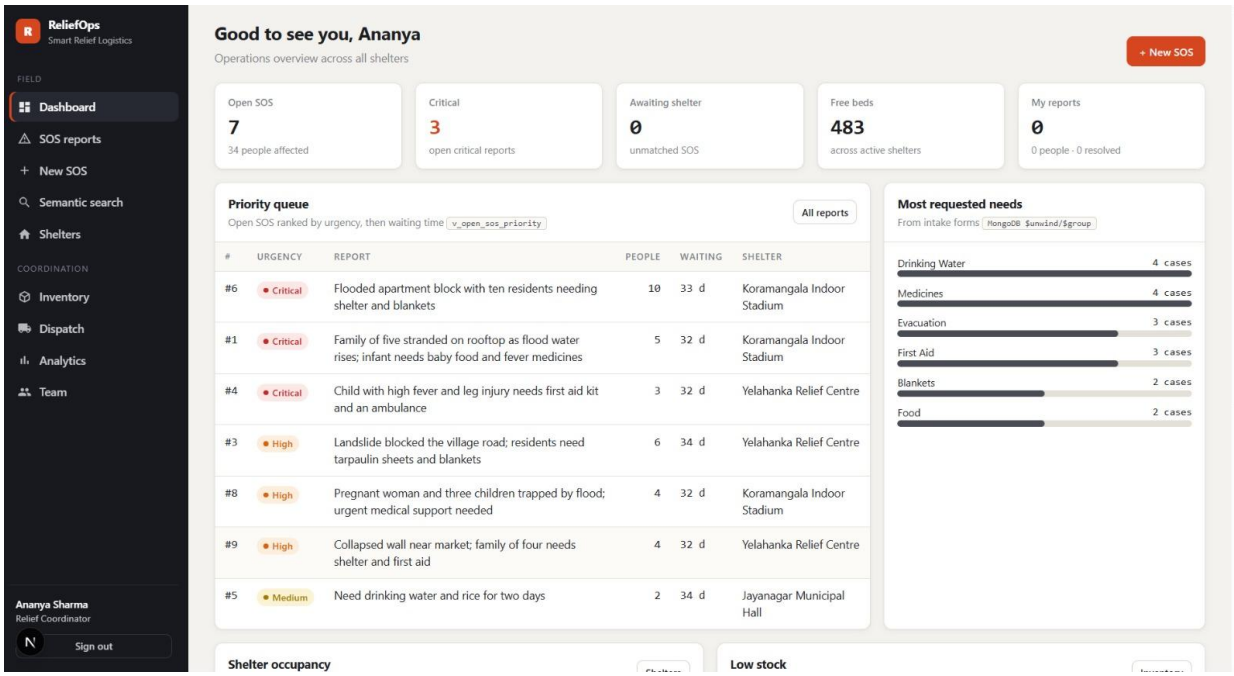



*Figure 12.2: Coordinator dashboard: summary cards, priority queue (from v_open_sos_priority) and most requested needs (from MongoDB)*

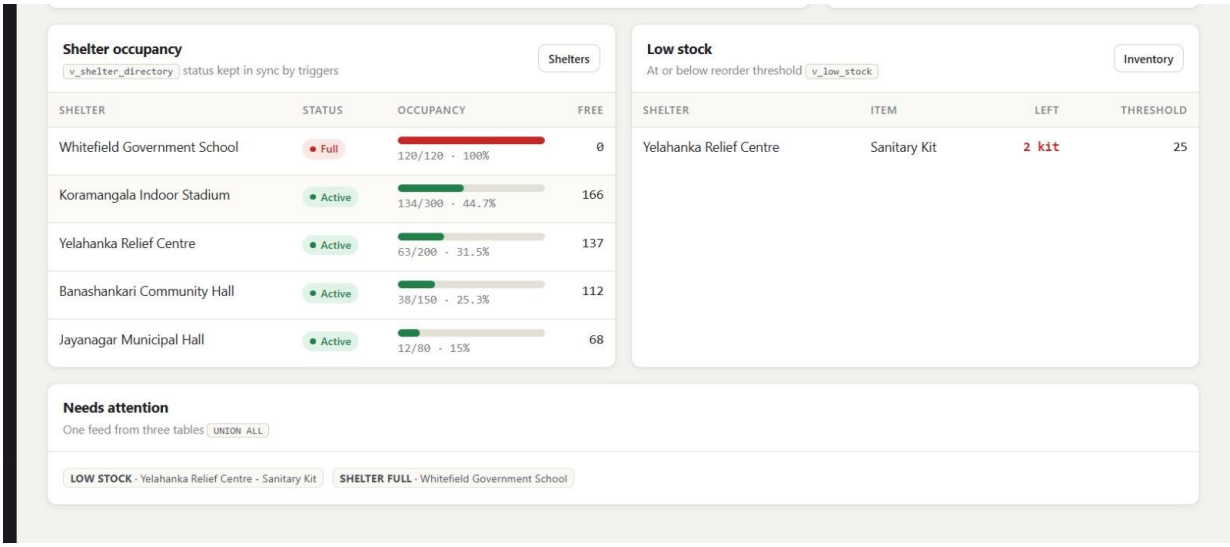



*Figure 12.3: Dashboard: shelter occupancy kept in sync by triggers, low stock, and the "needs attention" feed (UNION ALL)*

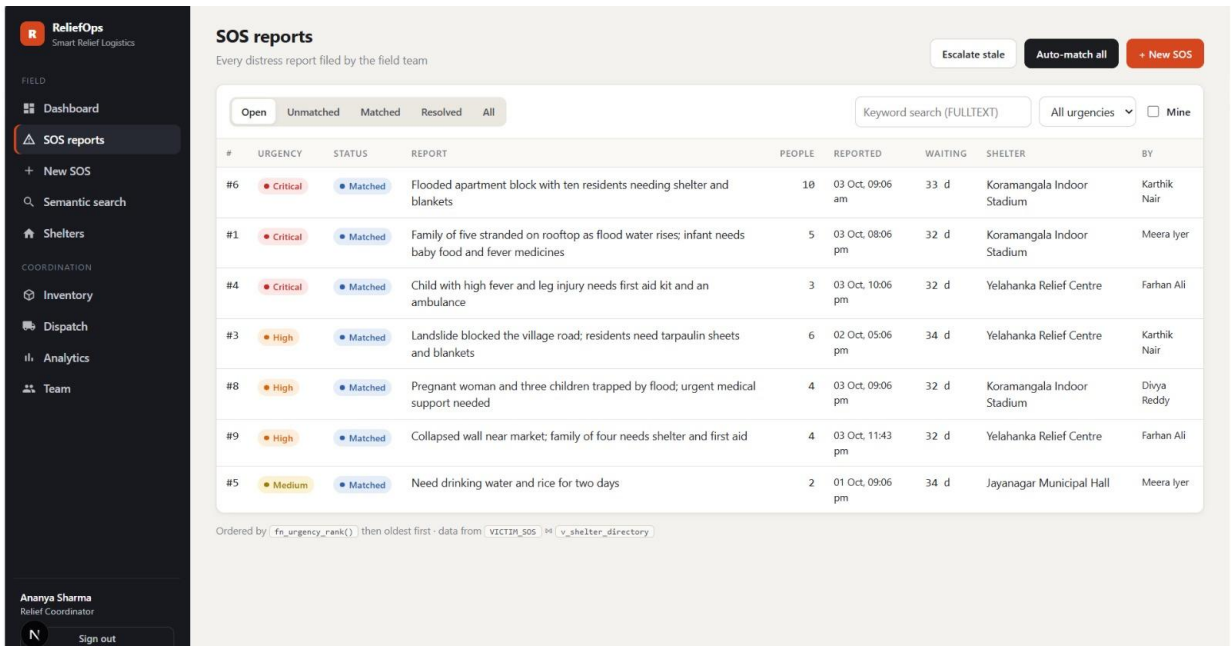



*Figure 12.4: SOS reports with status tabs, urgency filter and keyword (FULLTEXT) search*

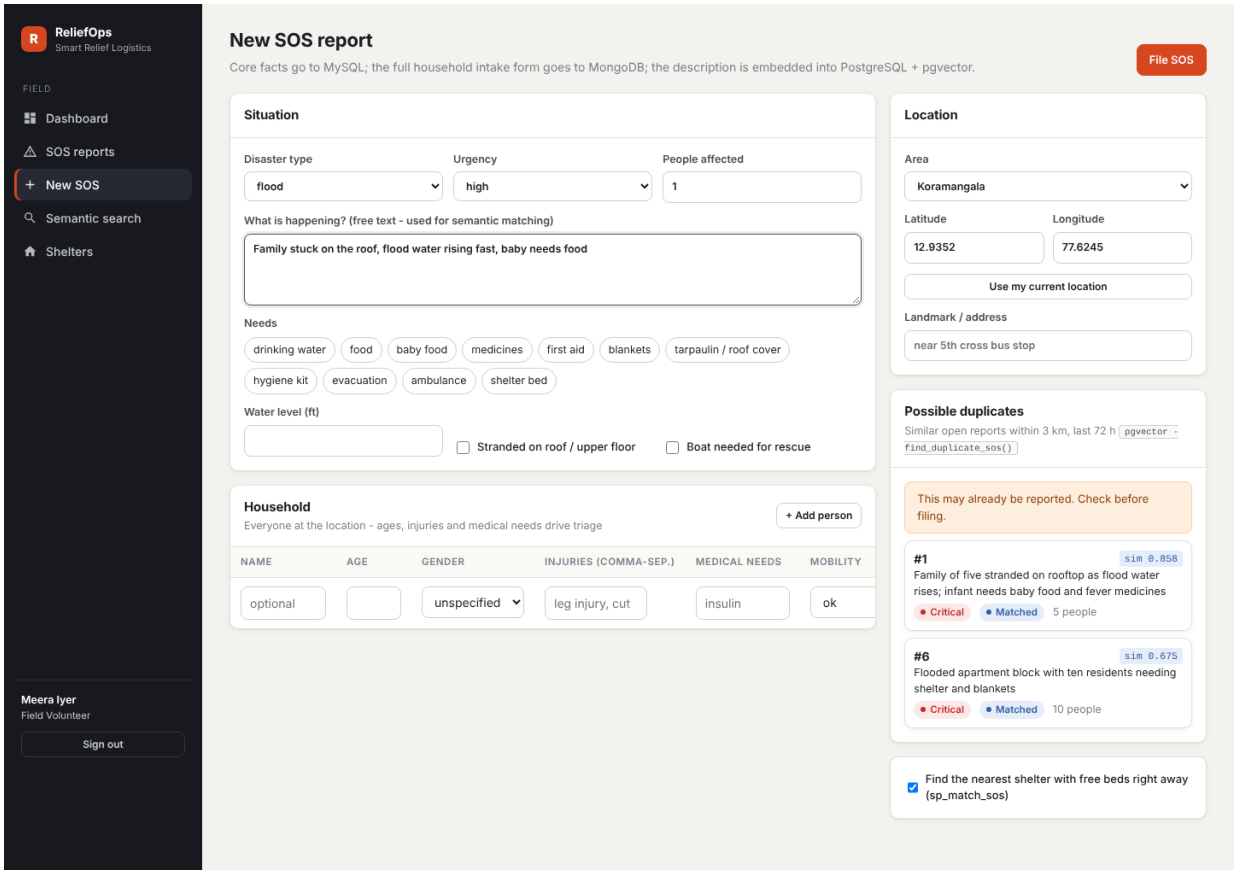



*Figure 12.5: New SOS form with the duplicate-report warning from pgvector*

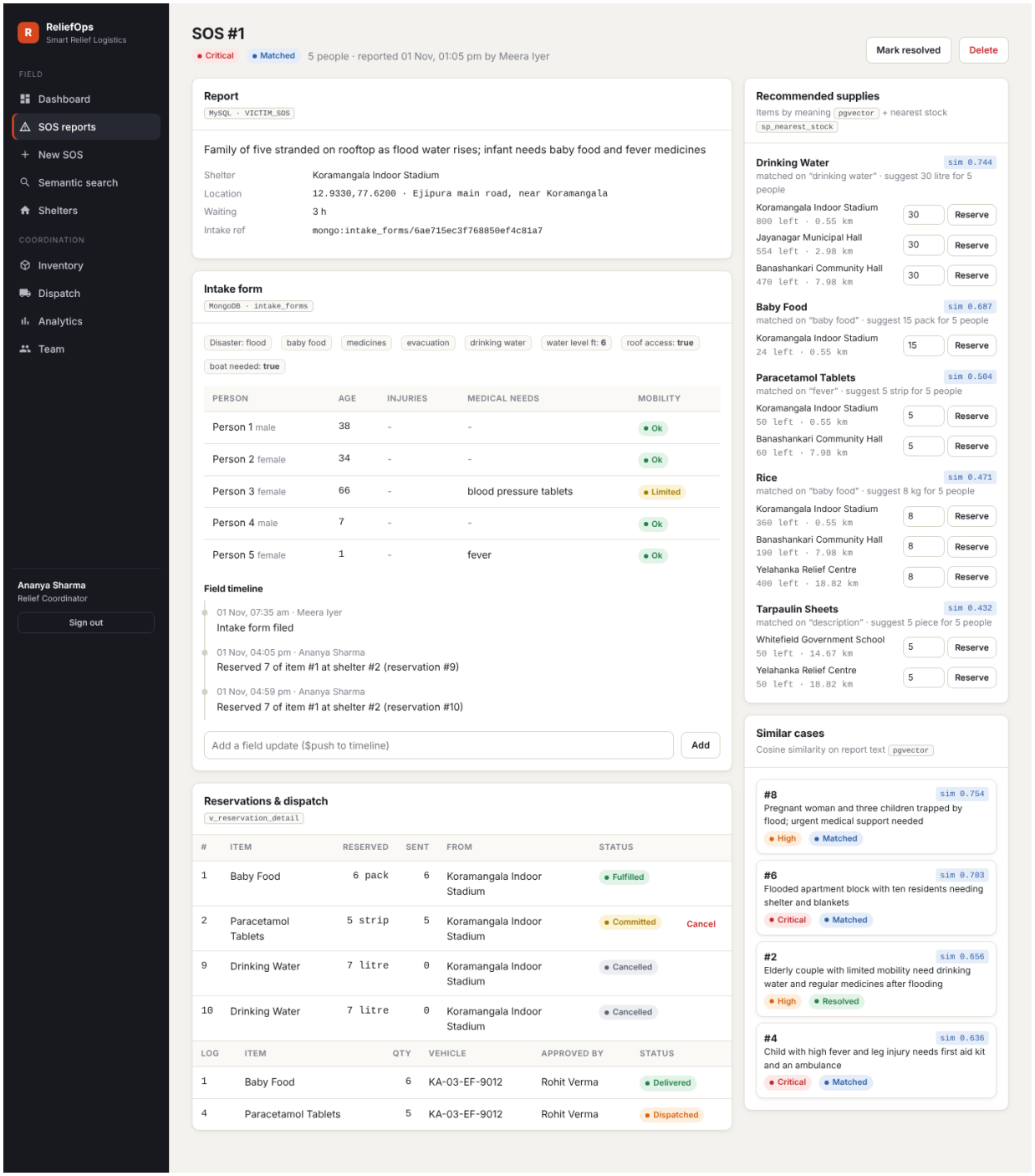



*Figure 12.6: SOS detail: intake form from MongoDB, recommended supplies and similar cases*

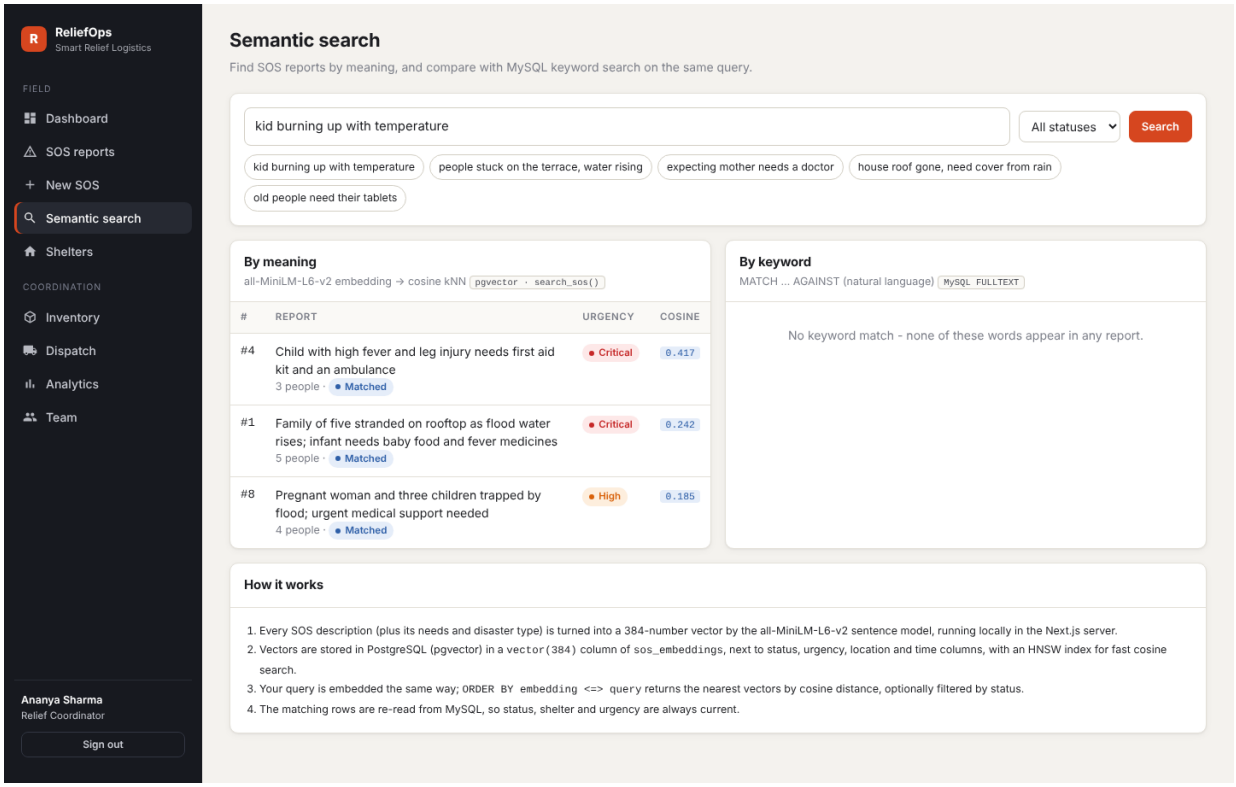



*Figure 12.7: Semantic search finds the fever case, while keyword search finds nothing*

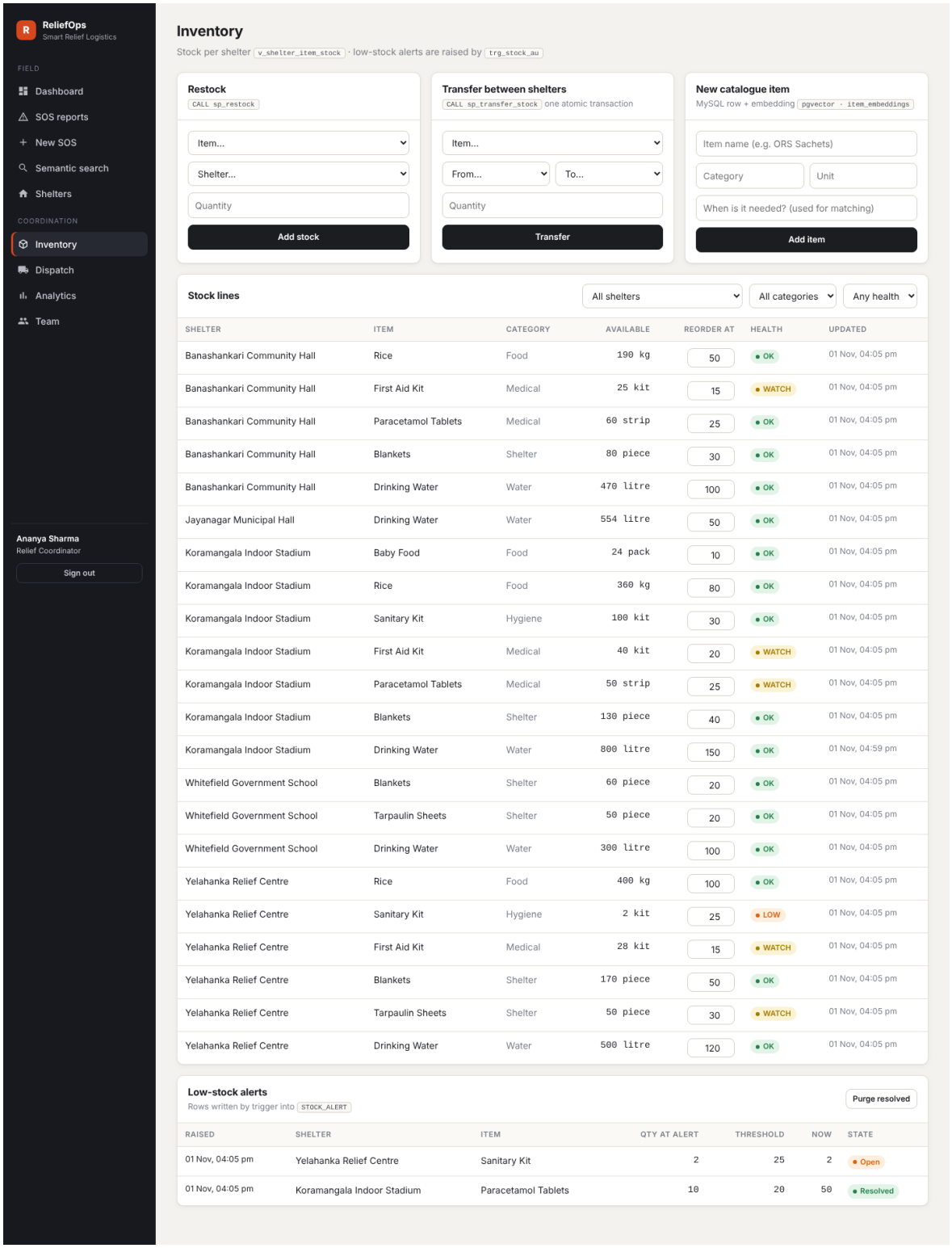



*Figure 12.8: Inventory: restock, transfer between shelters, new item and low-stock alerts*

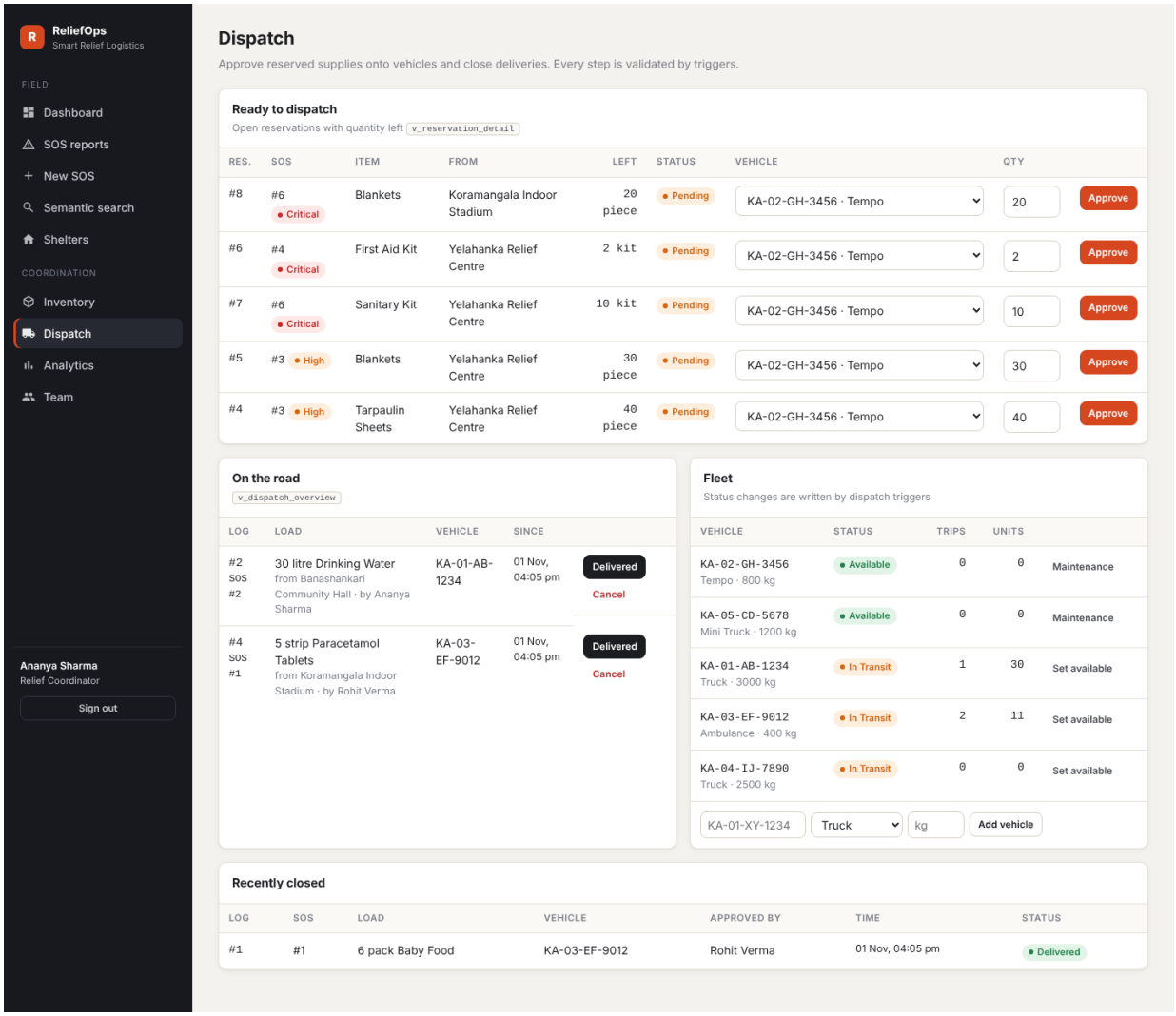



*Figure 12.9: Dispatch board: supplies ready to dispatch, vehicles on the road and history*

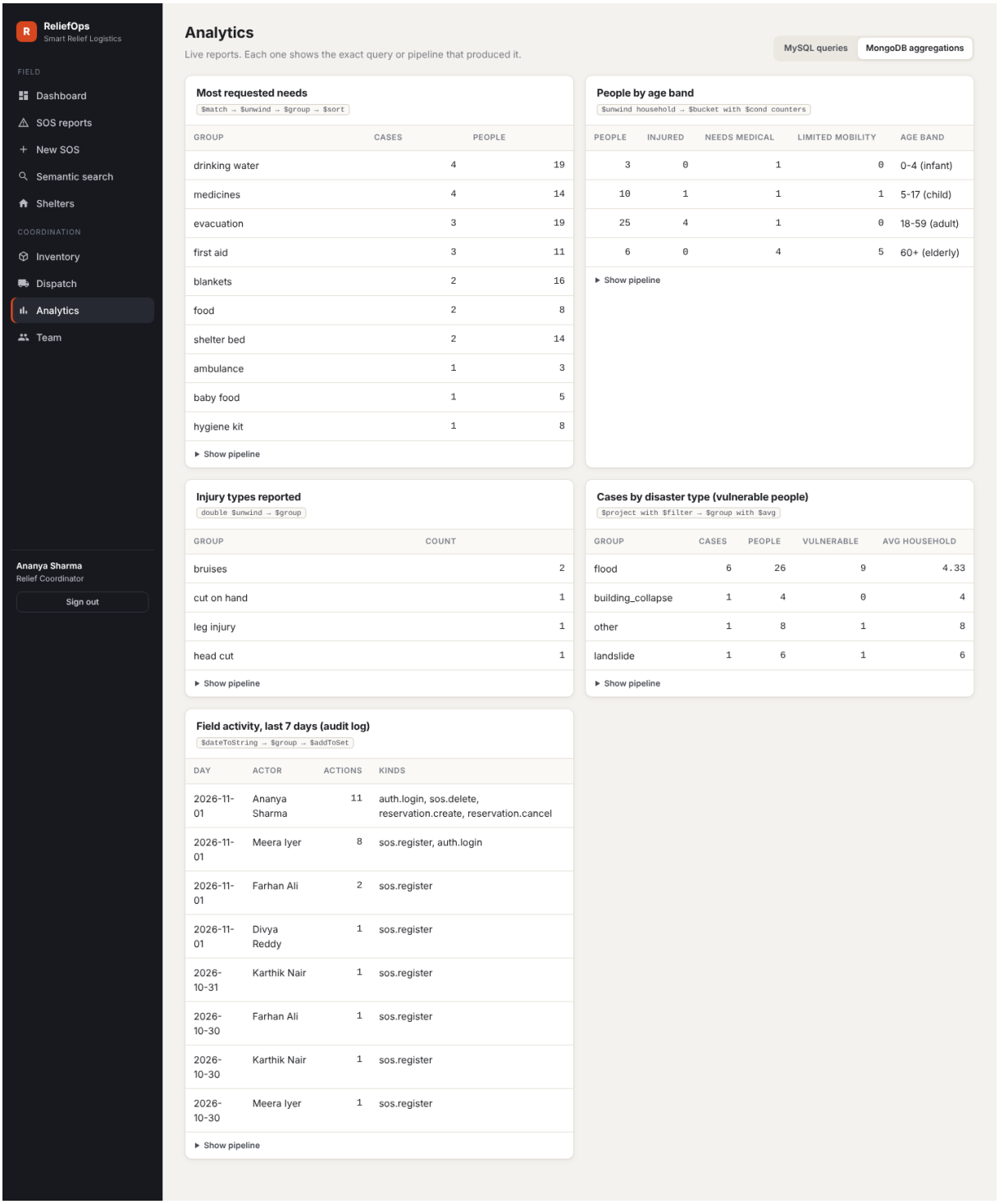



*Figure 12.10: Analytics page showing MongoDB aggregation results*

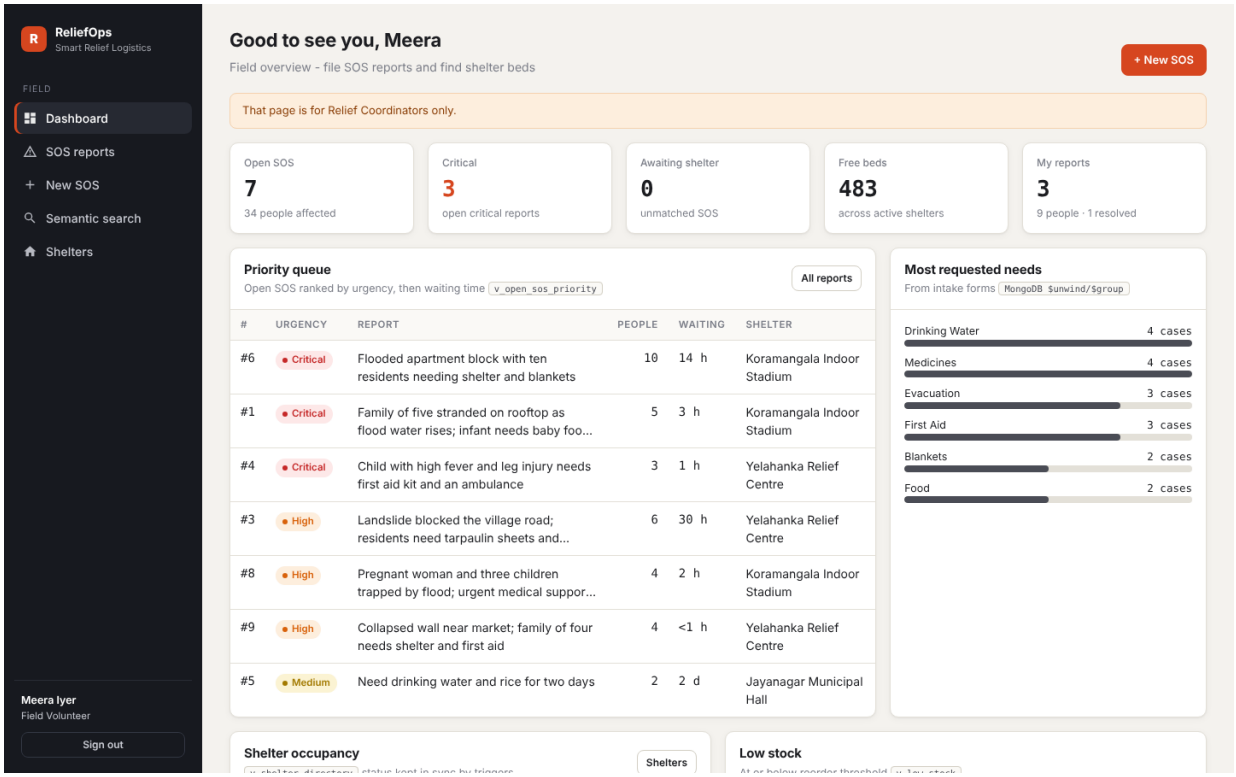



*Figure 12.11: A volunteer trying to open a coordinator page is blocked*

# 13. TESTING AND RESULTS

22 automated integration tests (tests/integration.test.mjs) were run against the running application and all three databases. They check both the application rules and that the databases themselves reject invalid operations.

**Table 13.1: Test cases and results**

| Test | What is checked | Result |
|---|---|---|
| TC01 | Wrong password is rejected | Pass |
| TC02 | API call without login is rejected | Pass |
| TC03 | Volunteer calling a coordinator API gets 403 | Pass |
| TC04 | Volunteer MySQL account cannot UPDATE STOCK | Pass |
| TC05 | Login account can only read APP_USER and SHELTER | Pass |
| TC06 | All passwords are stored as bcrypt hashes | Pass |
| TC07 | Invalid email is rejected by a CHECK constraint | Pass |
| TC08 | Occupancy above capacity is rejected by a CHECK constraint | Pass |
| TC09 | MongoDB rejects an invalid disaster_type | Pass |
| TC10 | Reserving more than the available stock is rejected (trigger) | Pass |
| TC11 | A reservation deducts stock and cancelling returns it (triggers) | Pass |
| TC12 | A volunteer cannot approve a dispatch (trigger) | Pass |
| TC13 | A failed transfer leaves both stock rows unchanged (rollback) | Pass |
| TC14 | An SOS with active reservations cannot be deleted (trigger) | Pass |
| TC15 | A new SOS is saved in all three databases, and delete removes it from all three | Pass |
| TC16 | Invalid SOS input is rejected before reaching the database | Pass |
| TC17 | Semantic search finds a report that keyword search cannot | Pass |
| TC18 | A similar report within 3 km is flagged as a duplicate | Pass |
| TC19 | Recommendations combine vector search with SQL stock lookup | Pass |
| TC20 | Analytics returns SQL rows and MongoDB groups | Pass |
| TC21 | The kNN query uses the HNSW index | Pass |
| TC22 | The vector app account cannot drop tables | Pass |

```
PS C:\Users\Shreya C Rao\OneDrive\Desktop\G24_relief-engine> npm.cmd test

> relief-logistics-engine@1.0.0 test
> node --test --test-concurrency=1 tests/integration.test.mjs

✓ TC01 wrong password is rejected (1174.9253ms)
✓ TC02 API refuses requests without a session (19.3184ms)
✓ TC03 volunteer cannot call coordinator APIs (app-level RBAC) (708.5424ms)
✓ TC04 volunteer MySQL account cannot UPDATE STOCK (DB-level RBAC) (26.2718ms)
✓ TC05 auth MySQL account can only read APP_USER/SHELTER (4.2892ms)
✓ TC06 passwords are stored as bcrypt hashes (13.0866ms)
✓ TC07 CHECK constraint rejects an invalid email (2.4631ms)
✓ TC08 CHECK constraint rejects occupancy above capacity (3.4357ms)
✓ TC09 MongoDB $jsonSchema rejects an invalid intake form (16.0428ms)
✓ TC10 trigger blocks over-reservation (486.3071ms)
✓ TC11 reservation deducts stock; cancelling returns it (triggers) (2074.9104ms)
✓ TC12 trigger: only a Relief Coordinator can approve a dispatch (16.3842ms)
✓ TC13 failed transfer rolls back the whole transaction (757.4407ms)
✓ TC14 SOS with active reservations cannot be deleted (trigger) (1995.3943ms)
✓ TC15 new SOS is written to MySQL, MongoDB and pgvector, and delete cleans all three (1263.1828ms)
✓ TC16 invalid SOS input is rejected before touching the database (61.0177ms)
✓ TC17 semantic search finds a report that keyword search cannot (601.204ms)
✓ TC18 duplicate detection flags a nearby similar open report (770.2016ms)
✓ TC19 recommendations combine vector relevance with SQL stock lookup (2010.1409ms)
✓ TC20 MongoDB aggregations return grouped results (733.6905ms)
✓ TC21 pgvector uses the HNSW index for unfiltered kNN (98.2403ms)
✓ TC22 vector app role cannot drop tables (least privilege) (3.3194ms)
ℹ tests 22
ℹ suites 0
ℹ pass 22
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 13456.0197
PS C:\Users\Shreya C Rao\OneDrive\Desktop\G24_relief-engine>
```

*Figure 13.1: npm test: all 22 integration tests pass*

## 13.1 Result Discussion

The tests show that the important rules are enforced by the databases and not only by the interface. Invalid data is stopped by CHECK constraints and the MongoDB validator, over-reservation and unauthorised dispatch are stopped by triggers, a failed transfer rolls back completely, and each role is limited by its own database account. For the query "kid burning up with temperature", semantic search returned the report of a child with high fever (cosine similarity 0.417), while MySQL full-text search returned no rows.

## 13.2 Limitations

- The sample data is small (5 shelters, 9 SOS), so the similarity scores are only a demonstration.
- Distances are straight-line distances, not road distances.

- The embedding model works only for English text.

# 14. CONCLUSION AND FUTURE ENHANCEMENTS

## 14.1 Conclusion

ReliefOps keeps the rules of relief logistics inside the database. Stock cannot go negative, cannot be reserved twice and cannot be dispatched by the wrong person, and these rules hold even if the application is bypassed. Each type of data is stored where it fits best: transactional data in MySQL, variable household data in MongoDB, and the meaning of free-text reports in PostgreSQL with pgvector. The Next.js application connects all three with role-based access enforced both in the application and in the databases.

## 14.2 Future Enhancements

- Accept SOS reports through SMS or WhatsApp, and support Kannada and Hindi with a multilingual embedding model.
- Use road-network routing instead of straight-line distance, and plan vehicle loads using capacity.
- Keep MongoDB and pgvector in sync automatically using change-data-capture.
- Add a map view of SOS and shelters, and an offline mobile app for volunteers.
- Add a donation and procurement module that feeds into restocking.

# REFERENCES

[1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed., Pearson, 2016.

[2] MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

[3] MongoDB Manual: Schema Validation, Indexes and Aggregation. https://www.mongodb.com/docs/manual/

[4] pgvector: Open-source vector similarity search for Postgres. https://github.com/pgvector/pgvector

[5] Next.js Documentation. https://nextjs.org/docs

[6] Transformers.js Documentation, Hugging Face. https://huggingface.co/docs/transformers.js

[7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks", EMNLP 2019.

[8] DBMS Experiential Learning Level 3 – Mini Project Requirements, PES University.

**GitHub Link:** https://github.com/shriyacrao/G24-relief-engine.git

# APPENDIX A: DEFINITIONS, ACRONYMS AND ABBREVIATIONS

**Table A.1: Definitions and abbreviations**

| Term | Meaning |
|---|---|
| ACID | Atomicity, Consistency, Isolation, Durability: properties of a transaction |
| API | Application Programming Interface |
| bcrypt | Salted, slow password-hashing function |
| Cosine similarity | How similar two vectors are based on the angle between them (1 = same direction) |
| CTE | Common Table Expression (WITH clause) |
| DDL / DML | Data Definition Language / Data Manipulation Language |
| Embedding | A list of numbers that represents the meaning of a piece of text |
| ER | Entity–Relationship model |
| HNSW | Hierarchical Navigable Small World: index for fast nearest-neighbour search |
| JWT | JSON Web Token, used for the login session |
| kNN | k nearest neighbours |
| pgvector | PostgreSQL extension that adds a vector type and similarity search |
| RBAC | Role-Based Access Control |
| SOS | A distress report about people who need help |